Contents

Notes on Cosmological Simulations **1**
Introduction . . . 1
The Anatomy of a Cosmological Simulation . . . 2
The Major Cosmological Codes . . . 2
The N-Body Problem . . . 3
Direct Summation . . . 3
Boundaries and Singularities . . . 3
Periodic Boundary Conditions . . . 4
Gravitational Softening . . . 4
The Integrator . . . 4
The Euler Method . . . 4
Velocity Verlet . . . 5
Symplectic Integration . . . 5
The Kick-Drift-Kick Integrator . . . 6
The Particle-Mesh Method . . . 6
Step 1. Finding the Potential . . . 7
Step 2. From Potential to Force . . . 10
Advanced Interpolation . . . 11
The Flaws of Nearest Grid Point (NGP) . . . 11
Cloud-in-Cell (CIC) . . . 11
Implementation: "Splatting" Mass and "Gathering" Forces . . . 12
Deconvolving the Mass Assignment . . . 13
The P^{3}M Algorithm . . . 14
Combining PP for Short-Range and PM for Long-Range . . . 14
The Subtractive Scheme . . . 14
Calculating the Mesh-Force Correction . . . 15
Choosing the Cutoff Radius (r_c) . . . 15
The Switching Function . . . 15
Fourier-Split Gravity . . . 16
The Flaws of the Subtractive Scheme . . . 16
Fixing the Grid in Frequency Space . . . 17
The Exact Analytical Complement . . . 17
An Expanding Space . . . 18
The Hubble Flow . . . 18
Comoving Coordinates . . . 18
The Equations of Motion . . . 19
Cosmological Models and the Friedmann Equations . . . 19
An Einstein-de Sitter Universe . . . 21
The ΛCDM Model . . . 21
General Relativity and Cosmic Expansion . . . 22
Natural Units . . . 25
Deriving the Gravitational Constant (G) . . . 25
Hubble: Physical vs. Code Units . . . 26
Translating Natural Units to Physical Reality . . . 27
The Cosmic Timeline . . . 28
The Scale Factor (a) and Redshift (z) . . . 28
Eras and epochs . . . 28
Initial Conditions . . . 30

The Unperturbed State . . . 30
The Zel'dovich Approximation . . . 30
Gravitational softening . . . 35
Determining the Optimal Softening Length . . . 35
The Physical Softening Cap . . . 36
Gravity Validation and Accuracy . . . 36
Conservation of Energy and Momentum . . . 36
The Two-Body Problem and Kepler's Laws . . . 38
Sources of Error . . . 39
Hydrodynamics . . . 39
The Hybrid Simulation Approach . . . 40
The Euler Equations . . . 40
An Operator-Split Hydro-Solver . . . 41
Coupling Hydrodynamics to the Expanding Universe . . . 47
Sub-Grid Coupling: The Intra-Cell Blind Spot . . . 47
Initial Conditions for the Gaseous Component . . . 49
Gas Physics . . . 50
Temperature . . . 50
Gravitational Compression and Shocks . . . 51
Radiative Cooling . . . 51
Stiff Equations . . . 52
Implicit Integration . . . 53
Coupling Cooling to the Simulation . . . 54
The Optically Thin Approximation . . . 56
The Subgrid Clumping Factor . . . 56
Validation of the Hydrodynamic Solver . . . 58
Conservation in a Closed Box . . . 58
The 1D Shock-Tube . . . 59
The Sedov-Taylor Blast Wave (Point Explosion) . . . 59
The Kelvin-Helmholtz Instability . . . 59
Adaptive Timestep . . . 59
The Courant-Friedrichs-Lewy (CFL) Condition for hydrodynamics . . . 60
The Cooling Timestep Limiter . . . 60
The Gravitational Timestep . . . 61
The Global Timestep . . . 61
Subcycled Operator Splitting . . . 61
Adaptive Multirate Operator Splitting . . . 62
Decoupling the Thermodynamics . . . 63
Accuracy and Symplecticity in a Subcycled Architecture . . . 63
Analytical Requirements for Resolution and Volume . . . 64
Cosmic Variance and the Box Size . . . 64
Full-size Representative Simulations . . . 65
Representative Sandbox Simulations . . . 67
The Spherical Collapse Limit . . . 68
Architectures of Cosmological Simulations . . . 69
Large-Volume "Uniform Box" Simulations . . . 69
Zoom-in Simulations . . . 70
Constrained Simulations . . . 70
The Physics Split: Gravity-Only vs. Hydrodynamics . . . 70
Diagnosing Cosmological Simulations . . . 71
Cosmic Expansion History . . . 71
Structure Growth and Linear Theory . . . 71
The 1-Point Density PDF . . . 72
The Matter Power Spectrum . . . 73
Global Gas Energy Inventory . . . 74
Maximum Gas Density Evolution . . . 75
Maximum Temperature Evolution . . . 76
The Temperature-Density Phase Diagram . . . 77
Cold Dense Gas Fraction (The Star Formation Precursor) . . . 77

Notes on Cosmological Simulations

This is a living document—a collection of knowledge that I have gathered while learning about cosmological simulations. It is not a formal text but rather a journal, an attempt to solidify concepts by structuring and explaining them in my own way.

Along the way, I have been developing a proof-of-concept N-body/hydrodynamics simulation, which allowed me to understand algorithms by implementing them, and to appreciate physical principles by seeing their effects in a virtual universe. The explanations in this document are reflected in this practical work.

This is my best effort to present this knowledge in the way that I would have found most helpful at the start of my learning process.

Víctor Álamo
vialamo@gmail.com
https://github.com/vialamo/nbody

Introduction

At its core, a cosmological simulation is a computational time machine. Because astrophysicists cannot physically experiment on stars or galaxies in a laboratory, they use computers to build virtual patches of the cosmos. By seeding a simulation volume with the initial mathematical conditions of the Big Bang and stepping it forward in time using the laws of physics, we can watch 13.8 billion years of cosmic evolution unfold in a matter of days or weeks.

The Origins

The endeavor to simulate the cosmos began in the 1970s. Early pioneers, such as Jim Peebles, ran the very first N-body simulations using just a few hundred particles to study how galaxies might cluster together under gravity. By the 1980s, researchers were running the first 3-dimensional models of Cold Dark Matter (CDM). These early simulations were severely limited by the computers of their time; they treated entire galaxies as single points of mass and modeled only the force of gravity, completely ignoring the complex fluid dynamics of gas.

Modern Applications

Today, cosmological simulations are some of the most intensive computational tasks on Earth. Projects like the Millennium Simulation, Illustris, and EAGLE utilize supercomputing clusters to track billions—and sometimes trillions—of particles.

Nowadays, these virtual universes are used as the primary theoretical laboratories for astrophysics, serving three main purposes:

- **Testing the Standard Model:** We can easily alter the parameters of a simulation—adding more dark energy, changing the mass of dark matter particles, or altering the laws of gravity. By comparing the resulting "mock universe" to the real one we see through telescopes, we can prove or disprove fundamental theories of physics.
- **Calibrating Telescope Data:** Massive modern observatories (like the James Webb Space Telescope and the Euclid satellite) gather an overwhelming amount of data. Simulations provide the theoretical maps required to interpret those observations, helping astronomers distinguish between optical illusions (like redshift-space distortions) and true physical structures.
- **Probing the Unobservable:** While a telescope can only capture a frozen snapshot of a galaxy at one specific moment in its life, a simulation allows us to continuously watch the dynamic, billion-year processes of galaxy mergers, black hole feeding, and cosmic web formation from beginning to end.

The Anatomy of a Cosmological Simulation

Before diving into the specific mathematics and algorithms, it is helpful to outline exactly what we are trying to simulate. A modern cosmological simulation is a carefully coupled system of different physical frameworks designed to mimic the actual composition and behavior of the universe.

To build a realistic virtual patch of the cosmos, a simulation must continuously balance three distinct computational pillars:

- **Collisionless Dark Matter :** Dark matter accounts for roughly 85% of the matter in the universe and dominates its gravitational landscape. Because it does not interact with light or experience thermodynamic pressure, we model it as "collisionless" particles. Computationally, this is handled by an **N-body solver**, which tracks millions (or billions) of discrete, massive particles moving purely under the influence of gravity to form the filaments and halos of the cosmic web.
- **Baryonic Gas:** Normal, visible matter (hydrogen and helium) behaves very differently from dark matter. Gas clouds crash into each other, heat up, form shockwaves, and exert pressure. To simulate this, gravity alone is insufficient; we require a **Hydrodynamics solver**. This typically involves dividing the simulation volume into a 3D grid (an Eulerian approach) to strictly track the conservation of mass, momentum, and energy as the fluid flows through space.
- **The Expanding Background:** Unlike a simulation of a single, static solar system, a cosmological simulation takes place in an expanding universe. The coordinate grid itself stretches over time. Both the N-body particles and the hydrodynamic gas must be subjected to a cosmological background model (driven by the Friedmann equations) to properly account for the dilution of density and the slowing of velocities due to cosmic expansion.

Gravity is the universal language that links the dark matter and the gas together. We will begin by breaking down the foundational engine that drives the formation of all large-scale structure: calculating the gravitational pull of N interacting particles.

The Major Cosmological Codes

The global astrophysical community relies on a handful of massive, highly optimized, open-source software packages to run these supercomputer simulations.

It is helpful to know that almost all modern codes are split by their approach to fluid dynamics: some treat gas as a collection of individual moving particles (**Lagrangian** methods like SPH), while others treat gas as a fluid flowing through a fixed or adaptive 3D grid (**Eulerian** methods like AMR).

Here is a summary of the most prominent cosmological codes used in modern research and the underlying engines that drive them:

Code Name	Gravity Solver	Hydrodynamics Solver	Notable Simulations	Website / Repository
GADGET	TreePM (Tree-based + Particle-Mesh)	SPH (Smoothed Particle Hydrodynamics)	Millennium, EAGLE	MPA Garching - Gadget-4
RAMSES	AMR-coupled Particle-Mesh	AMR (Adaptive Mesh Refinement)	Horizon-AGN	ramses.cnrs.fr / GitHub
ENZO	Particle-Mesh	AMR (Adaptive Mesh Refinement)	Renaissance Simulations	enzo-project.org
AREPO	TreePM	Moving Voronoi Mesh (Unstructured grid)	Illustris, IllustrisTNG	arepo-code.org
SWIFT	Fast Multipole Method (Tree)	Modern SPH / Meshless Finite Mass	Flamingo	swiftsim.com / GitHub

These codes represent decades of collaborative optimization. We will dive into the core mechanics of how gravity and gas interact in the following chapters.

The N-Body Problem

The **N-body problem** is the task of predicting the dynamical evolution of a system composed of N particles that interact through mutual gravitational attraction. Each particle experiences the combined gravitational influence of all others, and because these forces depend on the instantaneous positions of every particle, the motion of any one particle cannot be determined independently.

In Newtonian gravity, the equation of motion for particle i with position vector $\mathbf{x}_i$, velocity $\mathbf{v}_i$, and mass m_i is:

$$m_i \frac{d^2\mathbf{x}_i}{dt^2} = -Gm_i \sum_{\substack{j=1 \\ j\neq i}}^{N} m_j \frac{\mathbf{x}_i - \mathbf{x}_j}{|\mathbf{x}_i - \mathbf{x}_j|^3}$$

This coupled system of $3N$ second-order differential equations has no general analytic solution for $N > 2$, making it one of the foundational challenges of computational astrophysics.

Direct Summation

The most straightforward numerical method for solving the N-body problem is the **direct-summation algorithm**, which explicitly computes the gravitational force on each particle from every other particle. The procedure for a single time step can be described as follows:

1. **Select a particle**, say particle A.
2. **Loop over all other particles** $(B, C, D, \ldots)$.
3. **Compute the pairwise force** on A from each other particle using Newton's law of gravitation:

$$\mathbf{F}_{AB} = -G\frac{m_A m_B}{r_{AB}^2}\hat{\mathbf{r}}_{AB}$$

   where $\mathbf{r}_{AB} = \mathbf{x}_A - \mathbf{x}_B$ and $\hat{\mathbf{r}}_{AB} = \mathbf{r}_{AB}/|\mathbf{r}_{AB}|$.
4. **Sum all pairwise forces** to obtain the total force on particle A:

$$\mathbf{F}_A = \sum_{\substack{B=1 \\ B\neq A}}^{N} \mathbf{F}_{AB}$$

5. **Update** particle A's position and velocity using this total force (via an integration method such as Velocity Verlet).
6. **Repeat** the process for every particle in the system.

Although conceptually simple and physically exact, the direct-summation method is computationally prohibitive for large N. To compute the total force on one particle, we must evaluate $N-1$ pairwise interactions. Doing this for all N particles requires approximately $N(N-1) \approx N^2$ force evaluations per time step.

In computational complexity terms, this corresponds to $O(N^2)$ scaling — meaning that doubling the number of particles multiplies the total computational cost by roughly four. This quadratic growth rapidly becomes intractable: while $N \sim 10^3$ is easily manageable, $N \sim 10^6$ would require on the order of 10^{12} pairwise force evaluations per step.

Because of this steep scaling, the direct method is impractical for cosmological simulations, which often involve millions or billions of particles. To overcome this, we rely on **approximation schemes** — such as the **Particle-Mesh (PM)** and **Particle-Particle Particle-Mesh (P³M)** — that preserve physical accuracy while reducing computational cost from $O(N^2)$ to nearly $O(N \log N)$ or better.

Boundaries and Singularities

Two fundamental problems arise when trying to model gravity in a computer. The first is how to simulate an infinite universe in a finite box, and the second is how to handle the infinite force that occurs when two particles get too close.

Periodic Boundary Conditions

To simulate a small, representative patch of an infinite, uniform universe without having particles react to artificial "walls", simulations employ **periodic boundary conditions**. This method treats the simulation space as a seamless, repeating tile.

A particle exiting one face immediately re-enters from the opposite face. This means that when calculating the force between two particles, the "wrap-around" distance must be considered. We always use the shortest path between the two particles. This is known as the **Minimum Image Convention**, and it ensures that no particle ever feels an artificial "edge of the universe."

Gravitational Softening

Newton's law of gravity, $F \propto 1/r^2$, has a mathematical singularity: as the distance r between two particles approaches zero, the force between them approaches infinity. In a simulation that moves in discrete time steps, these immense forces can cause particles to be catapulted away at unrealistic speeds, compromising the simulation's stability and energy conservation.

To prevent this, we introduce gravitational softening. Instead of treating particles as infinitely small points, this technique treats them as extended, spherical clouds of mass. We modify Newton's law of gravity by introducing a softening length, ϵ (epsilon). The classic representation of this is the Plummer force law:

$$F = \frac{Gm_1m_2r}{(r^2 + \epsilon^2)^{3/2}}$$

When particles are far apart, they feel the normal $1/r^2$ force. However, when their separation becomes comparable to or smaller than ϵ, they begin to pass "inside" each other's density clouds. As r approaches zero, the force is softened and gradually drops to zero. This mimics reaching the center of a mass cloud, where the gravitational pull from all sides cancels out.

A simple rule of thumb is to base the softening length on the **mean inter-particle spacing**, d. For a box with side L and N particles, it's calculated as:

$$d = \frac{L}{N^{1/3}}$$

The softening length is then a small fraction of d, such as $\epsilon = 0.03d$. The gravitational softening will be explained in more detail in a later section.

Key Literature & Further Reading
Bagla, J. S., & Padmanabhan, T. (2004). *Cosmological N-Body Simulations.* arXiv:astro-ph/0411730. Available at https://arxiv.org/pdf/astro-ph/0411730.pdf

The Integrator

The Euler Method

To move the particles through time, we need an "integrator"—an algorithm that takes the current state of a particle (its position and velocity) and predicts its state a small moment later. The most intuitive and straightforward approach is the **Euler method**.

The Euler method assumes that the velocity and acceleration are constant over one small time step, Δt. It calculates the force on the particle at its current position to find its acceleration, and then takes a linear step forward.

The update equations are:

1. **Update Position:** $\mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{v}_n\Delta t$
2. **Update Velocity:** $\mathbf{v}_{n+1} = \mathbf{v}_n + \mathbf{a}_n\Delta t$

While simple, the Euler method's core assumption is almost always wrong. In a gravitational system, the force is constantly changing as a particle moves, but the Euler method is blind to any changes that occur during the step.

This error, while tiny on each step, is **systematic**. It always pushes the energy in the same direction. Over thousands of steps, this causes a simulated planet to slowly spiral outwards, gaining energy with every orbit until it eventually flies away. This failure to conserve energy makes the Euler method unsuitable for any simulation where long-term stability is important.

Velocity Verlet

The failure of the Euler method shows that a more robust integrator is needed—one that accounts for the fact that forces change *during* a time step. An effective solution is an algorithm called **Velocity Verlet**.

The core idea is to use a more accurate, averaged acceleration to update the velocity. Instead of just using the acceleration from the beginning of the step, it uses the average of the accelerations from the beginning and the end of the step.

The algorithm proceeds in three steps:

1. **Calculate the New Position:** First, advance the position using the current velocity and acceleration.
$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + \mathbf{v}(t)\Delta t + \frac{1}{2}\mathbf{a}(t)\Delta t^2$$
2. **Calculate the New Acceleration:** With the new position, calculate the new force vector $\mathbf{F}(\mathbf{x}(t + \Delta t))$ and from it, the new acceleration.
$$\mathbf{a}(t + \Delta t) = \frac{\mathbf{F}(\mathbf{x}(t + \Delta t))}{m}$$
3. **Calculate the New Velocity:** Finally, update the velocity using the **average** of the old acceleration $\mathbf{a}(t)$ and the new acceleration $\mathbf{a}(t + \Delta t)$.
$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t) + \frac{\mathbf{a}(t) + \mathbf{a}(t + \Delta t)}{2}\Delta t$$

This final step of averaging the accelerations is the key. It corrects for the systematic drift of the Euler method, and it is what makes Velocity Verlet a **symplectic integrator**. This crucial property is what enables the algorithm to produce a stable and accurate trajectory, conserving energy remarkably well over long periods.

Symplectic Integration

A **symplectic integrator** is an algorithm specifically designed to respect the underlying geometry of physics, a property that allows it to conserve a system's total energy over very long periods. The practical importance of this is best understood by comparing how different integrators handle a simple gravitational problem, like a planet orbiting a star.

- A **non-symplectic** integrator, like the Euler method, consistently makes an error in the same direction. It always "cuts the corner" of the orbit, pushing the planet slightly outwards. These errors add up, causing the planet's energy to systematically increase and its orbit to spiral away.
- A **symplectic** integrator, like Verlet, makes errors that are correlated. On one step, it might slightly overshoot the true orbit, but on a later step, it will undershoot it to compensate. The errors effectively cancel each other out over time.

Instead of a catastrophic spiral, the simulated planet executes a stable "wobble" along the correct orbital path. The shape of the orbit might oscillate slightly, but its average size and, crucially, its average energy, remain correct for millions of steps.

The deeper reason for this remarkable stability lies in a concept from classical mechanics called **phase space**. Phase space is an abstract map where every point represents the complete state of a particle—both its **position** and its **momentum**. For a system where energy is conserved, a fundamental rule known as **Liouville's Theorem** states that the "area" (or volume) of any group of states in phase space must stay constant as the system evolves.

Symplectic integrators are mathematically constructed to **perfectly preserve this phase space volume**. Because they respect this fundamental geometric rule, they are forbidden from having the systematic energy drift that plagues simpler methods. The bounded energy error (the "wobble") is a direct consequence of this property.

This is why symplectic integrators are chosen over the Euler method for any long-term simulation of a conservative system.

The Kick-Drift-Kick Integrator

The standard Velocity Verlet integrator is a powerful tool, but it was designed for a universe with static rules. The introduction of cosmic expansion adds a velocity-dependent term to the equations of motion (the "Hubble drag", explored in a later section). This new term creates a challenge for the standard Verlet algorithm because its symplectic nature is strictly defined for forces that depend only on position, not velocity. To handle this new term gracefully, we adopt a different formulation known as a **Leapfrog** scheme. The most common implementation, the **Kick-Drift-Kick (KDK)** integrator, is the standard choice in cosmological simulations.

The KDK scheme advances the system from a time t to $t + \Delta t$ in three stages:

1. First "Kick" (Velocity Half-Step)

The velocities are "kicked" forward by half a time step using the acceleration from the beginning of the step.

$$\mathbf{v}(t + \tfrac{1}{2}\Delta t) = \mathbf{v}(t) + \mathbf{a}(t)\frac{\Delta t}{2}$$

2. "Drift" (Position Full-Step)

The positions then "drift" for a full time step using the more accurate **mid-step velocity**.

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + \mathbf{v}(t + \tfrac{1}{2}\Delta t)\Delta t$$

3. Second "Kick" (Velocity Second Half-Step)

Finally, the new acceleration, $\mathbf{a}(t + \Delta t)$, is computed from the forces at the new positions, $\mathbf{x}(t + \Delta t)$, and the **mid-step velocity**, $\mathbf{v}(t + \frac{1}{2}\Delta t)$. The acceleration is then used to complete the velocity update for the second half of the time step.

$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t + \tfrac{1}{2}\Delta t) + \mathbf{a}(t + \Delta t)\frac{\Delta t}{2}$$

While mathematically equivalent to Verlet in simpler cases, this staggered formulation is particularly robust for handling the time-varying and velocity-dependent forces present in a cosmological simulation. The symmetric "kick-force-kick" structure gracefully incorporates these complexities, which is why the KDK leapfrog is the most common integrator in modern cosmological N-body codes.

Symmetry and Second-Order Accuracy

In numerical physics, the "order of accuracy" dictates how fast errors shrink when we take smaller timesteps (Δt). A basic, asymmetric algorithm like the Forward Euler method is only *first-order* accurate ($O(\Delta t)$); if we cut the timestep in half, the error only drops by half.

The KDK scheme achieves higher accuracy through its **symmetry**. By splitting the velocity kick into two equal halves that perfectly bracket the position drift, the algorithm becomes **time-reversible**. For conservative forces like gravity, this mieans that if we paused the simulation, reversed all the velocities, and ran the KDK math backward, the particles would trace their exact steps back to their starting positions.

In numerical calculus, any integration scheme that is symmetric and time-reversible causes the leading, first-order Taylor series error terms to cancel each other out. Because the first-order error is gone, the largest surviving error scales with the square of the timestep ($O(\Delta t^2)$). This means KDK is strictly **second-order accurate**—if we cut the timestep in half, the integration error drops by a factor of four.

Key Literature & Further Reading
Springel, V. (2005). *The cosmological simulation code GADGET-2. Monthly Notices of the Royal Astronomical Society*, 364(4), 1105-1134. arXiv:astro-ph/0505010. Available at https://arxiv.org/abs/astro-ph/0505010

The Particle-Mesh Method

The Particle-Mesh (PM) method is built on a different perspective. Instead of calculating the gravitational pull between every pair of particles, it simplifies the problem by describing the mass distribution on a regular grid. From this "mass map", the gravitational potential and forces can be solved on the grid itself. These are the steps:

1. **Potential calculation:** First, the gravitational potential (Φ) is calculated for the entire grid. The potential is a scalar "landscape" that describes the depth of the gravitational well at every point.

2. **Force calculation:** Second, the force (**F**) is determined by finding the steepest downhill slope (the gradient) of that potential landscape.

This `Mass -> Potential -> Force` pipeline is the foundation of the PM method. The following sections break down how each part of this process is achieved.

Step 1. Finding the Potential

The process of finding the potential begins by describing the mass distribution on the grid, which then serves as the input for the physical law that governs how that mass creates the potential.

Mass Assignment (NGP)

The first step in this process is **mass assignment**: the procedure for transferring the mass of our continuously positioned particles onto the discrete nodes of the grid.

The simplest and most intuitive way to do this is the **Nearest Grid Point (NGP)** scheme: for each particle, we find the single grid point (or cell center) that it is closest to, and assign the particle's *entire mass* to that one point.

The result is an array representing the mass density field, $\rho_{i,j,k}$. Mathematically, the density in a given cell (i, j, k) is the sum of the masses of all particles within that cell, divided by the cell's volume:

$$\rho_{i,j,k} = \frac{1}{L^3} \sum_{p \in \text{cell}(i,j,k)} m_p$$

Where m_p is the mass of a particle p, and L is the side length of a grid cell.

While NGP is very simple, it can introduce inaccuracies. As we will explore in a later section, more sophisticated schemes like Cloud-in-Cell (CIC) can be used to create a smoother and more accurate density field.

Poisson's Equation

The potential field Φ can be determined from the mass density field, $\rho_{i,j,k}$. The fundamental law linking mass to gravitational potential is **Poisson's Equation**.

$$\nabla^2 \Phi = 4\pi G\rho$$

Where:

- ρ is the mass density grid — the **input**.
- Φ is the gravitational potential field — the **output**.
- G is the gravitational constant.
- ∇^2 (the **Laplacian**) is a mathematical operator that measures how much a function curves around a point—the **net curvature**.

In this equation, mass acts as the source of curvature: where there is mass, the potential bends inward, forming gravitational wells that attract other masses. Where $\rho = 0$, there's no net curvature. This may seem counterintuitive, as the potential field clearly forms a curved, gravitational well even in the empty space around a mass. The key is that the Laplacian, $\nabla^2\Phi$, measures the **net curvature**. In the smooth $1/r$ shape of a potential in empty space, the radial inward bending of the field is perfectly balanced by a natural geometric spreading effect in three dimensions. These two effects cancel each other out, resulting in zero net curvature.

Possion's equation allows us to transition from Newton's "particle-to-particle" worldview to a continuous "field" worldview. Transitioning to this specific differential equation is a key element of the PM method: as we will see in the next section, writing gravity in this form allows us to take advantage of the Convolution Theorem and Fast Fourier Transforms (FFTs) to solve the gravitational field in a fraction of the time. Here is the step-by-step derivation from Newton's law of gravity to Poisson's equation.

Step 1: The Gravitational Field of a Point Mass We start with the standard Newtonian gravitational potential for a single point mass M at the origin:

$$\Phi = -\frac{GM}{r}$$

The gravitational field (the acceleration vector $\boldsymbol{g}$) is simply the negative gradient of this potential. Taking the derivative with respect to r gives us the classic inverse-square law:

$$\boldsymbol{g} = -\nabla\Phi = -\frac{GM}{r^2}\hat{\boldsymbol{r}}$$

Step 2: The Gravitational Flux (Gauss's Law) Imagine wrapping that point mass inside a perfectly spherical imaginary surface (a Gaussian surface) with radius r. We want to calculate the total "flux" of the gravitational field pointing inward through that surface.

To do this, we integrate the field $\boldsymbol{g}$ over the surface area A of the sphere ($A = 4\pi r^2$):

$$\oint_S \boldsymbol{g} \cdot d\boldsymbol{A} = \left(-\frac{GM}{r^2}\right)(4\pi r^2) = -4\pi GM$$

Notice how the r^2 terms perfectly cancel out. The total flux through the surface depends *only* on the mass enclosed inside it, regardless of the size or shape of the boundary.

Step 3: Transitioning to a Continuous Fluid In a cosmological simulation, we don't just have one point mass; we map billions of particles onto a continuous density grid, represented by $\rho(\boldsymbol{r})$.

The total enclosed mass M is simply the volume integral of the density field:

$$M = \int_V \rho(\boldsymbol{r}) dV$$

Substitute this into our flux equation from Step 2:

$$\oint_S \boldsymbol{g} \cdot d\boldsymbol{A} = -4\pi G \int_V \rho(\boldsymbol{r}) dV$$

Step 4: The Divergence Theorem Now we use a fundamental tool from vector calculus: the **Divergence Theorem**. This theorem states that the flux of a vector field across a closed boundary is exactly equal to the volume integral of the *divergence* ($\nabla\cdot$) of that field inside the boundary.

Applying it to our gravitational field gives:

$$\oint_S \boldsymbol{g} \cdot d\boldsymbol{A} = \int_V (\nabla \cdot \boldsymbol{g}) dV$$

Step 5: Equating the Integrals Because Step 3 and Step 4 are calculating the exact same flux, we can set them equal to each other:

$$\int_V (\nabla \cdot \boldsymbol{g}) dV = \int_V (-4\pi G\rho) dV$$

Because this must be true for *any* arbitrary volume V we choose in our simulation box, the integrands themselves must be identical. We can strip away the integrals entirely to get the differential form of Gauss's Law:

$$\nabla \cdot \boldsymbol{g} = -4\pi G\rho$$

Step 6: The Poisson Equation Remember from Step 1 that the gravitational field is the negative gradient of the potential ($\boldsymbol{g} = -\nabla\Phi$).

Substitute that definition back into our differential equation:

$$\nabla \cdot (-\nabla\Phi) = -4\pi G\rho$$

The divergence of a gradient ($\nabla \cdot \nabla$) is the **Laplacian operator**, denoted as ∇^2. Pulling the negative sign out and canceling it on both sides gives us the final equation:

$$\nabla^2\Phi = 4\pi G\rho$$

Solving Poisson's equation means finding the global shape of Φ given all the local sources ρ. Doing this directly is computationally expensive, but as we'll see next, the Fast Fourier Transform (FFT) offers an efficient way to compute it by turning this differential problem into simple multiplications in frequency space.

The FFT and the Convolution Theorem

Given the mass density grid, ρ, and the rule connecting it to the potential, Poisson's Equation, the challenge now is to solve it. Although Poisson's equation is written as a differential equation (∇^2), solving it in real space means integrating Newton's law: calculating the potential at every grid point by summing the $1/r$ influence from all other grid points. This sliding, brute-force operation is known mathematically as a **convolution**. It's a slow, computationally expensive task.

Fortunately, there is a more efficient mathematical tool that can solve this problem: the **Fast Fourier Transform (FFT)**. This algorithm is used to take advantage of the **Convolution Theorem**.

The Fourier Transform The **Fourier Transform** is a mathematical operation that rewrites any spatial field in terms of its **spatial frequencies** — the underlying wave patterns that make it up.

Suppose we have a density field $\rho(\mathbf{x})$ defined across space. Instead of describing it point by point, we can express it as a sum of smooth oscillating patterns — *plane waves* — each with its own wavelength, direction, and amplitude. The Fourier Transform tells us **how each wave** contributes to the total field.

Formally, it is written as:

$$\hat{\rho}(\mathbf{k}) = \int \rho(\mathbf{x}) e^{-i\mathbf{k}\cdot\mathbf{x}} d^3x$$

Here, $\mathbf{x}$ represents position in real space, and $\mathbf{k}$ is the **wavevector**, describing one of those plane waves. Each component (k_x, k_y, k_z) measures how rapidly the field oscillates along that direction, while its magnitude

$$k = |\mathbf{k}| = \frac{2\pi}{\lambda}$$

tells us the *spatial frequency*.

The result of the transform, $\hat{\rho}(\mathbf{k})$, is a **complex number**. Its magnitude $|\hat{\rho}(\mathbf{k})|$ gives the *amplitude*, and its phase $\arg(\hat{\rho}(\mathbf{k}) = \arctan(\text{Im}/\text{Re}))$ specifies the *offset* — where the wave's peaks and troughs occur in space.

Thus, while $\rho(\mathbf{x})$ is a **real-valued function of position**, $\hat{\rho}(\mathbf{k})$ is a **complex-valued function of wavevector**. They describe the same field, but from two complementary perspectives: one in **Real space**, one in **Frequency (or k-) space**. This dual representation is very useful in simulations.

> In what follows, we'll use the shorthand notation ρ_k and Φ_k to denote the discrete Fourier-transformed fields, corresponding to $\hat{\rho}(\mathbf{k})$ and $\hat{\Phi}(\mathbf{k})$ in the continuous case, but defined only at the discrete $\mathbf{k}$ values of our simulation grid.

The Convolution Theorem This theorem is the heart of the entire PM method. It states:

> A slow and complicated **convolution** in real space becomes a fast and simple element-by-element **multiplication** in frequency space.

This allows us to replace the slow, brute-force calculation with a faster three-step process:

1. **Transform to Frequency Space:** We use the **FFT** to transform our mass grid, ρ, into its frequency representation, ρ_k. A convenient property of the FFT is that it automatically treats the finite grid as if it were periodic. It represents the data as a sum of simple waves that fit perfectly end-to-end within the box, which is mathematically equivalent to assuming the grid repeats infinitely like a tiled pattern.

   To compute the gravitational potential, we must understand how a single unit of mass influences the space around it. In linear mathematics, the tool for this is a Green's function. Broadly speaking, a Green's function represents the "impulse response" of a system to a single, localized point of input. Once you know how a system reacts to one point, you can determine its reaction to any complex input by summing together the individual point responses.

For our specific case, the "system" is the gravitational field governed by Poisson's equation. Therefore, our specific Green's function, $G(\mathbf{r})$, is defined as the solution to Poisson's equation for a unit point source:

$$\nabla^2 G(\mathbf{r}) = 4\pi G\delta(\mathbf{r}).$$

Where $\delta(\mathbf{r})$ is the Dirac delta function, representing the mass density of an idealized point source located at the origin—a mathematical spike that is infinitely dense at that single spot and zero everywhere else, containing exactly one unit of total mass. In three dimensions, solving this gives the familiar Newtonian potential for a point mass: $G(\mathbf{r}) = -G/|\mathbf{r}|$. This mathematical kernel acts as the system's response to a point mass, linking the density field to the potential through Poisson's equation.

When we move to frequency space, derivatives become multiplications by $-k^2$, so the corresponding frequency-space form of the Green's function is

$$\mathcal{F}\{\nabla^2 G(\mathbf{r})\} = -k^2 G_k$$

$$\mathcal{F}\{4\pi G\delta(\mathbf{r})\} = 4\pi G$$

$$G_k = -\frac{4\pi G}{k^2}.$$

This is the function we use to compute the potential in the Fourier domain. This function has a mathematical singularity at the $k = 0$ **mode**. This mode (also known as the DC component) represents the **average potential** of the entire simulation box.

However, since physical forces depend only on the **gradient** of the potential ($\mathbf{F} = -\nabla\Phi$), not its absolute value, this average potential is physically arbitrary. To avoid the division-by-zero, we can redefine the $k = 0$ component of the potential to zero. Beyond just fixing a numerical error, this is mathematically equivalent to subtracting the mean background density of the universe from the source term. This is a standard technique in cosmology (often related to the "Jeans swindle"), which ensures that gravity is driven only by local *perturbations* (overdensities and underdensities) rather than the infinite mass of a periodic universe. It makes the calculation perfectly well-defined without affecting the final relative forces.

2. **Multiply:** In frequency space, the Poisson equation becomes an element-wise multiplication:

$$\Phi_k = G_k \cdot \rho_k$$

3. **Transform Back:** We take the resulting potential in frequency space, Φ_k, and use the **Inverse Fast Fourier Transform (IFFT)** to convert it back into the real-space potential grid, Φ, that we wanted.

By taking this detour through frequency space, we replace a slow algorithm that scales as $O(M^6)$ (for a 3D grid with $M \times M \times M$ cells) with one that scales as $O(M^3 \log M)$. This is what makes the Particle-Mesh method convenient, enabling simulations with millions or billions of particles.

Step 2. From Potential to Force

Now that we have the potential grid, Φ, we can calculate the force grid. The physical relationship is universal: force is the negative gradient of the potential.

$$\mathbf{F} = -\nabla\Phi$$

On a discrete grid, we can't take a true derivative. We approximate it using a **finite difference**. A common and accurate method is the **central difference**, which calculates the slope at a point by looking at the values of its neighbors on either side. For the x-component of the force at grid cell (i, j, k), the formula is:

$$F_{x,i,j,k} \approx -\frac{\Phi_{i+1,j,k} - \Phi_{i-1,j,k}}{2L}$$

With the force calculated at every point on the grid, the final step is to **interpolate** this force back to each particle's continuous position. This is done using the same scheme we used for mass assignment (e.g., NGP or CIC), completing the Particle-Mesh calculation.

Advanced Interpolation

The Flaws of Nearest Grid Point (NGP)

In a previous section, we introduced the Nearest Grid Point (NGP) scheme as the simplest way to assign mass to the grid. While its simplicity is appealing, it comes at a significant cost to the simulation's accuracy. The "blocky," pixelated density field it creates leads to an equally blocky and unphysical force field.

The primary flaw of NGP is that the force a particle feels is **discontinuous**. A real gravitational field is smooth and changes continuously with position. The jerky, stepwise force from an NGP grid is a poor and unphysical approximation. This leads to several significant problems:

1. **Poor Energy Conservation:** This is the most damaging consequence. Symplectic integrators like Velocity Verlet can only conserve energy if the force is the smooth gradient of a potential. The sudden "jumps" in force at the cell boundaries introduce small, systematic errors into the integration. These errors accumulate over time, causing the total energy of the simulation to **drift** upwards or downwards, rather than just oscillating around the true value.
2. **Violation of Newton's Third Law:** While the total momentum of the system may be globally conserved, the force between any specific pair of particles is not guaranteed to be equal and opposite. The force on particle A depends only on which cell it's in, and the force on particle B depends only on which cell *it's* in. This crude mediation by the grid breaks the pairwise symmetry required for good energy conservation.
3. **Grid-Imposed Artifacts:** The force field has an artificial, grid-like pattern. Particles can feel an unphysical pull along the grid axes (x and y) that is stronger than the pull along the diagonals. This can cause particles to artificially cluster along grid lines, a distracting and inaccurate artifact of the method.

Because of these flaws, NGP is rarely used in simulations where accuracy is a priority. To achieve the stable, energy-conserving behavior we need, we must adopt a smoother method for connecting the particles to the grid, which leads us to the Cloud-in-Cell scheme.

Cloud-in-Cell (CIC)

To achieve a stable simulation that conserves energy, we need a smooth way to connect the particles to the grid. The standard method for this is the **Cloud-in-Cell (CIC)** interpolation scheme.

Particles as Clouds

Instead of treating each particle as an infinitesimal point, the CIC method imagines each particle as a small, **cubic "cloud"** of mass, the same size as a grid cell. As this particle-cloud moves through the simulation space, it naturally overlaps with the **eight** nearest grid points that form the corners of the cell it currently occupies.

The mass of the particle is then distributed, or "splatted," onto these eight grid points. The amount of mass assigned to each point is simply proportional to the **volume of overlap** between the particle's cloud and the region surrounding each grid point. This is a form of **trilinear interpolation**. A particle in the exact center of a cubic cell would distribute 12.5% of its mass to each of the eight corners. A particle mostly in one corner of a cell would give most of its mass to that corner's node.

This process results in a much smoother and more physically realistic mass density grid. A small movement by a particle leads to a small, continuous change in the mass distribution on the grid, completely eliminating the sudden "jumps" of the NGP method.

At its core, CIC is a linear interpolation scheme. Higher-order schemes (like TSC or PCS) exist and provide smoother forces, but are not in the scope of this text.

Symmetric Interpolation

A smooth mass distribution is only half the story. The true magic of CIC, and the reason it conserves energy, lies in its perfect symmetry.

After the forces have been calculated on the grid, we must interpolate them back to the particle's continuous position. The rule for energy conservation is that the **force interpolation scheme must be consistent with the mass assignment scheme**.

CIC follows this rule perfectly. The force on the particle is calculated by taking a weighted average of the forces from the **same eight grid points**, using the **exact same volume-based weights** that were used to distribute the mass.

This symmetry between "splatting" the mass and "gathering" the force ensures that Newton's third law ($\mathbf{F}_{ij} = -\mathbf{F}_{ji}$) is precisely obeyed for any pair of particles, even though their interaction is being mediated by the grid. Because the forces are perfectly reciprocal, the force field is numerically conservative.

When this conservative force is fed into a symplectic integrator, the system's total energy is conserved remarkably well. The systematic energy *drift* seen with NGP is transformed into the small, bounded *oscillation* expected from a high-quality simulation. While slightly more complex to implement, the improvement in accuracy and stability makes CIC the standard choice for most modern Particle-Mesh codes.

Implementation: "Splatting" Mass and "Gathering" Forces

The conceptual idea of treating particles as "clouds" translates into a clean, two-part algorithm. In simulation jargon, these two parts are often called **"splatting"** (distributing the particle mass onto the grid) and **"gathering"** (interpolating the force from the grid back to the particle).

The key to energy conservation is that these two operations must be perfectly symmetric, using the exact same weights for both processes.

For simplicity, the following explanation is for a 2D case.

The "Splat" Step: Mass Assignment

This process is performed for every particle to create the final mass density grid, ρ.

1. Find the Reference Grid Point and Fractional Position

First, for a given particle p with position $\mathbf{x}_p = (x_p, y_p)$, we find the integer index of the grid point at its "bottom-left," denoted (i, j). We then find the particle's fractional position within that cell, (δ_x, δ_y), where both values range from 0 to 1. Let L be the side length of a grid cell.

The reference grid index is found using the floor function, $\lfloor \cdot \rfloor$:

$$i = \lfloor x_p/L \rfloor$$

$$j = \lfloor y_p/L \rfloor$$

The fractional position within the cell is then:

$$\delta_x = (x_p/L) - i$$

$$\delta_y = (y_p/L) - j$$

2. Calculate the Weights

Next, we calculate four weights based on these fractional positions. Each weight corresponds to the fractional area of the particle's "cloud" that overlaps with the four surrounding grid cells located at (i, j), $(i + 1, j)$, $(i, j + 1)$, and $(i + 1, j + 1)$.

The weights for the corners are:

$$w_{i,j} = (1 - \delta_x)(1 - \delta_y)$$

$$w_{i+1,j} = \delta_x(1 - \delta_y)$$

$$w_{i,j+1} = (1 - \delta_x)\delta_y$$

$$w_{i+1,j+1} = \delta_x\delta_y$$

Notice that these four weights always sum to 1.

3. Distribute the Mass

Finally, we add the mass of the particle, m_p, scaled by the appropriate weight, to each of the four corresponding grid points. This process is repeated for all particles in the simulation. The total mass on a given grid point is the sum of the contributions from all particles whose "clouds" overlap it.

The contribution from a single particle p to the mass at each of the four nodes is:

$$\Delta M_{i,j} = m_p \cdot w_{i,j}$$

$$\Delta M_{i+1,j} = m_p \cdot w_{i+1,j}$$

$$\Delta M_{i,j+1} = m_p \cdot w_{i,j+1}$$

$$\Delta M_{i+1,j+1} = m_p \cdot w_{i+1,j+1}$$

To get the final mass density grid, ρ, the total mass accumulated at each node is divided by the cell area, L^2. All grid indices are taken modulo the grid size to correctly handle the periodic boundaries.

The "Gather" Step: Force Interpolation

This step occurs after the forces have been calculated on the grid (creating an acceleration field, $\mathbf{a}_{\text{grid}}$) and is the mirror image of the splatting process.

To find the force on a particle, we use the **exact same** indices and weights we would calculate for it in the splatting step. We then perform a weighted average of the acceleration values from the four surrounding grid points to find the acceleration at the particle's precise location, $\mathbf{a}_p$.

Let the acceleration field on the grid be $\mathbf{a}_{i,j} = (a_{x,i,j}, a_{y,i,j})$ and the four CIC weights for a given particle be $w_{i,j}$, $w_{i+1,j}$, $w_{i,j+1}$, and $w_{i+1,j+1}$.

The x-component of the interpolated acceleration for the particle, $a_{x,p}$, is calculated as:

$$a_{x,p} = a_{x,i,j} \cdot w_{i,j} + a_{x,i+1,j} \cdot w_{i+1,j} + a_{x,i,j+1} \cdot w_{i,j+1} + a_{x,i+1,j+1} \cdot w_{i+1,j+1}$$

The y-component, $a_{y,p}$, is calculated in the exact same way using the y-components of the grid acceleration field.

The final force on the particle, $\mathbf{F}_p$, is its mass, m_p, times this interpolated acceleration vector:

$$\mathbf{F}_p = m_p \mathbf{a}_p$$

This symmetric "Splat-Gather" procedure ensures that the forces are conservative, which is the fundamental reason why CIC allows a symplectic integrator to conserve energy over long periods.

Deconvolving the Mass Assignment

While the symmetric "Splat-Gather" procedure guarantees energy conservation, it introduces a hidden numerical artifact into the gravity solver.

In mathematics, any time we take a set of discrete points and "smear" them across a spatial domain, we are performing a **convolution**. The CIC mass assignment smears a point mass into a shape equivalent to convolving a 1D uniform boxcar (a "top-hat") with itself, creating a triangular density cloud.

In Fourier analysis, the **Convolution Theorem** dictates that a complex convolution in real space is identical to a simple multiplication in frequency space. The Fourier transform of a 1D top-hat function is the sinc function, defined as:

$$\text{sinc}(x) = \frac{\sin(x)}{x}$$

Because the CIC shape is the convolution of two top-hats, its Fourier transform is sinc^2. In a 3D grid with cell size Δx, the continuous CIC assignment acts as a mathematical filter, $W(\mathbf{k})$, that multiplies the true density field in frequency space:

$$W(\mathbf{k}) = \text{sinc}^2\left(\frac{k_x \Delta x}{2}\right) \cdot \text{sinc}^2\left(\frac{k_y \Delta x}{2}\right) \cdot \text{sinc}^2\left(\frac{k_z \Delta x}{2}\right)$$

This creates a numerical problem often called the **"gravity pothole."** The simulation applies this CIC filter *twice*—once when splatting the mass onto the grid ($\rho_{\text{grid}} = \rho_{\text{true}} * W$), and once when gathering the forces back to the particles ($\mathbf{F}_{\text{particle}} = \mathbf{F}_{\text{grid}} * W$).

Because the filter is applied twice, the total numerical dampening applied is $W(\mathbf{k})^2$. This means the gravitational potential is accidentally multiplied by a factor of $\mathbf{sinc}^4$ along every axis. Since the sinc function decays as frequencies get higher, this accidental sinc^4 multiplication artificially weakens the gravitational pull at intermediate scales (typically distances of 1 to 3 grid cells).

To fix this, we must undo the accidental CIC convolution during the gravity calculation. This process is known as **deconvolution**.

When we solve for the gravitational potential in Fourier space (using the Fast Fourier Transform), we calculate the value of the sinc function for every specific wavevector (k_x, k_y, k_z). We then divide the potential by the sinc^4 penalty. The equation for the basic Particle-Mesh potential in Fourier space becomes:

$$\Phi_k = \rho_k \cdot \left(\frac{-4\pi G}{k^2}\right) \cdot \frac{1}{W(\mathbf{k})^2}$$

By explicitly dividing out the mass assignment window, we "sharpen" the raw grid back, ensuring that the forces behave as Newtonian physics demands.

Key Literature & Further Reading
Bagla, J. S., & Padmanabhan, T. (2004). *Cosmological N-Body Simulations.* arXiv:astro-ph/0411730. Available at https://arxiv.org/pdf/astro-ph/0411730.pdf

The P³M Algorithm

Combining PP for Short-Range and PM for Long-Range

We have seen that the Particle-Mesh (PM) method is efficient for calculating the gravitational field of a large number of particles. However, its speed comes at the cost of accuracy at small scales. The grid is good at capturing the overall "blurry" shape of the gravitational field, but it's innacurate at resolving the sharp, fine details of the force between two particles that are very close to each other. This inaccuracy at short ranges is the primary weakness of the pure PM method.

On the other hand, the direct Particle-Particle (PP) calculation is the exact opposite. While it is perfectly accurate at all scales, its weakness, is that its $O(N^2)$ complexity makes it too slow for a large number of particles.

This presents a classic trade-off: speed or accuracy. The **Particle-Particle Particle-Mesh (P³M)** algorithm provides an elegant solution by combining both methods, using each one only where it excels.

The P³M method splits the force calculation into two parts based on a **cutoff radius**, r_c:

1. **Long-Range Force (PM):** The smooth, gentle pull from all the **distant** particles (those farther than r_c) is calculated efficiently using the Particle-Mesh method.
2. **Short-Range Force (PP):** The sharp, strong force from the few **nearby** particles (those closer than r_c) is calculated precisely using the direct Particle-Particle method.

The Subtractive Scheme

Simply adding these two forces together would be incorrect, as the PM method already includes an inaccurate estimate of the short-range forces. Instead, we use the PP calculation to *correct* the PM force at short distances. This is often done with a **subtractive scheme**:

$$\mathbf{F}_{\text{total}} = \mathbf{F}_{\text{PM}} + \left(\mathbf{F}_{\text{PP}}^{\text{short}} - \mathbf{F}_{\text{PM}}^{\text{short}}\right)$$

The process is straightforward:

1. First, we calculate the baseline $\mathbf{F_{PM}}$ for all particles. This gives us the correct long-range force everywhere but an incorrect "blurry" force for nearby pairs.
2. Then, for any pair of particles closer than the cutoff radius, we calculate the **true, sharp force** between them, $\mathbf{F}_{\mathbf{PP}}^{\mathbf{short}}$.
3. We also calculate an approximation of the **blurry, inaccurate force** that the PM method produced for that same pair, $\mathbf{F}_{\mathbf{PM}}^{\mathbf{short}}$.
4. Finally, we subtract the inaccurate mesh force and add the correct direct force. This effectively replaces the blurry grid force with the sharp, accurate PP force, but only where it matters—at short distances.

By dividing the problem this way, P³M leverages the strengths of both methods. It uses the fast PM algorithm for the vast majority of interactions (the thousands of weak pulls from distant particles) and reserves the slow but accurate PP algorithm only for the few critical interactions between close neighbors. The result is a simulation that is nearly as fast as a pure PM code but nearly as accurate as a pure PP code—the true best of both worlds.

Calculating the Mesh-Force Correction

To implement the subtractive scheme, we need a mathematical function for $\mathbf{F}_{\text{PM}}^{\text{short}}$ that approximates the "blurry" force produced by the grid at short distances. We can't get this from the final grid itself, as it contains the combined influence of all particles.

Instead, we model this effect with a standard gravitational force formula that has been **softened** with a special parameter, ϵ_{PM}, chosen specifically to mimic the resolution of the Particle-Mesh grid.

The vector force that approximates the mesh's influence between two particles with masses m_1 and m_2 is given by:

$$\mathbf{F}_{\text{PM}}^{\text{short}} = \frac{G m_1 m_2 \mathbf{r}}{\left(r^2 + \epsilon_{\text{PM}}^2\right)^{3/2}}$$

The terms in this formula are:

- $\mathbf{r}$ is the vector separating the two particles.
- r is the magnitude of that vector, $r = \|\mathbf{r}\|$.
- G, m_1, m_2 are the gravitational constant and the particle masses.
- ϵ_{PM} is the crucial term: a **softening length** specifically chosen to match the grid's resolution. A standard and effective choice is to set this value to be proportional to the grid cell length, L. For example:

$$\epsilon_{\text{PM}} \approx 0.5 \cdot L$$

This formula creates a force that is significantly weakened at short distances (when $r \lesssim \epsilon_{\text{PM}}$), which successfully mimics the behavior of the full PM/FFT calculation. By subtracting this specific force in the correction step, we effectively cancel out the grid's primary error at short range.

Choosing the Cutoff Radius (r_c)

The choice of the cutoff radius, r_c, is a crucial tuning parameter in a P³M simulation. It represents a trade-off between accuracy and computational speed.

- A **small** cutoff radius means the fast PM method handles most of the work, but we risk losing accuracy if the cutoff is smaller than the region where the PM force is unreliable.
- A **large** cutoff radius ensures high accuracy at short ranges, but it forces the slow PP calculation to do much more work, which can bog down the entire simulation.

The optimal choice is not arbitrary; it is fundamentally linked to the resolution of the Particle-Mesh grid. The PM method's accuracy degrades significantly at distances smaller than about 2 to 3 grid cell sizes. Therefore, the cutoff radius must be large enough to ensure the accurate PP method is used throughout this entire "inaccurate zone."

A standard and robust rule of thumb is to set the cutoff radius to be a few times the grid cell length, L:

$$r_c \approx 2.5 \cdot L$$

This choice guarantees that the sharp, correct PP force is used wherever the PM force is most likely to fail. The primary parameter that is usually tuned is the grid size itself; once the grid size is chosen, the cutoff radius is set accordingly to maintain this balance.

The Switching Function

The subtractive scheme is a powerful way to correct for the short-range errors of the Particle-Mesh method. However, using a hard cutoff radius—where the correction is fully active if $r < r_c$ and instantly zero if $r \geq r_c$—can create a small, abrupt "jolt" in the force. This discontinuity, however small, can introduce numerical errors and impact the long-term energy conservation of the simulation.

To achieve the highest accuracy, we must ensure the total force is a perfectly smooth function at all distances. This is accomplished by introducing a **switching function**, $S(r)$, that smoothly "fades out" the short-range correction as the particle separation, r, approaches the cutoff radius, r_c.

The total force is then calculated as:

$$\mathbf{F}_{\text{total}} = \mathbf{F}_{\text{PM}} + S(r) \cdot \left(\mathbf{F}_{\text{PP}}^{\text{short}} - \mathbf{F}_{\text{PM}}^{\text{short}}\right)$$

The switching function $S(r)$ operates over a small **transition zone**, typically defined between a starting radius, r_{start}, and the cutoff radius, r_c. It has the following properties:

1. For $r \leq r_{\text{start}}$, the function is $S(r) = 1$. The correction is fully applied.
2. For $r \geq r_c$, the function is $S(r) = 0$. The correction is fully turned off.
3. In the transition zone, $r_{\text{start}} < r < r_c$, the function smoothly decreases from 1 to 0.

To ensure the force changes perfectly smoothly, the *derivative* of the switching function should also be zero at the start and end of the transition. A standard and effective way to achieve this is with a cubic polynomial.

First, we define a normalized distance, x, that goes from 0 to 1 across the transition zone:

$$x = \frac{r - r_{\text{start}}}{r_c - r_{\text{start}}}$$

Then, a polynomial that satisfies all the smoothness conditions is:

$$S(x) = 2x^3 - 3x^2 + 1$$

Using this function to taper the correction term eliminates the unphysical jolt at the cutoff. It creates a continuous and differentiable total force, which allows the symplectic integrator to perform optimally and leads to superior long-term energy conservation.

Key Literature & Further Reading
Shirokov, A., & Bertschinger, E. (2005). *GRACOS: Scalable and Load Balanced P3M Cosmological N-body Code.* arXiv:astro-ph/0505087. Available at https://arxiv.org/abs/astro-ph/0505087

Fourier-Split Gravity

The classical Particle-Particle Particle-Mesh (P^3M) algorithm allowed cosmologists to bypass the $O(N^2)$ scaling of direct summation in the 1980s. However, today, this method has been replaced in cosmological codes by a more elegant architecture known as **Fourier-Split PM** (often referred to as TreePM when paired with a tree-based short-range solver). To understand why this shift occurred, we must examine the discrete grid at short distances.

The Flaws of the Subtractive Scheme

In the classical P^3M approach, we attempted to correct the short-range errors of the Particle-Mesh grid by subtracting an analytical approximation of the grid's force ($F_{\text{PM}}^{\text{short}}$) and replacing it with the exact Newtonian force (F_{PP}). This scheme assumes that the grid's short-range force is smooth, isotropic (equal in all directions), and analytically predictable (modeled as a softened $1/r^2$ curve).

In reality, the force produced by a finite-difference grid at sub-cell distances ($r < \Delta x$) is violently **anisotropic** and polluted by grid artifacts. This happens because we are mapping continuous particles onto a discrete grid. Regardless of which specific mass assignment scheme is used (CIC, NGP, TSC...), the grid is incapable of resolving any structure smaller than a cell.

Imagine two dark matter particles, A and B, that are very close to each other, but happen to sit just on opposite sides of a grid cell boundary.

Because the grid can only "see" mass at its discrete nodes, it is blind to the subcell distance between the particles. Instead, it projects their mass into localized clouds anchored to the surrounding nodes:

- Particle A deposits the majority of its mass toward the left grid node.
- Particle B deposits the majority of its mass toward the right grid node.

Even though the particles are very close, the grid registers two distinct mass clouds localized on two different nodes. When the solver calculates the potential and takes the spatial derivative to find the force, the gradient naturally points "downhill" away from the center. Particle A is pulled to the left, and particle B is pulled to the right. The two particles are pulled apart. This is a purely numerical **artificial repulsion**.

Because the true PM force is noisy, direction-dependent, and occasionally repulsive, a smooth, isotropic analytical formula ($F_{\text{PM}}^{\text{short}}$) can't match it. When we subtract the smooth formula from the noisy grid, the subtraction fails to cancel out the error. Instead, it leaves behind a jagged, unphysical residual force field. This "mesh artifact" causes particles to artificially cluster along the x, y, and z grid axes and introduces microscopic jolts that slowly destroy the long-term energy conservation of the Kick-Drift-Kick integrator.

Fixing the Grid in Frequency Space

Given that we can't guess and subtract the grid's error in real space, the modern solution is to eliminate the error before it reaches real space.

Instead of allowing the PM solver to produce a jagged, anisotropic force, the Fourier-Split PM method forces the grid's potential to be smooth by intervening in the middle of the Fast Fourier Transform (FFT) pipeline.

Recall that in frequency space (k-space), solving Poisson's equation is a simple multiplication:

$$\Phi_k = \rho_k \cdot \left(\frac{-4\pi G}{k^2} \right)$$

In the Fourier-Split method, we introduce a blurring function into this equation by multiplying the potential by a **Gaussian decay filter**:

$$\Phi_k = \rho_k \cdot \left(\frac{-4\pi G}{k^2} \right) \cdot \exp(-k^2 r_s^2)$$

Here, r_s is the **Gaussian smoothing scale**, a tuning parameter typically set to roughly 1.25 to 1.5 times the grid cell width (Δx).

In Fourier analysis, high frequencies (large k) correspond to small-scale, sharp, jagged details. Because the exponent is negative and proportional to k^2, the term $\exp(-k^2 r_s^2)$ plummets rapidly to zero for high-frequency modes.

We should keep the Cloud-in-Cell (CIC) deconvolution we established earlier. While the Gaussian filter intentionally blurs the grid to remove anisotropy, we still need the deconvolution to fix the sinc^4 "gravity pothole" caused by the mass assignment and force interpolation steps. Combining both of these frequency-space corrections gives us the final equation for the gravitational potential:

$$\Phi_k = \rho_k \cdot \left(\frac{-4\pi G}{k^2} \right) \cdot \frac{\exp(-k^2 r_s^2)}{W(\mathbf{k})^2}$$

By applying this filter, we erase the grid's ability to resolve any structure smaller than r_s. When we perform the Inverse FFT to bring this potential back into real space, the resulting gravitational field is no longer jagged, it no longer suffers from aliasing along the cell boundaries, and it is strictly non-repulsive. The long-range grid force is now guaranteed to be isotropic and smooth.

However, by blurring the grid, we have also suppressed the strength of gravity at short distances. To recover the true physics, we must now define an exact short-range force to complement the newly smoothed grid.

The Exact Analytical Complement

In the classical P^{3}M scheme, the short-range correction was an educated guess designed to subtract the grid's errors. In the Fourier-Split PM method, the short-range correction is a mathematically exact derivation.

Because we explicitly defined the shape of the long-range potential using a Gaussian filter in frequency space ($\exp(-k^2 r_s^2)$), we can use the inverse Fourier transform to find the analytical shape of that smoothed potential in real space. For a point mass M, the Gaussian-smoothed potential is:

$$\Phi_{\text{long-range}}(r) = -\frac{GM}{r} \text{erf}\left(\frac{r}{2r_s} \right)$$

Where erf is the standard Error Function. Since the true Newtonian potential is $\Phi(r) = -GM/r$, the short-range potential we *missed* by blurring the grid is simply the difference between the true potential and the long-range potential:

$$\Phi_{\text{short-range}}(r) = -\frac{GM}{r} \left[1 - \text{erf}\left(\frac{r}{2r_s} \right) \right] = -\frac{GM}{r} \text{erfc}\left(\frac{r}{2r_s} \right)$$

Here, erfc is the **Complementary Error Function**. To find the actual force that our Particle-Particle (PP) solver needs to apply, we take the negative gradient ($-\nabla$) of this short-range potential. Taking the derivative of

the erfc function introduces an additional exponential term, giving us the final equation for the short-range force between two particles:

$$F_{\text{PP}}(r) = \frac{Gm_1m_2}{r^2}\left[\text{erfc}\left(\frac{r}{2r_s}\right) + \frac{r}{r_s\sqrt{\pi}}\exp\left(-\frac{r^2}{4r_s^2}\right)\right]$$

Observe the behavior of this equation:

- As $r \to 0$ (particles are very close), the terms in the brackets approach 1.0, recovering the standard $1/r^2$ Newtonian gravity.
- As $r \gg r_s$ (particles are far apart), the erfc and exponential terms rapidly drop to zero, ensuring the PP force vanishes where the PM grid takes over.

The sum of the Gaussian-filtered grid force and this erfc-damped PP force equals the true $1/r^2$ Newtonian force.

Key Literature & Further Reading
Bagla, J. S. (2002). *TreePM: A code for cosmological N-body simulations.* Journal of Astrophysics and Astronomy, 23(4), 185-196. Available at https://arxiv.org/pdf/astro-ph/9911025

An Expanding Space

Up to this point, our simulation has taken place in a static box. This is a good approximation for a star cluster or a single galaxy, but it is fundamentally wrong for a cosmological simulation. Our universe is not static; it is expanding. To accurately model the formation of structure, we must incorporate this expansion into our simulation.

This is achieved by switching from familiar "proper" coordinates to a more abstract but powerful system called **comoving coordinates**. Instead of tracking particles in a fixed box, we track them on a virtual grid that expands along with the universe itself.

The Hubble Flow

The dominant motion in the universe is the cosmic expansion, a phenomenon described by the **Hubble-Lemaître Law**. This law states that, on average, every galaxy is moving away from every other galaxy. The farther away a galaxy is, the faster it appears to recede. This is not a motion *through* space, but rather the expansion *of* space itself. This uniform expansion is the **Hubble Flow**. The velocity of this recession, $\mathbf{v}_{\text{Hubble}}$, for an object at a proper distance $\mathbf{r}$ is given by:

$$\mathbf{v}_{\text{Hubble}}(t) = H(t)\mathbf{r}(t)$$

Where $H(t)$ is the Hubble parameter at time t. This flow is the background upon which all other motions are superimposed.

Comoving Coordinates

Tracking particles whose primary motion is this rapid expansion is computationally difficult. It's much easier to factor out the expansion. We do this by defining a **scale factor**, $a(t)$, which describes the relative size of the universe at any time t. By convention, $a = 1$ today. In the past, a was smaller.

We can now define two types of coordinates:

- **Proper Coordinates (r):** The real, physical distance between two objects that you would measure with a ruler at time t. This distance grows as the universe expands.
- **Comoving Coordinates (x):** The coordinates of an object on our virtual, expanding grid. If an object is moved *only* by the Hubble Flow, its comoving coordinates **do not change**.

The relationship between them is simple:

$$\mathbf{r}(t) = a(t)\mathbf{x}$$

A particle's true velocity is a combination of the Hubble Flow and its own, small-scale motion relative to the expanding grid. This local motion, caused by the gravitational pull of nearby structures, is called the **peculiar velocity**, $\mathbf{v}_{\text{pec}}$.

The Equations of Motion

To understand how the equations of motion change, we start with the standard physical law in **proper coordinates**: a particle's physical acceleration is equal to the true gravitational acceleration at its location.

$$\frac{d^2\mathbf{r}}{dt^2} = \mathbf{g}_{\text{proper}}(\mathbf{r})$$

Here, $\mathbf{g}_{\text{proper}}(\mathbf{r})$ is the "real" gravitational acceleration created by the physical distribution of matter in the expanding universe.

Our goal is to rewrite this equation using **comoving coordinates**, $\mathbf{x}$, which are related by $\mathbf{r}(t) = a(t)\mathbf{x}$. After performing the necessary calculus to account for the fact that the scale factor $a(t)$ is changing over time, we arrive at the new equation of motion for a particle's comoving acceleration, $\frac{d^2\mathbf{x}}{dt^2}$:

$$\frac{d^2\mathbf{x}}{dt^2} = \frac{\mathbf{g}_{\text{comoving}}(\mathbf{x})}{a^3} - 2H(t)\frac{d\mathbf{x}}{dt}$$

Let's break down these two terms, which are the fundamental modifications needed for a cosmological simulation.

- **Modified Gravity:** The first term, $\frac{\mathbf{g}_{\text{comoving}}(\mathbf{x})}{a^3}$, represents the force of gravity. The term $\mathbf{g}_{\text{comoving}}(\mathbf{x})$ is the gravitational acceleration that our force solvers (like PM or PP) calculate—it's the acceleration that would exist in a *static* universe if the particles were at their current comoving positions. The division by the scale factor cubed, a^3, is the crucial cosmological correction. As the universe expands by a factor of a, the volume of any given region increases by a^3. This dilutes the physical density of matter as $\rho \propto 1/a^3$. Since gravity is sourced by density, its strength weakens accordingly, and this term correctly captures that effect.
- **Hubble Drag:** The second term, $-2H(t)\frac{d\mathbf{x}}{dt}$, is a new velocity-dependent "friction" term. The term $H(t)$ is the Hubble parameter ($\frac{1}{a}\frac{da}{dt}$), and $\frac{d\mathbf{x}}{dt}$ is the particle's **peculiar velocity** (its local motion relative to the expanding grid). This "Hubble drag" acts to slow down these peculiar velocities. In an expanding universe, a particle's local motion is constantly being damped by the stretching of space itself.

By solving this new equation of motion, our simulation correctly captures the delicate interplay between the cosmic expansion that tries to pull everything apart and the force of gravity that tries to pull everything together.

Cosmological Models and the Friedmann Equations

The equation of motion we just derived tells us how particles respond to gravity and expansion, but it relies on two crucial background variables: the scale factor, $a(t)$, and the Hubble parameter, $H(t)$. To actually integrate the particle trajectories, our simulation needs to know exactly how these values evolve. Their behavior is not arbitrary; it is dictated by the fundamental laws of General Relativity.

General Relativity is governed by **Einstein's Field Equations**. At their core, these equations describe the delicate balance between the geometry of the cosmos and the "stuff" inside it. They are elegantly summarized in tensor notation:

$$G_{\mu\nu} + \Lambda g_{\mu\nu} = \frac{8\pi G}{c^4} T_{\mu\nu}$$

In this formulation, the left side represents the canvas of spacetime itself: $G_{\mu\nu}$ measures the geometric curvature, $g_{\mu\nu}$ is the metric defining how distances are measured, and Λ is the cosmological constant representing the inherent energy of the vacuum. The right side represents the contents: $T_{\mu\nu}$ is the stress-energy tensor, which tallies up all the mass, light, and fluid pressure in a given region. As physicist John Archibald Wheeler famously summarized: *"Spacetime tells matter how to move; matter tells spacetime how to curve."* However, because this compact line actually hides ten grueling, interconnected differential equations, solving it directly for an irregular, clumpy universe filled with billions of scattered galaxies is mathematically impossible.

In the 1920s, physicist Alexander Friedmann simplified Einstein's field equations for a universe that is assumed to be uniform and isotropic on large scales. The resulting Friedmann equation acts as the master blueprint for cosmic expansion.

In modern cosmology, the geometric cosmological constant (Λ) from Einstein's equations is typically treated as an effective "vacuum energy density" (ρ_Λ). This fluid-like approach allows us to group dark energy alongside normal matter and radiation into a single total density term (ρ_{tot}), yielding a highly elegant and unified equation:

$$H(t)^2 = \left(\frac{1}{a}\frac{da}{dt}\right)^2 = \frac{8\pi G}{3}\rho_{tot} - \frac{kc^2}{a^2}$$

Where:

$$\rho_{tot} = \rho_m + \rho_r + \rho_\Lambda$$
$$\rho_\Lambda = \frac{\Lambda c^2}{8\pi G}$$
$$\rho_r = \frac{\epsilon}{c^2}$$
$$\epsilon = \frac{4\sigma}{c}T^4$$

Here, G is the gravitational constant, k is a constant representing the overall geometric curvature of space, and ρ_{tot} is the combined density of matter (ρ_m), radiation (ρ_r), and the inherent vacuum energy of space itself (ρ_Λ). For the radiation component, ϵ represents the thermodynamic energy density of the photon gas filling the universe, determined by the Stefan-Boltzmann constant (σ), the speed of light (c), and the temperature of the Cosmic Microwave Background (T).

Observations of the real universe strongly indicate that our cosmos is geometrically "flat," meaning $k = 0$. This simplifies the mathematics significantly. In a perfectly flat universe, the expansion rate is exactly balanced by the total energy density. This specific equilibrium point is known as the critical density (ρ_c). Evaluated at any time t, it is defined entirely by the Hubble parameter and the gravitational constant:

$$\rho_c(t) = \frac{3H(t)^2}{8\pi G}$$

Because the critical density acts as the universal balancing point, cosmologists express the composition of the universe using dimensionless density parameters (Ω_m for matter, Ω_r for radiation, and Ω_Λ for dark energy). These are defined simply as the ratio of a given component's density to the critical density:

$$\Omega_m = \frac{\rho_m}{\rho_c}, \quad \Omega_r = \frac{\rho_r}{\rho_c}, \quad \Omega_\Lambda = \frac{\rho_\Lambda}{\rho_c}$$

Under this unified framework, because we live in a flat universe where the total density equals the critical density ($\rho_{tot} = \rho_c$), the sum of these fractions must equal one:

$$\Omega_m + \Omega_r + \Omega_\Lambda = 1$$

For our simulation to be a consistent representation of reality, the mean matter density of our computational grid must equal the matter fraction of this critical density evaluated at the present day (H_0):

$$\bar{\rho}_m = \Omega_m \rho_{c,0}$$

While the complete Friedmann framework accounts for the presence of radiation (Ω_r), it is standard practice in N-body cosmological simulations to set $\Omega_r = 0$. This approximation is justified by the physics of cosmic expansion: while matter density dilutes as a^{-3} due to the increasing volume of space, radiation density dilutes much faster, scaling as a^{-4}, because the expansion of space also stretches the physical wavelength of the photons, and a photon's energy is inversely proportional to its wavelength ($E \propto 1/\lambda$). Consequently, by the time a standard N-body simulation initializes (typically around a redshift of $z = 100$), the gravitational influence of radiation has become mathematically negligible. To streamline the computational integrator, modern codes safely ignore it, operating strictly under the assumption that $\Omega_m + \Omega_\Lambda = 1$.

It is also important to note that the total matter density parameter, Ω_m, is actually a composite of two distinct types of mass: baryonic matter (Ω_b) and dark matter (Ω_{dm}). Baryonic matter accounts for all the "normal" matter in the universe—such as the cosmic gas, stars, and planets—while dark matter represents the invisible, collisionless mass that provides the dominant gravitational framework for structure formation. Mathematically, this is expressed simply as $\Omega_m = \Omega_b + \Omega_{dm}$. In a standard gravity-only N-body simulation, our particles represent this combined total mass. However, when we later introduce hydrodynamics to the code, we must explicitly separate these fractions, as the baryonic gas experiences fluid pressure and thermal dynamics, while the dark matter continues to respond exclusively to gravity.

A **cosmological model** is simply a specific "recipe" of these cosmic ingredients. By defining what our virtual universe is made of (the total density ρ and its composition (Ω_m, Ω_r and Ω_Λ) and plugging them into the Friedmann equation, we can mathematically solve for the exact historical trajectory of the expansion.

For the purposes of our simulation, there are two primary models of interest: the classic, matter-dominated model (Einstein-de Sitter) and the modern, dark-energy-driven model (ΛCDM).

An Einstein-de Sitter Universe

For a simulation to be physically meaningful, it must be based on a self-consistent cosmological model. The simplest and most classic model for a matter-dominated universe is the **Einstein-de Sitter (EdS)** model. This is a specific solution to Einstein's Friedmann equations that describes a flat, expanding universe containing only matter ($\Omega_m = 1$) and no dark energy ($\Omega_\Lambda = 0$):

$$H(t)^2 = \frac{8\pi G}{3}\rho(t)$$

In an EdS universe, the expansion of space is constantly being decelerated by the gravitational pull of its own mass. This physical reality is described by a simple power-law relationship between the scale factor, $a(t)$, and cosmic time, t:

$$a(t) \propto t^{2/3}$$

From this, the Hubble parameter also becomes a simple function of time:

$$H(t) = \frac{1}{a(t)}\frac{da(t)}{dt} = \frac{2}{3t}$$

The ΛCDM Model

The Einstein-de Sitter (EdS) model is mathematically elegant and perfectly describes a universe dominated entirely by the gravity of matter. For decades, it was the standard model of cosmology. However, in 1998, observations of distant supernovae revealed a shocking truth: the expansion of our universe is not slowing down due to gravity; it is accelerating.

To model the real universe, we must upgrade from EdS to the **ΛCDM (Lambda Cold Dark Matter)** model. This model introduces a new component to the cosmos: **Dark Energy**, represented by the cosmological constant, Λ. Dark energy acts as a repulsive negative pressure inherent to space itself, pushing the universe apart.

In a flat ΛCDM universe, the total density is made up of matter ($\Omega_m \approx 0.3$) and dark energy ($\Omega_\Lambda \approx 0.7$), such that $\Omega_m + \Omega_\Lambda = 1$. The Friedmann equation expands to include this new term:

$$H(t)^2 = H_0^2\left(\frac{\Omega_m}{a(t)^3} + \Omega_\Lambda\right)$$

Where H_0 is the **Hubble Constant**—the exact rate at which the universe is expanding right now, that is, the value of the Hubble parameter ($H(t)$) measured at the present.

Notice that the matter density dilutes as the universe expands ($1/a^3$), but the dark energy density (Ω_Λ) remains perfectly constant. This creates a fascinating cosmic tug-of-war. In the early universe, when $a(t)$ was very small, the dense matter term completely dominated the Friedmann equation. The universe behaved almost exactly like an EdS model, decelerating as gravity pulled matter together. However, as space expanded and matter diluted, the constant outward push of dark energy eventually overtook the fading pull of gravity. Today, dark energy dominates, and the expansion is accelerating.

The exact solution for the scale factor $a(t)$ in a flat ΛCDM universe gracefully captures both of these eras—the early deceleration and the late acceleration—using a hyperbolic sine function:

$$a(t) = \left(\frac{\Omega_m}{\Omega_\Lambda}\right)^{1/3} \sinh^{2/3}\left(\frac{3}{2}H_0\sqrt{\Omega_\Lambda}t\right)$$

By using this equation, along with its corresponding Hubble parameter $H(a)$, our simulation smoothly transitions from the matter-dominated epoch (where structures rapidly form) into the dark-energy-dominated epoch (where the cosmic web is stretched and frozen in place).

Dark energy actively fights against the formation of galaxies. In a pure matter universe, structures grow steadily forever. But in a ΛCDM universe, the accelerated stretching of space pulls matter apart faster than gravity can pull it together, eventually halting the hierarchical growth of the cosmos.

General Relativity and Cosmic Expansion

In our simulation model, the scale factor $a(t)$ is a global variable. It multiplies the distance between every single coordinate in the comoving grid equally. When dark matter clumps together to form a dense halo, the grid underneath it continues to expand, and the particles must generate inward "peculiar velocities" to fight against this background stretching and remain gravitationally bound.

However, under Einstein's General Relativity, **space does not expand independently of the matter inside it.** In this section we will understand why our simulation's approach still yields the correct physics.

To begin, we have to look at how General Relativity actually defines the shape of space. This is governed by the Einstein Field Equations:

$$G_{\mu\nu} + \Lambda g_{\mu\nu} = \frac{8\pi G}{c^4} T_{\mu\nu}$$

The right side of the equation ($T_{\mu\nu}$) tallies up the mass and energy in a given region. The left side describes how the geometry of spacetime bends and stretches in response. The foundational building block of that geometry is the metric tensor, $g_{\mu\nu}$.

In simple terms, a metric is a mathematical ruler. It defines exactly how physical distances and time intervals are measured in a curved universe.

In standard Newtonian physics, if we want to measure the spatial distance (ds) between two points on a flat, static grid, we use the 3D Pythagorean theorem: $ds^2 = dx^2 + dy^2 + dz^2$. However, if those coordinates are sitting in a spacetime warped by gravity or cosmic expansion, that flat-space formula no longer works. The metric tensor ($g_{\mu\nu}$) acts as the correction factor, modifying the Pythagorean theorem to account for the stretching of space and the dilation of time.

Mathematically, this relationship is expressed by multiplying the metric tensor against the differential coordinate steps (dx^μ and dx^ν). Using the Einstein summation convention—where repeating indices imply a sum over all four spacetime dimensions (one for time, three for space)—the universal formula for measuring spacetime intervals is written as:

$$ds^2 = g_{\mu\nu} dx^\mu dx^\nu$$

We can think of $g_{\mu\nu}$ as a 4×4 matrix, and the coordinates as a vector: (dt, dx, dy, dz). In a perfectly flat, empty universe (the "Minkowski" metric of Special Relativity), this matrix is incredibly simple. It consists of the speed of light for the time component, 1s for the spatial components, and 0s everywhere else:

$$g_{\mu\nu} = \begin{pmatrix} -c^2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

If we multiply this matrix through the ds^2 equation, the zeros eliminate all the cross-terms, and we get the classic flat-space distance formula:

$$ds^2 = -c^2 dt^2 + dx^2 + dy^2 + dz^2$$

If we freeze time so that the clock isn't ticking ($dt = 0$), this perfectly recovers the standard 3D Pythagorean theorem.

But when mass or cosmic expansion is introduced, the components of this $g_{\mu\nu}$ matrix change. The 1s might be replaced by time-dependent variables, or the 0s might become complex fractions. This alters the underlying geometry, meaning the exact same coordinate points will yield a completely different physical distance, ds.

When cosmologists solve Einstein's equations, the ultimate goal is finding the correct metric—the correct "ruler"—for the specific arrangement of mass they are studying. When we look at the universe, we are confronted with two very different arrangements of mass, which require two entirely different mathematical rulers.

The FLRW Metric: A Uniform Universe

To model the global expansion of the universe, cosmologists use a foundational solution to Einstein's equations known as the **FLRW metric** (Friedmann–Lemaître–Robertson–Walker). To derive this metric, we assume the universe is a perfectly smooth, uniform fluid with no localized clumps, stars, or galaxies. Solving the field equations for this uniform mist yields the following geometry:

$$ds^2 = -c^2dt^2 + a(t)^2(dx^2 + dy^2 + dz^2)$$

In this equation, the spatial coordinates (dx, dy, dz) are multiplied by the scale factor $a(t)$. This is the rigorous mathematical origin of expanding space: because the scale factor depends on time, the physical distance (ds) between two stationary coordinate points strictly increases as the clock ticks. This perfectly describes the vast, empty cosmic voids between distant galaxy clusters.

The Schwarzschild Metric: A Clumped Universe

However, the real universe is not a perfectly smooth mist. Matter collapses into dense, localized structures like stars, galaxies, and clusters, leaving vast empty vacuums between them. If we plug a dense, spherical clump of mass into the Einstein Field Equations surrounded by empty space, the FLRW metric is no longer a valid solution. Instead, the geometry of space is described by the **Schwarzschild metric**:

$$ds^2 = -\left(1 - \frac{2GM}{rc^2}\right)c^2dt^2 + \left(1 - \frac{2GM}{rc^2}\right)^{-1}dr^2 + r^2(d\theta^2 + \sin^2\theta d\phi^2)$$

Crucially, **there is no time-dependent scale factor $a(t)$ attached to the spatial coordinates** (dr, $d\theta$, $d\phi$). The metric depends solely on the mass (M) and the radius from it (r).

Mathematically, this proves that the space inside a dense, gravitationally bound system is completely static. The space between the Earth and the Sun, or between stars in the Milky Way, is not expanding. Gravity does not "compensate" for expansion in these regions; the expansion simply does not exist within this localized geometry. Physicists often model the universe as an "Einstein-Straus vacuole"—a "Swiss cheese" universe where the cheese is the smooth, expanding FLRW space, and the holes are static, non-expanding Schwarzschild regions where mass has concentrated.

Validity of the Simulation's Approximation

This brings us to the core contradiction of an N-body simulation. Our model explicitly applies the expanding FLRW ruler ($a(t)$) to every single coordinate cell in the grid universally. According to strict General Relativity, that's fundamentally wrong: we are forcing space to expand inside our simulated galaxy clusters. To keep those clusters from ripping apart, our Newtonian integrator generates artificial "peculiar velocities," making the particles physically swim inward to constantly fight against a background expansion that shouldn't even be there.

How can a computational model violating relativistic geometry accurately simulate the cosmos? The answer requires a mathematical bridge that proves our Newtonian "cheat" actually mimics the true geometry of spacetime: **Cosmological Perturbation Theory**.

Cosmological Perturbation Theory

The Newtonian approximation is not an exact, universal equivalence to General Relativity; rather, it is a highly constrained, rigorously proven limit. There is a mathematical bridge—**Cosmological Perturbation Theory**—that demonstrates exactly how and when Einstein's equations simplify into our model.

In General Relativity, we cannot blindly superimpose Newtonian gravity on top of the expanding FLRW universe. Instead, physicists define a metric that represents an expanding universe containing slight localized "wrinkles" or "dimples" of gravity. This is known as the **Conformal Newtonian Gauge**:

$$ds^2 = -\left(1 + \frac{2\Phi}{c^2}\right)c^2dt^2 + a^2(t)\left(1 - \frac{2\Phi}{c^2}\right)(dx^2 + dy^2 + dz^2)$$

Notice the composition of this metric:

1. The global expansion factor $a^2(t)$ still multiplies the spatial coordinates.

2. We have introduced Φ, the local gravitational potential representing the "wrinkles" in spacetime caused by clumps of dark matter.
3. The speed of light, c, is explicitly included to anchor the relativity.

To prove that our N-body simulation is valid, cosmologists insert this "wrinkled" metric into the complex Einstein Field Equations and apply three strict mathematical limits to mirror the conditions of a standard galaxy cluster:

1. **Weak Gravity:** The local gravitational potential is tiny compared to the speed of light squared ($\Phi \ll c^2$).
2. **Slow Motion:** The peculiar velocities of the particles are much slower than the speed of light ($v \ll c$).
3. **Sub-Horizon Scales:** The comoving size of the simulation domain (L) is much smaller than the observable universe ($L \ll c/H$).

When these three limits are applied, the massive complexity of General Relativity collapses. The vast majority of the relativistic terms safely cancel out to zero. What remains is a single, elegant equation defining how the gravitational potential Φ behaves within the expanding grid:

$$\nabla^2\Phi = 4\pi G a^2(\rho - \bar{\rho})$$

This is the cosmological Poisson equation, and it is the exact mathematical foundation of our gravity solver. Notice the right side of the equation: ρ is the actual density in a local region, and $\bar{\rho}$ is the average background density of the universe. The term $(\rho - \bar{\rho})$ proves that in an expanding universe, gravity is only generated by the *overdensity*—the amount of mass that exceeds the background average.

Historically, simply subtracting this background density was a controversial mathematical trick known as the "Jeans Swindle." In 1902, physicist James Jeans tried to calculate the gravitational collapse of gas in a static, infinite universe using Newtonian physics. He immediately ran into a mathematical paradox: according to strict Newtonian rules, an infinite universe filled with a uniform background mass ($\bar{\rho}$) would generate an infinite gravitational potential, breaking the equations entirely. To force his calculations to work, Jeans simply "swindled" the math. He arbitrarily declared that the uniform background mass exerted no net gravity, explicitly ignored it, and applied Poisson's equation only to the local density variations.

For decades, this was viewed as a mathematical cheat because it is strictly illegal in a static Newtonian framework. However, the perturbation theory we just applied provides the rigorous justification for using the Jeans Swindle in our model. In the limit of slow-moving particles and weak gravity, the shifting spacetime of the universe mathematically reduces to a modified Newtonian Poisson equation where the background density is naturally and correctly subtracted. The expansion of space acts as a perfect mathematical sink for the background mass, leaving only the local overdensities to drive the structural collapse.

Limitations of the approximation

Because our model is built upon these three limits, it inherently defines the boundaries of what it can physically simulate. If we violate them, the Newtonian approximation does not just lose accuracy; it fundamentally misrepresents the physics:

- **Breaking the slow-motion limit ($v \ll c$):** If a particle were accelerated to a significant fraction of the speed of light (e.g., material ejected in a relativistic jet), our Newtonian integrator would calculate the wrong trajectory. Under Newtonian mechanics, a continuous force causes continuous acceleration, which would eventually push our simulation particles faster than the speed of light. In reality, relativity dictates that as an object approaches c, its momentum scales non-linearly, requiring infinite energy to accelerate further. Our code lacks the Lorentz transformations required to enforce this cosmic speed limit.
- **Breaking the weak gravity limit ($\Phi \ll c^2$):** If a supermassive black hole formed in our box, the local gravitational potential Φ would approach c^2. Our code operates on the assumption that space is a flat, rigid Cartesian grid, and that gravity is merely a force vector pulling particles through it. In reality, extreme gravity severely warps the geometry of spacetime, creating phenomena like event horizons and severe local time dilation. A simple $1/r^2$ Newtonian force calculation completely fails to capture the complex, non-Euclidean behavior of matter falling into deeply curved gravitational wells.
- **Breaking the sub-horizon limit ($L \ll c/H$):** If we attempted to simulate a massive domain 10,000 Megaparsecs across, our treatment of large-scale gravity would fail. Our code's Poisson solver calculates the gravitational potential of the entire grid at a single, frozen timestep. Inherently, it assumes gravity travels at infinite speed. For a 100 Mpc box, the light-travel delay is negligible compared to the slow movement of galaxies. But for a 10,000 Mpc box, our instantaneous calculation would physically violate causality, allowing a galaxy on one side of the universe to immediately feel the gravitational pull of a cluster on the far opposite side. To prevent faster-than-light gravity over vast cosmic distances, we would

have to abandon the simple Poisson solver and use relativistic "retarded potentials" that account for the billions of years it takes for gravitational ripples to travel.

However, for a 100 Mpc domain forming standard dark matter halos and galactic filaments, the Newtonian approximation holds so tightly that the dynamic difference between our code and a supercomputer solving raw General Relativity is practically indistinguishable.

Key Literature & Further Reading
Springel, V. (2005). The cosmological simulation code GADGET-2. *Monthly Notices of the Royal Astronomical Society*, 364(4), 1105-1134. Available at: https://arxiv.org/abs/astro-ph/0505010

Green, S. R., & Wald, R. M. (2012). Newtonian and relativistic cosmologies. *Physical Review D*, 85. Available at: https://arxiv.org/pdf/1111.2997

Natural Units

To simplify the implementation, and for numerical stability, it is standard practice to work in a system of **natural units**. Any system of units requires three fundamental base quantities: Mass, Length, and Time.

It is standard convention in modern cosmological codes to define define the mass and length units by setting the total mass of the system to $M_{total} = 1$ and the comoving side length of the box to $L = 1$. It is also universally standard to set the present-day scale factor to $a_0 = 1$.

To complete the system, we must define a code unit of time. While we could theoretically choose any arbitrary duration (such as one billion physical years), doing so would cause our calculated value for G to change every time we simulate a different cosmology. Instead, we want to define a time unit that mathematically absorbs the cosmological parameters, allowing G to remain a permanent, universal constant in our code.

To do this, we base our time unit on the fundamental dynamical timescale of a purely matter-dominated universe. In a classic Einstein-de Sitter model, the physical age of the universe is exactly $\frac{2}{3H_0}$. We adapt this foundational timescale to serve as our base code unit of time (t_{unit}) by explicitly dividing it by the square root of the matter fraction:

$$t_{unit} = \frac{2}{3H_{0,\text{phys}}\sqrt{\Omega_m}}$$

Because the physical Hubble parameter has units of inverse time ($1/t$), converting it into our code's internal time units ($H_{0,\text{code}} = H_{0,\text{phys}} \times t_{unit}$) forces the numerical value of the expansion rate in our simulation to become exactly:

$$H_0 = \frac{2}{3\sqrt{\Omega_m}}$$

By anchoring our time unit in this very specific way, we have intentionally entangled the internal expansion rate with the matter density. As we will see, this deliberate mathematical choice acts as a perfect counterweight in the Friedmann equations, allowing us to lock the strength of gravity to a single, unchanging number across all flat cosmological models.

It may seem counterintuitive that we derived our fundamental time unit from the mathematics of an Einstein-de Sitter (matter-only) universe, especially when modern simulations almost exclusively model ΛCDM (dark-energy-dominated) universes. However, this is a deliberate computational trick. Dark energy is a perfectly smooth vacuum energy; it does not cluster, and therefore it does not enter the Poisson equation for local gravity. By defining our time unit using the EdS timescale, we cleanly isolate the matter density (Ω_m) and absorb it into the time variable, leaving the gravity solver completely independent of the cosmology. The complex effects of dark energy are handled entirely elsewhere in the code—specifically, within the calculation of the global background expansion, $a(t)$.

Deriving the Gravitational Constant (G)

To derive the value of the gravitational constant (G) required for our code, we must ensure that the mean density of our simulation grid matches the physical matter density of a flat universe. As established, this physical matter density is a fraction of the critical density: $\bar{\rho}_m = \Omega_m \rho_c$.

In our natural unit system, the present-day Hubble parameter is defined as $H_0 = \frac{2}{3\sqrt{\Omega_m}}$ to ensure the correct timeline for cosmic expansion. If we substitute this definition into the physical matter density equation, a mathematical cancellation occurs:

$$\bar{\rho}_m = \Omega_m \left(\frac{3\left(\frac{2}{3\sqrt{\Omega_m}}\right)^2}{8\pi G} \right)$$

When we expand the squared Hubble term, the Ω_m in the denominator perfectly cancels out the Ω_m scaling the equation:

$$\bar{\rho}_m = \Omega_m \left(\frac{3\left(\frac{4}{9\Omega_m}\right)}{8\pi G} \right) = \frac{12}{72\pi G} = \frac{1}{6\pi G}$$

Now, we look at the computational side. The mean density of our simulation box is defined by its total mass divided by its comoving volume:

$$\bar{\rho}_m = \frac{M_{total}}{L^3}$$

Equating our mean density to the physical matter density gives us our final consistency relation for G:

$$\frac{M_{total}}{L^3} = \frac{1}{6\pi G}$$

$$G = \frac{L^3}{6\pi M_{total}}$$

Because we chose $M_{total} = 1$ and $L = 1$ for our natural units, this simplifies to a final, permanent constant:

$$G = \frac{1}{6\pi}$$

The Ω_m parameter drops out of the calculation. Because the code's definition of the expansion rate (H_0) inherently absorbs the matter fraction, the required strength of gravity in our simulation units becomes a universal constant. For any flat cosmology—whether it is a simple matter-only EdS model or a complex dark-energy-driven ΛCDM model—this exact value for G ensures that the gravitational pull in our code is perfectly balanced against the expansion of space.

Hubble: Physical vs. Code Units

Because we intentionally constructed our time unit to force our internal expansion rate to $H_0 = \frac{2}{3\sqrt{\Omega_m}}$, a sharp reader might wonder how we input the *actual* observed expansion rate of the universe. In observational cosmology, the matter density (Ω_m) and the true physical Hubble constant are completely independent parameters.

To resolve this, cosmological codes split the concept of the Hubble parameter into two distinct roles:

- **The Physical Hubble Parameter (h):** In astronomy, the real-world Hubble constant is traditionally written as $H_{0,\text{phys}} = 100 \cdot h$ km/s/Mpc, where h is a dimensionless scaling factor (typically around 0.7). This parameter represents the true physical expansion speed. It is used exclusively during the setup and analysis phases to calculate the real physical size of the primordial density fluctuations, set the initial conditions, and translate the final simulation outputs back into standard physical units like Megaparsecs and Kelvin.
- **The Internal Code-Unit Hubble Parameter (H_0):** Once the simulation starts integrating, it stops thinking in kilometers and seconds. It strictly uses the derived internal rate ($H_0 = \frac{2}{3\sqrt{\Omega_m}}$) to step the time clock forward, stretch the scale factor $a(t)$, and apply Hubble drag to the particles on the grid.

By managing this strict separation, the code's internal gravity solver remains mathematically elegant and constant, while still allowing the user to configure their initial conditions with any real-world combination of Ω_m and h they choose.

Translating Natural Units to Physical Reality

While natural units keep the computer's floating-point math stable by bounding variables between 0.0 and 1.0, we eventually need to translate our simulation data back into Megaparsecs, Solar Masses, and Gigayears to compare our "mock universe" with actual telescope observations.

Because we locked our natural unit system to the fundamental cosmological expansion parameters (Ω_m and H_0), we can rigorously derive the exact physical value of 1.0 code unit for Length, Mass, Time, and Velocity.

1. The Length Unit ($[L]$) The length unit is the foundational choice of the simulation, representing the comoving size of the virtual patch of space we wish to model. If we decide our simulation box represents a region 100 Megaparsecs across, then 1.0 code unit of distance is strictly defined as $L_{box} = 100$ Mpc.

Because this is a comoving length, the coordinate system expands with the universe. The *physical* distance measured by a hypothetical tape measure at any point in the simulation is simply $a(t) \times L_{box}$.

2. The Mass Unit ($[M]$) Because we defined the total mass of our system as $M_{total} = 1$, one unit of code mass represents the total physical mass of the entire simulated universe.

To calculate this in Solar Masses ($M_\odot$), we must find the total matter density of the present-day universe and multiply it by the comoving volume of our box. The background matter density is the critical density, derived from fundamental constants:

$$G = 6.674 \times 10^{-11} \text{ m}^3 \text{ kg}^{-1} \text{ s}^{-2}$$

$$H_0 = 100h \text{ km s}^{-1} \text{ Mpc}^{-1}$$

$$\rho_{crit,0} = \frac{3H_0^2}{8\pi G} \approx 2.775 \times 10^{11} h^2 \ M_\odot/\text{Mpc}^3$$

It is important to remember that the critical density ($\rho_{crit,0}$) represents the total mass-energy threshold required to make the universe's spatial geometry perfectly flat. In a standard ΛCDM universe, matter only makes up a fraction of this total energy budget (typically $\Omega_m \approx 0.3$), with dark energy making up the remainder. Because our simulation exclusively tracks mass, we cannot use the raw critical density to weigh our universe. We must multiply it by Ω_m to isolate the actual physical matter density residing inside our simulated box:

$$[M] = \Omega_m \rho_{crit,0} L_{box}^3$$

If a single dark matter particle in the code has a mass of m_p, its physical mass is simply $m_p \times [M]$.

3. The Time Unit ($[T]$) To find out how many real years pass when our code's internal clock ticks from $t = 0.0$ to $t = 1.0$, we rely on the foundational time unit we defined earlier to lock our gravitational constant. As established, our code unit of time is defined by the physical Hubble parameter and the matter fraction:

$$[T] = \frac{2}{3H_{0,phys}\sqrt{\Omega_m}}$$

The physical Hubble parameter ($H_{0,phys}$) is typically expressed as $100h$ km s^{-1} Mpc^{-1}. To get our time unit in Gigayears (Gyr), we convert $H_{0,phys}$ to $\approx 0.10227h$ Gyr^{-1}. Substituting this into our equation gives us our exact translation factor:

$$[T] = \frac{2}{3(0.10227h)\sqrt{\Omega_m}} \text{ Gyr}$$

4. The Velocity Unit ($[V]$) Because velocity is kinematically derived from distance over time, fixing our Length and Time units strictly dictates our internal Velocity unit.

$$[V] = \frac{[L]}{[T]}$$

Using the derivations above, the Megaparsecs neatly cancel out, yielding an exact conversion factor to translate code velocities into kilometers per second:

$$[V] = 150 \cdot h \cdot \sqrt{\Omega_m} \cdot L_{box} \text{ km/s}$$

Once these four fundamental units are established, all other physical quantities in the simulation—such as the internal energy of the gas, pressure, and temperature—can be seamlessly derived using standard dimensional analysis.

The Cosmic Timeline

The Scale Factor (a) and Redshift (z)

Before we look at the timeline of the universe, we need to define how cosmologists actually tell time. While it is intuitive to talk about "years since the Big Bang," it is incredibly difficult to calculate. Instead, astronomers rely on two interlocked variables to track the evolution of the cosmos: the scale factor (a) and cosmological redshift (z).

The Scale Factor (a) As we established, $a(t)$ is the mathematical engine of the universe's expansion. It tracks the relative, physical size of the coordinate grid. By convention, the scale factor today is set to exactly $a = 1$. When the universe was a quarter of its current size, $a = 0.25$. As we run our code forward from the early universe, a continuously ticks upward toward 1.

Cosmological Redshift (z) While a is perfect for writing computer code, an astronomer cannot point a telescope at the sky and directly measure the "size of the coordinate grid." What they *can* measure is light.

As a photon travels across the universe to reach our telescopes, the space it is traveling through is expanding. This expansion physically stretches the photon's wavelength, shifting its color toward the red end of the spectrum. We call this stretching **Cosmological Redshift**, denoted by the letter z.

The Mathematical Link Because the stretching of the light is tied exactly to the stretching of space itself, redshift and the scale factor are inversely related by a very simple, but universally important equation:

$$a = \frac{1}{1+z}$$

Because redshift is the actual, tangible thing we measure in the real world, it has become the universal "clock" for the history of the universe. In literature we will almost always see time denoted by z:

- $z = 0$**:** Today ($a = 1$).
- $z = 1$**:** The universe was exactly half its current size ($a = 0.5$).
- $z = 9$**:** The universe was 1/10th its current size ($a = 0.1$).
- $z = 49$**:** A typical starting time for generating simulation's initial conditions ($a = 0.02$).
- $z = 1100$**:** The release of the Cosmic Microwave Background ($a \approx 0.0009$).
- $z \to \infty$**:** The Big Bang ($a \to 0$).

When reading cosmological texts, we can think of redshift as a measure of looking backward: the higher the z, the further back in time we are looking, the further away the object is, and the smaller and denser the universe was when that light was emitted.

Eras and epochs

In cosmology, we divide the 13.8-billion-year history of the universe into distinct "eras." These divisions are not arbitrary historical chapters; they are strictly defined by the mathematics of the Friedmann equation.

$$H^2(a) = H_0^2 \left(\Omega_{r,0} a^{-4} + \Omega_{m,0} a^{-3} + \Omega_{k,0} a^{-2} + \Omega_{\Lambda,0} \right)$$

Whichever component of the universe (radiation, matter, or dark energy) has the highest energy density dictates the expansion rate of space and the physical rules for how structures form.

For the purposes of cosmological simulations, the universe's history is defined by three major eras:

The Radiation-Dominated Era (The Big Bang to ~50,000 Years)

In the very early universe, space was incredibly small, dense, and unimaginably hot. During this time, the universe's energy budget was completely dominated by the kinetic energy of photons and relativistic particles (neutrinos).

- **The Physics:** In the early universe, the initial rate of spatial expansion triggered by the Big Bang was still very high. Even though the immense density of the radiation was acting as a gravitational "brake" to slow the universe down, the speed of expansion was still blistering. This rapid expansion acted like a cosmic treadmill: space stretched the dark matter particles away from each other much faster than their local gravity could pull them together. Consequently, the local gravitational collapse of dark matter was completely frozen—a phenomenon known as the **Mészáros effect**. It was only after the universe expanded enough for this "treadmill" to significantly slow down that local gravity finally took control and began building cosmic structures.

- **Simulation Context:** We rarely simulate this era directly with N-body codes. Instead, its effects are mathematically "baked in" to our initial conditions. The suppression of small-scale structures during this era is exactly what creates the mathematical curve of the **BBKS Transfer Function** (explained in a later section).
- **The Transition:** As the universe expanded, radiation diluted much faster than physical matter. Around 50,000 years after the Big Bang, the density of radiation dropped below the density of matter, marking a shift in cosmic physics. Shortly after this transition (at about 380,000 years, or a redshift of roughly z = 1100), the universe cooled enough for the first neutral atoms to form, releasing the Cosmic Microwave Background (CMB).

The Matter-Dominated Era (~50,000 Years to ~9.8 Billion Years)

Once the radiation diluted, the gravity of cold dark matter and baryonic gas took control of the energy budget. Unlike radiation, cold matter acts like a cosmic "dust"—it has mass, but it exerts practically zero large-scale macroscopic pressure. The global expansion rate dropped low enough that local gravity could finally fight back. Dark matter overdensities decoupled from the expanding background and collapsed inward, initiating the bottom-up, hierarchical assembly of the cosmic web.

- **The Physics:** This is the golden age of structure formation. With radiation pressure gone and the expansion slowing, gravity was finally free to pull matter together. The tiny primordial ripples left over from the Big Bang collapsed into the cosmic web, forming the first stars, galaxies, and galaxy clusters.
- **Simulation Context:** This era is the primary sandbox for computational cosmology. Our simulations typically start right near the beginning of this era (e.g., at redshift z = 49). Because dark energy is negligible here, the universe behaves like a simple Einstein-de Sitter model where the linear growth factor scales perfectly with the size of the universe: $D(t) \propto a(t)$.
- **Key Epochs:** Inside this era, hydrodynamic simulations track two vital milestones:
  - *The Dark Ages:* The time before the first stars ignited, when the universe was filled with cold, neutral hydrogen gas.
  - *Cosmic Dawn & The Epoch of Reionization:* The violent period when the first massive stars and quasars (actively feeding supermassive black holes at the centers of infant galaxies) ignited. These objects emitted such intense ultraviolet radiation that they blasted the surrounding cold, neutral hydrogen back into a hot plasma. This process started locally, blowing expanding "bubbles" of ionized gas around the UV emitters. Over hundreds of millions of years, these bubbles grew and merged until the entire intergalactic medium was completely reionized, fundamentally changing the fluid dynamics and thermal pressure of the universe.

The Dark Energy-Dominated Era (~9.8 Billion Years to Present)

Matter dilutes as the universe expands, but the density of Dark Energy (the cosmological constant, Λ) remains perfectly constant. About 9.8 billion years after the Big Bang (around redshift z = 0.3), the density of matter finally diluted so much that Dark Energy became the dominant force in the cosmos.

- **The Physics:** Dark Energy acts as a repulsive pressure inherent to the vacuum of space. Under its dominance, the expansion of the universe stopped slowing down and began to violently accelerate.
- **Simulation Context:** This late-time acceleration physically stretches the cosmic web apart faster than gravity can pull it together. This marks the epoch where the growth factor $D(t)$ stalls out and stops tracking the scale factor. Large-scale mergers between galaxy clusters become increasingly rare, and the largest structures begin to freeze in place.

A Note on the "Initial Kick" of the Big Bang

Throughout these eras, we describe the universe's expansion as a continuous fight against the immense gravitational brakes of radiation and matter. However, this raises a profound physical paradox: if the early universe was so unimaginably dense, standard General Relativity dictates it should have immediately collapsed into a black hole under its own weight. So, where did the initial outward expansion come from?

Standard General Relativity cannot answer this question. If we run the Friedmann equations backward, they perfectly describe how the universe behaves *after* the expansion started, but they offer absolutely no physical mechanism for what caused the initial "kick." In classical Big Bang cosmology, this impossibly fast initial expansion cannot be derived; it is simply accepted as a given initial condition. While modern theoretical physics attempts to solve this paradox with models like **Cosmic Inflation**—a brief, pre-radiation epoch where quantum fields exerted a temporary repulsive anti-gravity—standard cosmological codes do not need to simulate these

quantum mechanics. For the purposes of computational cosmology, we simply accept the universe's initial outward expansion rate as a foundational mathematical starting line.

Initial Conditions

The outcome of a cosmological simulation depends on its starting point. To accurately model our universe, we must generate a physical snapshot that captures the primordial density fluctuations that seeded all future cosmic structure.

Fortunately, in the early universe, these density fluctuations were tiny. Because the variations in the gravitational field were so weak and smooth, the initial clustering of matter was entirely **linear**. This is a massive advantage for computational cosmology: it means we do not need to waste computational resources running an N-body solver from the very beginning of time.

Instead, we can fast-forward through the earliest epochs using an analytical framework called the **Zel'dovich Approximation**. Because the physics were still linear, this mathematical approximation can predict how those tiny primordial ripples displaced matter over millions of years.

However, as matter gradually clumps together, local gravity becomes stronger and non-linear. Eventually, dark matter streams cross and gas violently collides. At this point, the linear Zel'dovich approximation breaks down. That breaking point is our simulation's starting line: we use the Zel'dovich approximation to instantly generate the universe's geometry right up to the edge of the linear regime—typically around a scale factor of $a = 0.02$, roughly 50 million years after the Big Bang. Where this analytical approximation starts to fail, our numerical solvers take over to compute the chaotic, non-linear birth of galaxies.

The Unperturbed State

To represent the nearly uniform matter distribution of the early universe, we begin by placing particles on a uniform cubic lattice. For a simulation with N particles in a cubic box of side length L, the initial grid position, $\mathbf{x}_{\text{grid}}$, of a particle is determined by its integer indices (i, j, k).

The position is calculated by determining the number of particles per side, N_s, the spacing between them, d, and then placing each particle at the center of its virtual cubic cell.

The number of particles per side is:

$$N_s = N^{1/3}$$

The spacing between grid points is:

$$d = \frac{L}{N_s}$$

The position vector, $\mathbf{x}_{\text{grid}}$, for the particle at indices (i, j, k) is then given by its components:

$$x = \left(i + \frac{1}{2}\right) d$$

$$y = \left(j + \frac{1}{2}\right) d$$

$$z = \left(k + \frac{1}{2}\right) d$$

Where the indices i, j, and k each run from 0 to $N_s - 1$. The addition of $1/2$ ensures that each particle is placed in the center of its cell, rather than at the corner.

The Zel'dovich Approximation

To create the seeds of galaxies and clusters, we must apply small, correlated **perturbations** to the particle lattice. The **Zel'dovich Approximation** generates a smooth displacement field to "nudge" each particle from its perfect grid position.

It is important to understand the role of the Zel'dovich Approximation in this context. We use it *only once* to generate a single, self-consistent snapshot of the universe at our chosen start time, $t_{initial}$. This snapshot provides both the initial particle positions and their corresponding initial peculiar velocities. From that moment forward, the N-body simulation takes over, calculating the full, non-linear evolution of these particles on its own.

Mathematically, the Zel'dovich Approximation is an application of **first-order Lagrangian perturbation theory (1LPT)**. For higher accuracy more advanced schemes such as **second-order Lagrangian perturbation theory (2LPT)** are often employed, but not covered in this text.

The Growth Factor

Although we only use it to generate a single initial snapshot, the Zel'dovich Approximation is a dynamic theory in which the displacement field, $\mathbf{\Psi}$, is not constant. As the universe evolves, the tiny initial overdensities attract more matter, causing the perturbations to grow stronger. In the linear regime, the spatial pattern of the displacement field remains fixed, while its amplitude grows over time. This growth is described by a single function of time, the **linear growth factor**, $D(t)$.

The full, time-dependent displacement field can therefore be written as:

$$\mathbf{\Psi}(\mathbf{x}, t) = D(t)\mathbf{\Psi}_0(\mathbf{x})$$

Here, $\mathbf{\Psi}_0(\mathbf{x})$ is the primordial displacement pattern at some reference time (conventionally, today, where $a = 1$ and $D = 1$), and $D(t)$ scales this entire pattern up or down depending on the cosmic epoch. In a simple Einstein-de Sitter universe model (or a very early ΛCDM, e.g., $a = 0.02$), the growth factor is conveniently proportional to the scale factor, $D(t) \propto a(t)$.

It is crucial to understand that $\mathbf{\Psi}_0(\mathbf{x})$ does not describe the actual highly non-linear universe today. Rather, it is a linearly extrapolated field: a mathematical representation of what the universe would look like today if gravity had remained linear for 13.8 billion years.

The $D(t)\mathbf{\Psi}_0(\mathbf{x})$ separability is very powerful. It means we only need to compute $\mathbf{\Psi}_0(\mathbf{x})$ once, anchoring it to present-day observational parameters (like σ_8). To generate the actual state of the universe at our starting epoch, we simply scale this pattern backward in time by multiplying it by the appropriate value of $D(t)$. Because the fluctuations at this early time are tiny, the linear Zel'dovich approximation remains accurate, providing the initial conditions for our particles before gravity drives them into non-linear collapse.

Generating the Displacement Pattern

The process of generating the spatial pattern, $\mathbf{\Psi}_0(\mathbf{x})$, begins in Fourier space with the **power spectrum**, $P(k)$. The power spectrum is the statistical recipe for our universe's initial conditions, specifying the amplitude of density fluctuations (density contrast) at different spatial scales, or wavenumbers (k).

The density contrast field $\delta(\mathbf{x})$ is defined as:

$$\delta(\mathbf{x}) = \frac{\rho(\mathbf{x}) - \bar{\rho}}{\bar{\rho}} = \frac{\rho(\mathbf{x})}{\bar{\rho}} - 1$$

Where $\bar{\rho}$ is the mean background density of the universe.

By taking the Fourier transform of the density contrast field, we convert our 3D grid of densities into a 3D grid of waves, denoted as $\tilde{\delta}(\mathbf{k})$.

The **Matter Power Spectrum**, $P(k)$, is the variance of these Fourier amplitudes as a function of their scale. For a given wavenumber k (which corresponds to a physical spatial scale $\lambda = 2\pi/k$), the power spectrum is:

$$P(k) = \langle|\tilde{\delta}(\mathbf{k})|^2\rangle$$

The spectral index, n, is the most important term for defining the *character* of the initial cosmic structure, controlling the balance of power between large-scale (low frequency, k) and small-scale (high frequency, k) fluctuations.

In cosmology, the special "flat" or **scale-invariant** spectrum is defined by a spectral index of $n = 1$. This case, known as the Harrison-Zel'dovich spectrum, represents a universe where the initial fluctuations have the same strength on all physical scales. All real-world spectra are described by how they "tilt" away from this baseline.

- If $n = 1$, we have a **scale-invariant** spectrum. This is the theoretical "white noise" or baseline for cosmology.
- If $n < 1$, we have a **"red-tilted"** spectrum. There is more power in large-scale (low k) fluctuations and less power in small-scale (high k) ones, compared to the scale-invariant case. This results in a universe with large, gentle, rolling waves of density.
- If $n > 1$, we have a **"blue-tilted"** spectrum. There is less power on large scales and more power on small scales, compared to the scale-invariant case.

Observations of the early universe show that our cosmos has a **"red-tilted"** spectrum, with a primordial spectral index n_s very close to 1 (specifically, $n_s \approx 0.96$). This means the initial density ripples were slightly stronger on larger scales than on small ones, providing the seeds for the vast cosmic web we see today.

To generate the displacement pattern, $\mathbf{\Psi}_0(\mathbf{x})$, we use the following steps:

1. **Define the Wavevectors and Physical Scale.** In a 3D Fourier grid, every wave is defined by a **wavevector**, $\mathbf{k} = (k_x, k_y, k_z)$, which points in the direction the wave is traveling. The magnitude of this vector is the **wavenumber**, $k = |\mathbf{k}|$, which represents the wave's spatial frequency. To simulate the real universe, we must anchor the discrete grid to a physical scale. By defining the comoving size of the simulation box in Megaparsecs (L_{box}), we can calculate the physical wavenumber for every mode:

$$k_{\text{phys}} = \sqrt{k_x^2 + k_y^2 + k_z^2} \cdot \left(\frac{2\pi}{L_{\text{box}}}\right)$$

   This physical wavenumber tells the simulation what size structure each wave represents, from massive superclusters to small dwarf galaxies.

2. **Generate and Shape the Random Field.** We start by creating a grid of random complex numbers, $\delta(\mathbf{k})$, that satisfies **conjugate symmetry**, $\delta(\mathbf{k}) = \delta^*(-\mathbf{k})$, to ensure the final field in real space is real-valued. Each Fourier mode is then scaled so its amplitude follows the **ΛCDM power spectrum**. While the primordial universe started with a nearly scale-invariant spectrum (k^{n_s}), the fast rate of cosmic expansion during the radiation-dominated era suppressed the gravitational collapse of small-scale structures. To capture this physics, we multiply the primordial spectrum by a **Cosmological Transfer Function**, $T(k)$. A standard analytical approximation for this is the **BBKS Transfer Function** (Bardeen, Bond, Kaiser, Szalay, 1986)—explained later. The final shaped power spectrum is evaluated using our physical wavenumbers:

$$P(k_{\text{phys}}) = A \cdot k_{\text{phys}}^{n_s} T(k_{\text{phys}})^2$$

   Here, A is a master normalization constant that scales the overall strength of the fluctuations. Each random mode is scaled by the square root of this power spectrum: $\delta_\rho(\mathbf{k}) = \delta(\mathbf{k})\sqrt{P(k_{\text{phys}})}$.

3. **Compute the displacement field.** The random field $\delta_\rho(\mathbf{k})$ we just generated is a scalar map of density (where the mass is). However, our goal is the displacement field $\mathbf{\Psi}_0(\mathbf{x})$, which is a vector map telling particles which way to move. To bridge this gap, we must use the physics of gravity. In the Zel'dovich approximation, particles are displaced by falling down the gradient of the gravitational potential created by the density fluctuations. First, we use Poisson's equation to convert the density field into a gravitational potential $\hat{\Phi}(\mathbf{k})$, which requires an inverse Laplacian operation (dividing by $-k^2$). Second, we take the gradient of this potential to find the displacement vector, which in Fourier space corresponds to multiplying by $i\mathbf{k}$. Because this derivative is taking place purely on our discrete grid, we use the dimensionless internal grid-unit wavevectors ($\mathbf{k}_{\text{grid}}$):

$$\hat{\mathbf{\Psi}}_0(\mathbf{k}) \propto i\mathbf{k}_{\text{grid}} \frac{\delta_\rho(\mathbf{k})}{k_{\text{grid}}^2}$$

4. **Transform back to real space.** Finally, we apply the inverse Fourier transform to recover the displacement pattern in real space:

$$\mathbf{\Psi}_0(\mathbf{x}) = \mathcal{F}^{-1}\{\hat{\mathbf{\Psi}}_0(\mathbf{k})\}$$

Normalizing the Power Spectrum (σ_8) In the previous step, we left the overall amplitude multiplier, A, undefined. To anchor our initial conditions to observational reality, this amplitude cannot be arbitrary. In cosmology, it is pinned to a standard measured value known as σ_8 (Sigma-8).

σ_8 represents the root-mean-square (RMS, $\sigma_8 = \sqrt{\langle \delta_R(\mathbf{x})^2 \rangle}$) of mass density fluctuations within a sphere of radius 8 Mpc/h in the present-day universe. If we were to drop spheres of this size randomly throughout the cosmos, the mass inside them would vary depending on whether they landed in an empty void or a dense supercluster. σ_8 quantifies this variance—but with a caveat. The real universe today at this scale is mildly non-linear. However, the $\sigma_8 \approx 0.81$ value provided by CMB surveys (like Planck) is a linearly extrapolated parameter. It represents what the variance would be today if structures had grown purely linearly since the Big Bang. Because our initial conditions generator (using the Zel'dovich approximation) relies entirely on linear theory, this standard convention is exactly what we need.

To enforce this in our simulation, we must normalize our theoretical power spectrum so that its mathematical variance at $R = 8$ Mpc/h equals σ_8^2. The variance σ_R^2 of a field smoothed over a physical scale R is found by

integrating the power spectrum multiplied by a "window function," $\tilde{W}(kR)$, in Fourier space:

$$\sigma_R^2 = \frac{1}{2\pi^2}\int_0^\infty P_{\text{unnorm}}(k)\tilde{W}^2(kR)k^2 dk$$

For a spherical volume, the appropriate filter is the **spherical top-hat window function**, whose Fourier transform is:

$$\tilde{W}(kR) = \frac{3\left(\sin(kR) - kR\cos(kR)\right)}{(kR)^3}$$

By numerically integrating our unnormalized BBKS power spectrum (where $A = 1$) using this window function at $R = 8h^{-1}$Mpc, we calculate the unnormalized variance. The master normalization constant, A, is then simply the ratio of the target observational variance to this theoretical variance:

$$A = \frac{\sigma_8^2}{\sigma_{R=8,\text{unnorm}}^2}$$

Applying this constant A ensures our theoretical power spectrum matches the baseline amplitude expected by linear theory for the present day, calibrating our $z = 0$ field before we scale it back to our starting redshift.

The Physics of the Transfer Function: BBKS If the universe contained only dark matter, the primordial power spectrum ($P(k) \propto k^{n_s}$) would remain nearly a straight line across all scales. However, the early universe was a chaotic, boiling soup dominated by intense radiation. The radiation era fundamentally altered how structures of different sizes were allowed to grow, a process we capture mathematically using the **Cosmological Transfer Function**, $T(k)$.

The shape of $T(k)$ is driven by a cosmic race between gravity and the expansion of space, dictated by the **horizon** (the maximum distance light, and therefore any causal physical interaction, could have traveled since the Big Bang).

1. **Large Scales (Small k):** These density fluctuations were physically larger than the cosmic horizon in the early universe. Because they were larger than the distance light could travel, local gravity could not pull them together. However, because overdense regions on these massive scales contained more energy than the cosmic average, their local space expanded slightly slower than the background universe. As the background diluted, these denser regions diluted more slowly, differentiating them more from the background. By the time the cosmic horizon grew large enough to encompass them, the universe had already transitioned into the calm, **matter-dominated era**. Because they grew steadily the entire time, they never lost any growth potential. We therefore define them as our unsuppressed baseline: $T \approx 1$.
2. **Small Scales (Large k):** These tiny fluctuations were small enough to enter the horizon very early on, right in the middle of the radiation-dominated era. During this epoch, the energy density of the universe was overwhelmingly dominated by radiation, which acted as a brake to the expansion. Because the radiation had immense pressure, it could not clump together on these small scales; it remained a smooth, uniform background. During this epoch, the speed of the cosmic expansion was still very high. The dark matter particles were trying to collapse under their own gravity, but because the uniform radiation provided no localized gravitational help, they were entirely on their own. The rapid expansion acted like a cosmic treadmill, pulling the particles apart faster than they could fall together. Consequently, the gravitational collapse of these small-scale density ripples was stalled—a phenomenon known as the **Mészáros effect**. They could only begin to collapse later, once the radiation diluted, the universe transitioned into the matter-dominated era, and the expansion slowed down enough for gravity to finally win.

Calculating the shape of this suppression requires solving complex differential equations tracking dark matter, baryons, photons, and neutrinos. In 1986, Bardeen, Bond, Kaiser, and Szalay (BBKS) published an analytical fitting formula that approximates the result of these calculations.

The BBKS transfer function depends on a scaled wavenumber, q, which adjusts the physical wavenumber k (measured in Mpc^{-1}) based on the density of the universe. For a standard model, it is approximated as:

$$q = \frac{k}{\Omega_m h^2 \exp(-\Omega_b - \sqrt{2h}\Omega_b/\Omega_m)}$$

(Note: In purely dark-matter-dominated, simplified models, the denominator is often reduced to simply the "shape parameter" $\Gamma \approx \Omega_m h$.)

The BBKS formula itself is:

$$T(q) = \frac{\ln(1+2.34q)}{2.34q}\left[1+3.89q+(16.1q)^2+(5.46q)^3+(6.71q)^4\right]^{-0.25}$$

Mathematically, this equation perfectly captures the physics of the early universe:

- As $q \to 0$ (huge cosmic scales), the function approaches 1, preserving the primordial k^{n_s} shape.
- As $q \to \infty$ (tiny cosmic scales), the function falls off proportionally to q^{-2}.

Because the final power spectrum is proportional to $T(k)^2$, this causes the small-scale power to drop off by a factor of k^{-4}. This creates a distinct "turnover" peak in the total power spectrum. The exact location of this peak is a signature of the transition from the radiation era to the matter era (representing the size of the horizon at matter-radiation equality), while the steep drop-off that follows is the signature of the radiation epoch itself. Together, they form the curve that dictates the abundance and sizes of galaxies in our simulation.

Applying the Displacements and Velocities

With the spatial pattern $\mathbf{\Psi}_0(\mathbf{x})$ calculated and normalized to the present-day universe ($a = 1$), we can now set the initial state of our simulation at its starting time. Because the normalization is baked into the field, applying it to the early universe relies entirely on cosmological scaling.

The final initial position of each particle is its grid position plus the displacement field, scaled back in time to the starting epoch. In the very early universe (e.g., $a = 0.02$), matter overwhelmingly dominates over Dark Energy. Because of this, we can safely approximate that the early growth factor scales directly with the expansion of space, $D(a) \approx a$.

However, because our field $\mathbf{\Psi}_0(\mathbf{x})$ is normalized to the present day ($a = 1$), we cannot simply multiply by a_{initial}. In a ΛCDM universe, late-time Dark Energy stalls structure formation, meaning the present-day growth factor is actually suppressed to a value $g_1 = D(1) < 1$. To correctly unwind this late-time suppression and recover the true early-universe amplitude, we must divide by g_1 (typically evaluated using the Carroll, Press, and Turner 1992 fitting formula):

$$\mathbf{x}_{\text{final}} = \mathbf{x}_{\text{grid}} + \frac{a_{\text{initial}}}{g_1}\mathbf{\Psi}_0(\mathbf{x}_{\text{grid}})$$

where g_1 is defined as:

$$g_1 = \frac{2.5\Omega_m}{\Omega_m^{4/7} - \Omega_\Lambda + \left(1+\frac{\Omega_m}{2}\right)\left(1+\frac{\Omega_\Lambda}{70}\right)}$$

Calculating the initial "peculiar" velocity (a particle's motion on top of the Hubble flow) requires more care. The velocity is the time derivative of the comoving displacement, meaning it depends on the rate of change of the growth factor:

$$\mathbf{v}_{\text{pec}} = \left.\frac{dD(t)}{dt}\right|_{t_{\text{initial}}} \mathbf{\Psi}_0(\mathbf{x}_{\text{grid}})$$

In a pure Einstein-de Sitter universe, this derivative simplifies neatly to $\frac{dD}{dt} = H(t)D(t)$. But in a full ΛCDM universe, we must account for the fact that Dark Energy is actively suppressing the rate at which these structures grow. To express this physically, cosmologists define the **Logarithmic Growth Rate**, f:

$$f = \frac{d\ln D}{d\ln a}$$

The foundational approximation for this rate was first derived by P.J.E. Peebles in 1980 as $f \approx \Omega_m^{0.6}$ for a purely matter-dominated universe. However, in a modern flat ΛCDM universe, Dark Energy alters this suppression slightly. Today, the standard approximation used in cosmological codes is:

$$f \approx \Omega_m(a)^{0.55}$$

By substituting this growth rate into our derivative, we arrive at the generalized cosmological equation for the initial peculiar velocities. It is proportional to the displacement field itself, scaled by the Hubble parameter, the critical suppression factor f, and our fully unwound growth factor:

$$\mathbf{v}_{\text{pec}} = H_{\text{initial}} \cdot f \cdot \frac{a_{\text{initial}}}{g_1}\mathbf{\Psi}_0(\mathbf{x}_{\text{grid}})$$

This method produces a self-consistent set of initial conditions for both position and velocity, tailored to the chosen cosmology. The particle motions are correlated over large distances, forming the beginnings of the filaments and voids that will later evolve into galaxies and clusters.

Key Literature & Further Reading
Hahn, O. (2024). Bridging perturbation theory and simulations: initial conditions and fast integrators for cosmological simulations. *SciPost Physics Lecture Notes*. Available at https://scipost.org/preprints/scipost_20 2507_00057v2/

Bardeen, J. M., Bond, J. R., Kaiser, N., & Szalay, A. S. (1986). The statistics of peaks of Gaussian random fields. *The Astrophysical Journal*, 304, 15-61. Available at: https://articles.adsabs.harvard.edu/pdf/1986ApJ...304...15B See Appedix G.

Linder, E. V. (2005). Cosmic growth history and expansion history. *Physical Review D*, 72(4), 043529. Available at: https://arxiv.org/pdf/astro-ph/0507263

Carroll, S. M., Press, W. H., & Turner, E. L. (1992). The cosmological constant. *Annual Review of Astronomy and Astrophysics*, 30, 499-542. Available at: https://www.physics.rutgers.edu/~saurabh/physics690.spring08/Carroll-Press-Turner-1992.pdf

Gravitational softening

In a previous chapter, we introduced gravitational softening as a technique to avoid numerical singularities. Newton's law of gravity, $F \propto 1/r^2$, dictates that as the distance r between two point masses approaches zero, the force approaches infinity. Softening prevents these forces to cause numerical problems:

$$F = \frac{G m_1 m_2 r}{(r^2 + \epsilon^2)^{3/2}}$$

However, gravitational softening also serves a physical purpose: **enforcing the collisionless limit of Dark Matter.**

Real Dark Matter consists of microscopic particles. Because the number of particles is large, they move smoothly through a collective gravitational potential well, behaving as a continuous, collisionless fluid governed by the Vlasov-Poisson equations.

Our simulations, however, use a smaller number of "tracer" particles, where a single particle might represent milions of solar masses. If we treat these massive particles as pure point masses with unmodified $1/r^2$ gravity, they can violently slingshot off one another in close encounters, transferring kinetic energy in a numerical artifact known as **artificial two-body relaxation**. Over time, this artificial scattering evaporates the cores of simulated Dark Matter halos.

The softening length (ϵ) smears the mass of the macro-particles over a finite spherical volume, simulating soft, overlapping clouds of density. This results in a smooth, collisionless fluid behavior.

Determining the Optimal Softening Length

The softening length must balance two requirements:

- **The Lower Bound (Preventing Relaxation):** If ϵ is too small, the point-mass nature of the macro-particles dominates. Artificial two-body relaxation will transfer kinetic energy into the halo cores, causing them to artificially heat up, and dissolve over time.
- **The Upper Bound (Resolving Forces):** If ϵ is too large, physical forces are artificially smoothed out. The simulation will fail to resolve the gravitational potential wells necessary for matter to collapse.

To find a proper factor, we can use the convergence criteria derived for individual, virialized structures. Power et al. (2003) demonstrated that to resolve the dense core of a Dark Matter halo without triggering artificial two-body relaxation, the optimal softening length must scale with the square root of the particle number inside the halo:

$$\epsilon_{opt} = \frac{4 r_{200}}{\sqrt{N_{200}}}$$

Here, r_{200} is the virial radius of the halo, and N_{200} is the number of particles enclosed within it (the *200* suffix refers to the density contrast of the halo in this study).

However, this formula can't be applied when the sizes and masses of the halos to be formed are not known beforehand. To statistically predict the maximum halo mass expected to form within a specific simulation volume we could use the Halo Mass Function (e.g., Tinker et al. 2008), which is not covered in this text. Instead, our code exposes a configurable parameter that multiplies the mean inter-particle distance (d) to define the softening length, empirically set to 1/30 by default.

The Physical Softening Cap

Cosmological simulations are typically computed in comoving coordinates to factor out the expansion of the universe. If the softening length is defined as a fixed comoving distance ($\epsilon_{comoving}$), a physical conflict arises when a Dark Matter halo collapses and reaches **Virial Equilibrium**. In the real universe, a virialized halo decouples from the Hubble flow; its *physical* size remains constant over time.

If the simulation maintains a fixed *comoving* softening length, the physical size of that softening length is forced to grow alongside the scale factor (a):

$$\epsilon_{physical} = a \times \epsilon_{comoving}$$

When a simulated halo collapses down to the resolution limit of the code, its core becomes trapped by the softening length. Because $\epsilon_{physical}$ is artificially expanding with the universe, the physical size of the Dark Matter core is dragged outward with it.

Since the mass of the core remains roughly constant, but its physical volume expands proportional to a^3, the physical density of the halo decreases over time:

$$\rho_{physical} = \frac{M_{core}}{a^3 \times \epsilon^3_{comoving}} \propto \frac{1}{a^3}$$

The halo is losing density as it's being artificially inflated by the comoving coordinate system. To allow halos to maintain virial equilibrium, we can implement a **Maximum Physical Softening Cap**.

The solution is to dynamically alter the softening length through the simulation. Once the simulation reaches a predefined epoch (for example around $z \approx 2$ or $a \approx 0.33$), the physical softening length is not allowed to grow any further.

To achieve a constant physical softening length while the background universe continues to expand, the comoving softening length must be dynamically shrunk proportional to $1/a$. If a_{cap} is the scale factor at which the cap engages, the active comoving softening becomes:

$$\epsilon_{comoving}(a) = \epsilon_{comoving,base} \times \left(\frac{a_{cap}}{a}\right)$$

By implementing this dynamic cap, the gravitational forces inside dense cores counteract the cosmic expansion. When Dark Matter particles collapse into a halo, they hit a fixed *physical* resolution floor. The core stabilizes, the physical density remains constant, and the halo correctly holds its structural shape against the expanding background universe.

Gravity Validation and Accuracy

Conservation of Energy and Momentum

A physically meaningful simulation must obey the same fundamental laws as the universe it models. For a closed system governed by gravity, the two most powerful checks are therefore the laws of conservation of energy and momentum. If a simulation violates these "golden rules", it is a clear sign of a fundamental error in its implementation or underlying model.

It is crucial to understand that the rules of **conservation of energy and momentum** are only valid for a closed, non-expanding system. The total energy of a system of particles is *not* a conserved quantity in an expanding universe. The expansion of space itself does work on the system. The peculiar velocities of particles decrease due to Hubble drag, and the potential energy changes as the physical distances between all particles grow. Therefore, checking for energy conservation during a cosmological run is not a valid test; the energy is expected to change over time.

To use conservation as a test of a simulation's accuracy, we must first disable the cosmic expansion. This is done by setting the scale factor to a constant value, $a(t) = 1$, for the entire run. In this "static box" mode, the universe does not expand, and the Hubble drag term is zero. The simulation becomes a pure gravitational N-body problem.

The standard practice is to first validate the code in a non-expanding box to confirm these conservation laws hold to a high degree. Once the core engine is verified, we can then enable the cosmic expansion, confident that any subsequent physical behavior is a result of the cosmology, not a bug in the integrator.

Conservation of Momentum

In a closed system with no external forces, the total momentum must remain constant. The conservation of momentum is a direct consequence of Newton's third law: for every force $\mathbf{F}_{ij}$ that particle j exerts on particle i, there is an equal and opposite force $\mathbf{F}_{ji} = -\mathbf{F}_{ij}$. When we sum the forces over the entire system, all these internal forces perfectly cancel out. If the code correctly implements this symmetry, total momentum will be conserved.

The total momentum $\mathbf{P} = (P_x, P_y, P_z)$ can be computed as:

$$P_x = \sum_i m_i v_{ix}$$

$$P_y = \sum_i m_i v_{iy}$$

$$P_z = \sum_i m_i v_{iz}$$

Where m_i is the mass of particle i, and v_{ix}, v_{iy} and v_{iz} are its velocity components. The values of P_x, P_y and P_z should remain unchanged along the simulation to within machine precision. Any systematic drift indicates a bug in the force calculation.

In a closed system with no external torques, the total **angular momentum** must also be conserved. For a system of particles, the total angular momentum is the vector sum of each particle's individual angular momentum, $\mathbf{L} = \mathbf{r} \times \mathbf{p}$.

We can calculate the total angular momentum vector $\mathbf{L} = (L_x, L_y, L_z)$ using the formula:

$$L_x = \sum_i m_i (y_i v_{iz} - z_i v_{iy})$$

$$L_y = \sum_i m_i (z_i v_{ix} - x_i v_{iz})$$

$$L_z = \sum_i m_i (x_i v_{iy} - y_i v_{ix})$$

Just like with linear momentum, the values of L_x, L_y, and L_z should each remain unchanged. Any systematic drift in any component signals a subtle bug in the geometric implementation of your force.

Conservation of Energy

For a conservative system like gravity, the total energy—the sum of the **Kinetic Energy (KE)** from motion and the **Potential Energy (PE)** from gravitational attraction—must remain constant.

This is a much more sensitive and comprehensive test than momentum conservation. It validates the entire simulation loop, especially the accuracy of the **time integrator**. While momentum checks the symmetry of the forces at a single instant, energy conservation checks how well the simulation evolves the system from one state to the next.

Because the simulation moves in discrete time steps, we don't expect the energy to be conserved perfectly. Instead, the *behavior* of the error tells us if the integrator is working correctly:

- **A good (symplectic) integrator** will produce an energy error that **oscillates** around the initial value. The energy will wobble, but it will not show a long-term trend of increasing or decreasing.
- **A bad or inconsistent integrator** will cause the energy to **drift** steadily over time, indicating a systematic error that is continuously adding or removing energy from the system.

To calculate the total energy, $E(0) = KE + PE$ we just add the kinetic and potential energies as described below.

- **Kinetic Energy (KE):** The total kinetic energy is the sum of the kinetic energy of every particle in the system.

$$KE = \sum_{i=1}^{N} \frac{1}{2} m_i v_i^2$$

  Where m_i is the mass of particle i and v_i is its speed ($v_i^2 = v_{ix}^2 + v_{iy}^2 + v_{iz}^2$).

- **Potential Energy (PE):** The total potential energy is the sum of the potential energy for every *unique pair* of particles. The most meaningful way to measure the simulation's accuracy is to check how well it conserves the total energy of the **ideal, unsoftened system**.

$$PE = \sum_{i=1}^{N} \sum_{j>i}^{N} \frac{-G m_i m_j}{r_{ij}}$$

  Where r_{ij} is the distance between particles i and j.

We can periodically recalculate this total energy, $E(t)$, as the simulation runs and monitor the relative error over time: $\frac{E(t)-E(0)}{E(0)}$.

A small, bounded oscillation in this value is the signature of a healthy, stable simulation. A consistent drift, no matter how small, points to a deeper issue that needs to be fixed.

The Two-Body Problem and Kepler's Laws

The conservation laws of energy and momentum are powerful checks on the overall stability of our simulation, but they don't tell us if the individual trajectories of our particles are physically correct. For that, we need a "ground truth"—a simple problem with a known, exact mathematical solution that we can compare our simulation against.

In celestial mechanics, the perfect "unit test" is the **two-body problem**. It is one of the very few gravitational problems that can be solved perfectly with pen and paper, and its solution is described by **Kepler's Laws of Planetary Motion**. If a simulation can't reproduce this fundamental result, it can't be trusted with more complex systems.

The setup requires to simplify the simulation to just two bodies:

1. **The "Star":** Create one particle with a very large mass and place it near the center of the simulation box. Give it zero initial velocity to keep it mostly stationary.
2. **The "Planet":** Create a second particle with a much smaller mass and place it some distance away from the star, for example, along the x-axis. Give the planet an initial velocity purely in the y-direction. The magnitude of this velocity is crucial; a specific value will produce a circular orbit, while slightly different values will produce stable elliptical orbits.

After running the simulation, we can check the planet's trajectory against Kepler's predictions:

1. Is the Orbit a Closed Ellipse? (Kepler's First Law) The most important check. The planet should trace a stable, closed ellipse with the star at one of the foci. Common failure modes are:

- **Energy Drift:** The orbit spirals inwards or outwards, indicating a non-symplectic integrator or a bug.
- **Unphysical Precession:** The ellipse itself rotates over time. A large, rapid precession is a sign of numerical inaccuracy. A stable, non-precessing ellipse is a sign of a healthy integrator.

2. Does it Speed Up and Slow Down Correctly? (Kepler's Second Law) This law states that a planet sweeps out equal areas in equal times, which is a consequence of angular momentum conservation. This means the planet must move **fastest** when it is closest to the star (perihelion) and **slowest** when it is farthest away (aphelion).

3. Does it Obey the Law of Periods? (Kepler's Third Law) The law states that the square of the orbital period (P) is proportional to the cube of the orbit's semi-major axis (a), or $P^2 \propto a^3$. The time it takes the simulated planet to complete one full orbit and the size of its orbit must satisfy this mathematical relationship.

Passing the two-body test is a critical milestone. It builds confidence that the implementations of the gravitational force and the time integrator are fundamentally sound, and it's a prerequisite for tackling the chaotic dance of a true N-body system.

Sources of Error

A perfect simulation is impossible. The goal is not to eliminate all errors but to understand where they come from, control them, and ensure they are small enough that the results are physically meaningful. In a P^3M simulation, the errors arise from the three fundamental approximations we make to turn the smooth, continuous problem of gravity into a discrete problem a computer can solve.

Time Discretization Error

Physics happens continuously, but a simulation must leap forward in discrete chunks of time, Δt. The time integrator works by assuming that the acceleration changes in a simple, predictable way during that small leap.

However, the true acceleration, especially during a close encounter, can be more complex. The difference between the integrator's assumed path and the true physical path is called **truncation error**. This error scales with the square of the time step ($O(\Delta t^2)$) and is a primary contributor to the oscillations in the total energy.

Softening Bias

To prevent infinite forces when particles get too close, we modify the force law with a softening parameter, ϵ.

$$F = \frac{Gm_1m_2}{r^2 + \epsilon^2}$$

This is not a numerical error but a **physical modeling error**. We are knowingly simulating a slightly different, "softer" version of gravity. This error is most significant at very short distances ($r \lesssim \epsilon$) and prevents the formation of very "hard," dense structures. We accept this trade-off because it grants us numerical stability, but the simulation becomes blind to any physics occurring at scales smaller than the softening length.

Spatial Discretization Error

The Particle-Mesh method gains its speed by calculating long-range forces on a grid. This introduces several errors related to the grid's finite resolution.

- **Aliasing:** The grid can only represent waves with a wavelength larger than about two cell sizes. When two particles get closer than this, their sharp, high-frequency density spike is misinterpreted by the grid as a smooth, low-frequency wave. This is **aliasing**, and it is the primary source of inaccuracy in the PM force, making it "blurry" and even repulsive at short distances.
- **Interpolation Error:** The schemes used to assign mass to the grid and interpolate forces back (like NGP or CIC) are approximations that contribute to the total error.
- **Finite Difference Error:** The force on the grid is calculated using a finite difference approximation of the gradient. This is not a perfect derivative and adds a small amount of error.

Ultimately, running a successful simulation implies balancing these interconnected errors. A smaller softening length demands a smaller time step. A finer grid reduces aliasing but increases computational cost. Understanding these trade-offs is the key to producing results that are both stable and physically reliable.

Key Literature & Further Reading
Dehnen, W., & Read, J. I. (2011). *N-body simulations of gravitational dynamics.* arXiv:1105.1082. Available at https://arxiv.org/abs/1105.1082.

Hydrodynamics

A simulation that focuses on the evolution of collisionless N-body particles is an excellent model for the dark matter that forms the universe's invisible scaffolding. However, to simulate the formation of the luminous structures—stars, galaxies, and galaxy clusters—it must include the physics of **baryonic matter**. In cosmology, "baryons" is the term for all normal matter, which primarily exists as a vast, diffuse **gas**.

Unlike dark matter, gas is not collisionless. Its particles (atoms and ions) interact with each other, giving rise to familiar fluid properties like **pressure** and **temperature**.

However, it is natural to wonder how a medium so incredibly empty—often containing just a single atom in a volume the size of a small room—can behave like a fluid and push back against gravity. The answer lies in the sheer scale of the universe and the physics of plasmas.

Out in the deep cosmic voids, the primordial gas is very cold and diffuse. However, because cosmological structures are millions of light-years across, the distance an atom travels before colliding with another is still microscopic compared to the size of the system. On these vast scales, even a near-vacuum experiences enough microscopic collisions to possess macroscopic fluid properties like pressure and density.

The physics changes dramatically when this cold gas falls into the immense gravity of a dark matter halo. As the gas crashes into the halo at hypersonic speeds, shockwaves heat it to millions of degrees, turning it into a highly energetic plasma. In this state, the charged ions and electrons do not need to physically collide like billiard balls to interact. Instead, they repel each other across vast distances via electromagnetic forces and are threaded together by large-scale magnetic fields. These long-range interactions ensure that even inside the hottest, most violent galaxy clusters, the diffuse matter continues to act as a cohesive fluid.

The Hybrid Simulation Approach

To mathematically model this continuous behavior across both the cold voids and the hot halos, a simulation must solve the laws of **hydrodynamics**.

In a **hybrid code**, the universe is modeled using two distinctly different computational perspectives. The dark matter is treated as a collection of **Lagrangian** N-body particles (tracking individual masses as they move freely through space). Conversely, the gas is treated using an **Eulerian** approach (tracking the continuous properties of a fluid as it flows past stationary points).

To seamlessly link these two components, the Eulerian fluid dynamics are solved on the exact same fixed spatial grid used by the Particle-Mesh gravity solver. The two components communicate via the force of gravity, which is sourced by their combined density on this shared grid. This hybrid approach is the foundation of all modern cosmological simulations that aim to model the formation of the visible universe.

The Euler Equations

For a simple, non-viscous gas (a good approximation for most cosmic fluids), its motion is governed by a set of conservation laws known as the **Euler Equations**. These equations describe how the density, momentum, and energy of the gas change over time.

The full set of cosmological hydrodynamic equations is complex, as it couples the conservation laws of fluid dynamics with the source terms from gravity in an expanding universe. The equation for the vector of conserved quantities, $\mathbf{U}$, can be written as:

$$\frac{\partial \mathbf{U}}{\partial t} + \nabla \cdot \mathbf{F}(\mathbf{U}) = \mathbf{S}(\mathbf{U})$$

Where:

- $\mathbf{U}$ is the vector of conserved state variables.
- $\nabla \cdot \mathbf{F}(\mathbf{U})$ is the "flux" term, which describes how quantities move due to pressure and advection (the fluid flowing).
- $\mathbf{S}(\mathbf{U})$ is the "source" term, which describes changes due to external forces, like gravity ($\rho\mathbf{g}$).

While standard fluid dynamics relies only on three fundamental variables (mass, momentum, and total energy), simulating the universe requires a mathematical safety net. In cosmological flows, gas moves at extreme, supersonic speeds. In these conditions, a grid cell's kinetic energy becomes vastly larger than its thermal energy. Trying to numerically calculate the tiny internal temperature by subtracting a massive kinetic energy from a massive total energy leads to catastrophic floating-point errors.

To solve this, cosmological codes use the **Dual Energy Formalism** (which will be explored in detail in a later section), explicitly tracking a fourth variable: **internal energy density (ie)**. Therefore, our expanded state vector becomes a four-element array:

$$\mathbf{U} = [\rho, \rho\mathbf{v}, E, ie]$$

Expanding this into our continuous differential equation yields the complete physics model for our gas grid:

$$\frac{\partial}{\partial t}\begin{bmatrix}\rho\\ \rho\mathbf{v}\\ E\\ ie\end{bmatrix}+\nabla\cdot\begin{bmatrix}\rho\mathbf{v}\\ \rho\mathbf{v}\otimes\mathbf{v}+P\mathbf{I}\\ (E+P)\mathbf{v}\\ ie\mathbf{v}\end{bmatrix}=\begin{bmatrix}0\\ \rho\mathbf{g}\\ \rho\mathbf{v}\cdot\mathbf{g}\\ -P(\nabla\cdot\mathbf{v})\end{bmatrix}$$

Here is the breakdown of exactly what each row represents:

- **Row 1 (Conservation of Mass):**
  - **Flux:** $\rho\mathbf{v}$ is the advection (flow) of mass.
  - **Source:** 0, because the universe does not spontaneously create or destroy mass.
- **Row 2 (Conservation of Momentum):**
  - **Flux:** $\rho\mathbf{v}\otimes\mathbf{v}+P\mathbf{I}$. This combines the momentum advection tensor ($\rho\mathbf{v}\otimes\mathbf{v}$) with the thermal pressure (P). The advection tensor is a 3x3 matrix that tracks how each directional component of momentum is transported along every spatial axis. The $\mathbf{I}$ is the identity matrix, ensuring the pressure gradient acts perpendicular to the cell faces—this is the force that allows the gas to resist gravitational collapse.
  - **Source:** $\rho\mathbf{g}$, the external acceleration due to gravity pulling on the gas. This links the fluid to the Particle-Mesh gravity solver.
- **Row 3 (Conservation of Total Energy):**
  - **Flux:** $(E+P)\mathbf{v}$. This is the total energy sliding along with the gas ($E\mathbf{v}$), plus the mechanical PdV work being done by the fluid as it compresses and expands against its neighbors ($P\mathbf{v}$).
  - **Source:** $\rho\mathbf{v}\cdot\mathbf{g}$. This is the mechanical work done *by gravity*. As gas falls into a dark matter halo (velocity aligns with gravity), gravity does work on the gas, adding to its total energy.
- **Row 4 (Internal Energy / Dual Energy Formalism):**
  - **Flux:** $ie\mathbf{v}$. This is the pure advection of thermal energy (hot gas flowing from one place to another).
  - **Source:** $-P(\nabla\cdot\mathbf{v})$. This term represents the macroscopic PdV **work** (Pressure × change in Volume). In Eulerian fluid dynamics, the divergence of velocity ($\nabla\cdot\mathbf{v}$) is a direct measurement of the fractional rate of change of a fluid parcel's volume (which is physically equivalent to a change in its density). In the standard Total Energy equation (Row 3), thermodynamic heating and cooling are handled automatically by the pressure flux ($P\mathbf{v}$). However, because we isolated the internal energy from the main equation to avoid floating-point errors, we broke that automatic link. We must explicitly add the PdV work back in as a manual source term:
    - **Adiabatic Compression:** As gas falls into a dark matter halo and converges ($\nabla\cdot\mathbf{v}<0$), its specific volume shrinks and its density spikes. The math yields a positive source term, meaning the environment does work *on* the gas, violently heating it.
    - **Adiabatic Expansion:** As gas flows outward into a cosmic void ($\nabla\cdot\mathbf{v}>0$), its volume expands and its density drops. The math yields a negative source term, meaning the gas spends its own internal energy to push outward, causing it to cool.

Finally, to calculate the pressure (P) required for these equations, we relate it to the density and internal energy using an **equation of state**. For a simple, ideal gas, this equation is:

$$P=(\gamma-1)ie$$

Here, γ is the adiabatic index, a constant which is typically 5/3 for a monatomic gas like the hydrogen and helium that fill the cosmos. It describes how much the pressure of a gas responds to a change in volume during an adiabatic process—when the gas is compressed or expands so quickly that it doesn't have time to exchange heat with its surroundings. A higher γ means the pressure rises more sharply for the same amount of compression.

An Operator-Split Hydro-Solver

Solving this equation all at once is difficult. In a real cosmological fluid, multiple physical processes are happening simultaneously: gas is flowing through space, pressure waves are expanding, gravity is accelerating the mass, and thermal energy is radiating away. Mathematically, these distinct physical phenomena are represented by different "operators" (the flux and source terms in the differential equation) that are deeply tangled together. Solving them concurrently in a single, massive calculation is incredibly difficult and often numerically unstable.

A common and effective technique called **operator splitting** breaks the problem into simpler, sequential steps. Instead of attempting to solve everything at once, we temporarily decouple the physics.

For a tiny slice of time (Δt), we allow the gas to *only* flow across the grid and respond to its own internal pressure, completely ignoring external forces. Once we calculate the new state of the fluid, we freeze its motion. Then, assuming that same time slice (Δt) has passed, we apply *only* the pull of gravity to this newly updated state.

By taking turns applying each physical process sequentially, the combined result accurately approximates the true simultaneous physics, provided the time step Δt is small enough. This "divide and conquer" approach also allows us to use the best, specialized mathematical solver for each individual physical process without them interfering with one another.

Here is the step-by-step process to advance the gas grid from time t to $t + \Delta t$ following the **Kick-Drift-Kick (KDK)** structure.

Step 1: Convert to Primitive Variables

(A quick note on notation: In the generalized Euler equations above, we used $\mathbf{S}(\mathbf{U})$ *to represent the full "Source Vector." However, from this point forward, we will adopt the standard simulation nomenclature where the vector* $\mathbf{S}$ *(and its components* S_x, S_y, S_z*) refers specifically to the* ***momentum density****,* $\rho\mathbf{v}$*.)*

The solver begins with the grid of **conserved variables** (ρ, $S_x = \rho v_x$, $S_y = \rho v_y$, E) from the previous step. To calculate fluxes and forces, it first computes the **primitive variables** (velocity $\mathbf{v}$ and pressure P):

$$v_x = \frac{S_x}{\rho} \quad , \quad v_y = \frac{S_y}{\rho}$$

$$P = (\gamma - 1)\left(E - \frac{1}{2}\rho|\mathbf{v}|^2\right)$$

Step 2: Gravity Half-Step (Kick)

First, the external forces from gravity are applied for half a time step, $\Delta t/2$. The gravitational acceleration field, $\mathbf{g}$, is provided by the Particle-Mesh solver. The acceleration field must be derived from the **total matter density**—the sum of the dark matter density (from the N-body particles) and the gas density (from the hydro grid). This step updates the momentum and total energy of the gas:

$$\mathbf{S}(t + \tfrac{1}{2}\Delta t) = \mathbf{S}(t) + (\rho\mathbf{g})\frac{\Delta t}{2}$$

$$E(t + \tfrac{1}{2}\Delta t) = E(t) + (\mathbf{v} \cdot \rho\mathbf{g})\frac{\Delta t}{2}$$

The energy is updated by the **power density** (Force Density · Velocity, or $\mathbf{v} \cdot \rho\mathbf{g}$), which is the rate at which the gravitational field does work on the gas.

Step 3: Hydrodynamic Full-Step (Drift)

With the gravitational source terms applied, we can solve the pure hydrodynamic equations (the "flux" part) for a full time step, Δt. To calculate how much gas flows from one cell to the next, we must first estimate the fluid's properties exactly at the boundary between them—a process known as **spatial reconstruction**.

If we use a high-accuracy, second-order spatial reconstruction scheme (like MUSCL, detailed in the next section), taking a simple first-order "forward Euler" step in time would introduce instabilities and dampen the sharp shocks that we need to resolve. Instead, we must match the spatial accuracy with **second-order time integration**. A highly robust method for finite-volume hydrodynamics is the **Strong Stability Preserving Runge-Kutta (SSP-RK2)** scheme.

Because operator splitting temporarily isolates the fluid's motion (the "Drift" stage) into a purely hydrodynamic problem, we are free to solve it using its own specialized mathematical engine. Therefore, nested inside the global KDK loop, this step utilizes the SSP-RK2 scheme. KDK orchestrates the overall physics, while SSP-RK2 acts as the high-precision sub-engine that actually pushes the gas across the grid.

In multi-stage Runge-Kutta methods, it is standard to denote the fluid state at the current time t using the step index n (written as $\mathbf{U}^n$), and the final state at $t + \Delta t$ as $\mathbf{U}^{n+1}$. The SSP-RK2 method achieves second-order

accuracy by breaking the fluid advection into two intermediate mathematical stages (denoted by parenthetical superscripts) and averaging the results:

1. The Predictor Stage: The solver takes a full Euler step using the current fluid state ($\mathbf{U}^n$) to calculate the fluxes. This generates an intermediate, predicted state:

$$\mathbf{U}^{(1)} = \mathbf{U}^n + \Delta t \cdot \mathbf{L}(\mathbf{U}^n)$$

Where $\mathbf{L}(\mathbf{U})$ acts as the **discrete spatial operator**. Earlier in the chapter, the fluid's motion was described by the continuous calculus term $\nabla \cdot \mathbf{F}(\mathbf{U})$. Because our simulation cannot perform infinite calculus, we must approximate this physics on a discrete grid. $\mathbf{L}(\mathbf{U})$ serves as the numerical approximation of that physics on that grid. Mathematically, it is defined as:

$$\mathbf{L}(\mathbf{U}) \approx -\nabla \cdot \mathbf{F}(\mathbf{U}) + \mathbf{S}_{\text{internal}}$$

Expanded:

$$\mathbf{L}(\mathbf{U}^n) = \underbrace{\begin{bmatrix} -\nabla \cdot (\rho \mathbf{v})^n \\ -\nabla \cdot (\rho \mathbf{v} \otimes \mathbf{v} + P\mathbf{I})^n \\ -\nabla \cdot ((E+P)\mathbf{v})^n \\ -\nabla \cdot (ie\mathbf{v})^n \end{bmatrix}}_{-\nabla \cdot \mathbf{F}(\mathbf{U}^n)} + \underbrace{\begin{bmatrix} 0 \\ \mathbf{0} \\ 0 \\ -P^n(\nabla \cdot \mathbf{v}^n) \end{bmatrix}}_{\mathbf{S}_{\text{internal}}}$$

Here, the negative sign appears because divergence ($\nabla \cdot \mathbf{F}$) measures the net outflow, whereas $\mathbf{L}(\mathbf{U})$ calculates the net influx across the cell boundaries. Instead of evaluating a continuous derivative, this discrete divergence for a given cell i is calculated as the algebraic difference between the fluxes crossing its right and left boundaries, divided by the cell width (Δx):

$$\nabla \cdot \mathbf{F}(\mathbf{U}) \approx \frac{\mathbf{F}_{i+1/2} - \mathbf{F}_{i-1/2}}{\Delta x}$$

These specific boundary fluxes ($\mathbf{F}_{i+1/2}$ and $\mathbf{F}_{i-1/2}$) are exactly what the HLLC Riemann solver (explained later) will compute.

The $\mathbf{S}_{\text{internal}}$ term accounts for internal thermodynamics, specifically the PdV work required by the Dual Energy Formalism. Because the Dual Energy Formalism tracks internal energy separately from total energy, we must explicitly add this PdV source term so that the gas physically heats up when compressed by gravity and cools down when expanding into voids.

Noticeably absent is the external gravity source term ($\rho\mathbf{g}$); because we are using an operator-split Kick-Drift-Kick architecture, gravity is handled entirely separately during the half-steps.

To see exactly how this discrete spatial operator updates the fluid, we can expand the equation $\mathbf{U}^{(1)} = \mathbf{U}^n + \Delta t \cdot \mathbf{L}(\mathbf{U}^n)$ into its full column-vector form:

$$\begin{bmatrix} \rho^{(1)} \\ (\rho\mathbf{v})^{(1)} \\ E^{(1)} \\ ie^{(1)} \end{bmatrix} = \begin{bmatrix} \rho^n \\ (\rho\mathbf{v})^n \\ E^n \\ ie^n \end{bmatrix} + \Delta t \cdot \begin{bmatrix} -\nabla \cdot (\rho\mathbf{v})^n \\ -\nabla \cdot (\rho\mathbf{v} \otimes \mathbf{v} + P\mathbf{I})^n \\ -\nabla \cdot ((E+P)\mathbf{v})^n \\ -\nabla \cdot (ie\mathbf{v})^n - P^n(\nabla \cdot \mathbf{v}^n) \end{bmatrix}$$

2. The Corrector Stage: The solver calculates an entirely new set of fluxes based on the *intermediate* state ($\mathbf{U}^{(1)}$). It takes another full Euler step from this intermediate state to generate a second state:

$$\mathbf{U}^{(2)} = \mathbf{U}^{(1)} + \Delta t \cdot \mathbf{L}(\mathbf{U}^{(1)})$$

3. Final Averaging: Finally, the true state at the next time step ($\mathbf{U}^{n+1}$) is found by averaging the original state and the twice-stepped state:

$$\mathbf{U}^{n+1} = \frac{1}{2}\mathbf{U}^n + \frac{1}{2}\mathbf{U}^{(2)}$$

This staggered, averaging approach allows the gas to flow across the grid with true second-order accuracy in both space and time. While calculating the fluxes twice per cycle essentially doubles the computational cost of the hydrodynamics, the dramatic improvement in shock resolution and overall stability makes it a strict requirement for modern cosmological simulations.

Step 4: Second Gravity Half-Step (Kick)

Finally, the gravitational source terms are applied again for the second half of the time step, $\Delta t/2$, using the updated values from the "Drift" step:

$$\mathbf{S}(t+\Delta t) = \mathbf{S}(t+\tfrac{1}{2}\Delta t) + (\rho \mathbf{g})\frac{\Delta t}{2}$$

$$E(t+\Delta t) = E(t+\tfrac{1}{2}\Delta t) + (\mathbf{v}\cdot\rho \mathbf{g})\frac{\Delta t}{2}$$

At the end of this sequence, the conserved variables of the gas grid have been fully advanced to time $t+\Delta t$, accounting for both the internal fluid dynamics and the external force of gravity.

Spatial Reconstruction and the MUSCL Scheme

In a basic finite-volume grid, the simplest way to determine the state of the fluid at the interface between two cells is to assume the fluid properties (density, velocity, pressure, calso called **primitive variables**) are constant across the entire cell. This is known as **Piecewise Constant** reconstruction.

While computationally cheap, piecewise constant reconstruction is highly diffusive. It acts like a blur filter, smearing out sharp shock waves across many grid cells. To capture the sharp, violent shocks of a cosmological simulation, we need a higher-order approach.

The **MUSCL (Monotonic Upstream-centered Scheme for Conservation Laws)** scheme upgrades our spatial resolution to second-order by replacing the flat constant states with **Piecewise Linear** slopes. Instead of assuming a fluid variable q (which represents $\rho, \mathbf{v}$, or P) is flat inside cell i, we calculate a gradient (slope) based on its neighbors, $i-1$ and $i+1$. We then use this slope, Δq_i, to linearly extrapolate the fluid's properties from the cell center to the left and right interfaces of the cell.

Mathematically, to find the fluid states on the exact left (L) and right (R) sides of the interface located between cell i and cell $i+1$, we extrapolate outwards from the cell centers:

$$q_{L,i+1/2} = q_i + \frac{1}{2}\Delta q_i$$

$$q_{R,i+1/2} = q_{i+1} - \frac{1}{2}\Delta q_{i+1}$$

Where:

$$\Delta q_i = \frac{q_{i+1} - q_{i-1}}{2}$$

However, a naive linear extrapolation creates a disastrous numerical artifact at shock fronts. When a steep slope tries to extrapolate across a discontinuous shock, it inevitably overshoots the true value, causing wild, unphysical oscillations (ringing) known as the Gibbs phenomenon.

To safely use linear reconstruction, we must introduce a **Slope Limiter**. A slope limiter acts as a localized safety switch. We first calculate two separate slopes for cell i: the backward difference (looking left) and the forward difference (looking right):

$$\Delta q_{\text{backward}} = q_i - q_{i-1}$$

$$\Delta q_{\text{forward}} = q_{i+1} - q_i$$

- If the fluid is smooth, these two slopes will be similar, and the limiter allows the second-order linear slope.
- If a shock is present, the slopes will violently disagree or change signs. The limiter detects this extremum and immediately forces the slope Δq_i to zero, temporarily dropping the local reconstruction back to a safe, flat first-order state just for that specific shock front.

A common and highly robust choice is the **Minmod limiter**. It compares the forward and backward slopes and selects the one with the smallest magnitude if they point in the same direction, returning zero if they point in opposite directions:

$$\text{minmod}(a,b) = \begin{cases} a & \text{if } |a| < |b| \text{ and } ab > 0 \\ b & \text{if } |b| < |a| \text{ and } ab > 0 \\ 0 & \text{if } ab \leq 0 \end{cases}$$

By setting our cell slope to $\Delta q_i = \text{minmod}(\Delta q_{\text{backward}}, \Delta q_{\text{forward}})$, we guarantee that the reconstructed values at the interfaces will never create new, artificial extrema. By applying this MUSCL reconstruction to the primitive variables $(\rho, \mathbf{v}, P)$, the simulation can resolve sharp shock fronts without sacrificing stability.

The HLLC Riemann Solver

Once the fluid states have been extrapolated to the left ($\mathbf{U}_L$) and right ($\mathbf{U}_R$) sides of an interface, the code must determine the flux of mass, momentum, and energy passing through it ($\mathbf{F}(\mathbf{U})$). This is done using an **approximate Riemann solver**.

When two different fluid states collide at an interface, they spawn a complex fan of waves propagating outwards. The Harten-Lax-van Leer (HLL) solver is a classic approximation that assumes this complex interaction can be simplified into just two waves: the fastest wave moving left and the fastest wave moving right. These wave speeds are denoted by the scalar variables S_L and S_R (where S stands for "Signal velocity", which should not be confused with the momentum density vector $\mathbf{S}$ used earlier in the macroscopic solver).

While incredibly stable, the standard HLL solver has a fatal flaw: it ignores the middle of the wave structure. Specifically, it completely misses the **Contact Discontinuity**—a distinct middle boundary where fluid density and temperature jump abruptly. Across this boundary, pressure and velocity remain perfectly continuous, as the faster acoustic waves in the fan have already propagated outward and equalized the mechanical forces. Because HLL averages over this middle region, it heavily smears out contact discontinuities and shear flows, which are vital for resolving the intricate gas filaments of the cosmic web.

To fix this, modern cosmological codes use the **HLLC** solver (where "C" stands for Contact). The HLLC solver restores the missing middle physics by introducing a third wave speed, S_*, which represents the speed of the contact discontinuity. As the left and right waves move outward from the interface over the time step, they sweep out an expanding region of interacting gas. The S_* wave divides this middle region into two distinct "star" states: $\mathbf{U}_L^*$ and $\mathbf{U}_R^*$.

The calculation of the HLLC flux at the interface ($\mathbf{F}_{HLLC}$) proceeds as follows:

1. Estimate the Signal Velocities First, the local sound speeds for the left ($c_{s,L}$) and right ($c_{s,R}$) states are calculated using the adiabatic index γ, pressure P, and density ρ:

$$c_s = \sqrt{\frac{\gamma P}{\rho}}$$

These sound speeds, along with the normal velocities of the fluid ($v_{n,L}$ and $v_{n,R}$), are used to estimate the outermost wave speeds bounding the Riemann problem. To guarantee numerical stability, the solver must encompass the fastest possible waves traveling in either direction. A robust and standard estimate achieves this by taking the minimum and maximum possible signal velocities across the interface:

$$S_L = \min(v_{n,L} - c_{s,L}, v_{n,R} - c_{s,R})$$

$$S_R = \max(v_{n,L} + c_{s,L}, v_{n,R} + c_{s,R})$$

2. Calculate the Contact Wave Speed (S_*) The speed of the middle contact wave is derived directly from the conservation of momentum across the interface:

$$S_* = \frac{P_R - P_L + \rho_L v_{n,L}(S_L - v_{n,L}) - \rho_R v_{n,R}(S_R - v_{n,R})}{\rho_L(S_L - v_{n,L}) - \rho_R(S_R - v_{n,R})}$$

3. Compute the Star States Using the contact wave speed S_*, the solver can calculate the exact conserved variables for the intermediate "star" regions ($\mathbf{U}_L^*$ and $\mathbf{U}_R^*$) wedged between the shock fronts and the contact discontinuity.

For either the Left or Right state (denoted by the subscript K, where $K \in \{L, R\}$), the density of the star state is determined by mass conservation across the outermost wave:

$$\rho_K^* = \rho_K \left(\frac{S_K - v_{n,K}}{S_K - S_*} \right)$$

Next, we must determine the velocities and energy of these star states by looking at the physics of the contact discontinuity itself. This middle wave (S_*) is the literal boundary where the two original gas masses meet. Because they are pushing against each other with matching pressure and velocity, they do not mix; the interface acts as an impermeable wall that moves with the fluid.

Since no mass actually flows across this boundary, any property tied strictly to the fluid mass—specifically the transverse velocities (v_{t1}, v_{t2}) and the specific internal energy (ie/ρ)—remains constant and simply slides along with the flow. Meanwhile, the normal velocity of the fluid, by definition of moving perfectly with the boundary, becomes exactly S_*.

Therefore, the full vector of conserved variables for the star state $\mathbf{U}_K^* = [\rho^*, (\rho v_n)^*, (\rho v_t)^*, E^*, ie^*]_K$ is given by:

$$\mathbf{U}_K^* = \rho_K^* \begin{bmatrix} 1 \\ S_* \\ v_{t,K} \\ \frac{E_K}{\rho_K} + (S_* - v_{n,K}) \left(S_* + \frac{P_K}{\rho_K (S_K - v_{n,K})} \right) \\ \frac{ie_K}{\rho_K} \end{bmatrix}$$

4. Calculate the Final Flux The solver determines the flux at the interface based on the direction of these three wave speeds:

- If $S_L \geq 0$, the entire wave structure is moving right. The flux is simply the original Left flux: $\mathbf{F}_L$.
- If $S_L < 0 \leq S_*$, the interface is inside the left star region. The flux is: $\mathbf{F}_L^* = \mathbf{F}_L + S_L(\mathbf{U}_L^* - \mathbf{U}_L)$.
- If $S_* < 0 \leq S_R$, the interface is inside the right star region. The flux is: $\mathbf{F}_R^* = \mathbf{F}_R + S_R(\mathbf{U}_R^* - \mathbf{U}_R)$.
- If $S_R \leq 0$, the entire wave structure is moving left. The flux is the original Right flux: $\mathbf{F}_R$.

By properly resolving the contact wave, the HLLC solver accurately tracks the boundaries between hot and cold gas, allowing the simulation to capture the complex, multi-phase thermodynamics of galaxy formation.

The Dual Energy Formalism

In cosmology, gas falling into a dark matter halo routinely hits hypersonic speeds, reaching Mach 10 to Mach 100+ (because the primordial intergalactic gas is very cold, its local speed of sound is exceptionally low, allowing infalling gas to achieve massive Mach numbers). At these extreme velocities, the macroscopic kinetic energy (E_{kin}) of the gas completely dominates its internal thermal energy (E_{int}). Note that the ratio of kinetic to internal energy scales with the square of the Mach number.

This creates a numerical problem. In a standard finite-volume solver, we only track the total energy ($E_{tot} = E_{kin} + E_{int}$). To find the gas pressure, we must extract the internal energy by subtracting the kinetic energy:

$$E_{int} = E_{tot} - E_{kin}$$

When E_{kin} is 99.999% of E_{tot}, the limits of floating-point arithmetic cause a catastrophic cancellation. The subtraction destroys the precision of the thermal energy, often resulting in exactly zero or even negative internal energies, which instantly crashes the simulation with unphysical negative pressures.

To solve this, modern cosmological codes employ the **Dual Energy Formalism**. Alongside the standard conserved variables, the simulation tracks the internal energy density as an independent, actively advected fluid field. This independent field is updated by its own fluxes and the physical work done by compression or expansion ($-P\nabla \cdot \mathbf{v}$).

The code then employs a dynamic "switch" during the calculation of the primitive variables:

- If the local flow is subsonic or undergoing a strong shock (e.g., where thermal energy is $> 0.1\%$ of the total energy), the code relies on the standard Total Energy equation. This guarantees perfect energy conservation across shock fronts:
$$P = (\gamma - 1) \left(E - \frac{1}{2} \rho |\mathbf{v}|^2 \right)$$
- If the local flow is hypersonic ($E_{int}/E_{tot} < 0.001$), the total energy calculation is deemed mathematically polluted. The code "switches" to calculating pressure strictly from the independently tracked internal energy:
$$P = (\gamma - 1) ie$$

This dual-tracking approach guarantees that the gas temperature remains physically valid even when plummeting into the deepest gravitational wells of the cosmic web.

Coupling Hydrodynamics to the Expanding Universe

In a static box, the hydrodynamic equations are only sourced by gravity and their own internal pressure. However, in a cosmological simulation, the gas, just like the dark matter, is defined in **comoving coordinates** and is therefore subject to the physics of the expanding universe.

To correctly model this, the standard Euler equations must be modified with new "source terms" that account for the expansion. These terms are mathematically identical to the ones used for the N-body particles.

The cosmological effects are applied as "source terms" in the main integrator "Kick" steps, alongside the gravity. During each "Kick" step, an **external source acceleration** is applied to the gas in every grid cell, which is the sum of two components:

1. **Modified Gravity:** The comoving gravitational acceleration, $\mathbf{g}_{\text{comoving}}$, is calculated from the total density of *both* dark matter and gas. This acceleration is then scaled by $1/a^3$ to account for the dilution of physical density as the universe's volume ($V \propto a^3$) increases.
2. **Hubble Drag:** A velocity-dependent "friction" term, $-2H\mathbf{v}$, is added. This term, where H is the Hubble parameter and $\mathbf{v}$ is the gas's peculiar velocity, correctly damps the gas's peculiar momentum as space stretches.

The **external source acceleration** applied to the gas in each cell is therefore:

$$\mathbf{a}_{\text{source}} = \frac{\mathbf{g}_{\text{comoving}}}{a^3} - 2H\mathbf{v}$$

This acceleration, which is a force per unit mass, is then used to update the gas grid's conserved quantities over the half-time-step ($\Delta t/2$):

- **Momentum Update:** The momentum density $\mathbf{S} = \rho\mathbf{v}$ is updated by the force density ($\rho\mathbf{a}_{\text{source}}$):

$$\Delta\mathbf{S} = (\rho\mathbf{a}_{\text{source}})\frac{\Delta t}{2}$$

- **Energy Update:** The total energy density E is updated by the power density (Force $\cdot$ Velocity) delivered by these forces:

$$\Delta E = (\mathbf{v} \cdot \rho\mathbf{a}_{\text{source}})\frac{\Delta t}{2}$$

By applying these cosmological source terms to the gas grid within the same KDK integrator as the dark matter particles, we ensure that both components feel the same gravity and the same cosmic expansion, allowing them to evolve in a physically consistent manner.

Sub-Grid Coupling: The Intra-Cell Blind Spot

Upgrading the gravity engine to Fourier-Split PM solves the stability and accuracy problems for the collisionless dark matter particles. However, in a **hybrid** simulation code—where dark matter is treated as Lagrangian particles and gas is treated on an Eulerian grid—there is an architectural problem.

Dark matter particles have sub-grid resolution; they use the grid for long distances, but the PP step allows them to interact accurately even when they occupy the same grid cell. Gas, however, is limited to the grid. It only feels the gravitational acceleration vector ($\mathbf{g}_{\text{comoving}}$) calculated from the grid's potential.

If we apply the Gaussian filter $\exp(-k^2 r_s^2)$ to fix the gravity solver, we are intentionally blurring the grid's gravitational potential at scales smaller than r_s. Furthermore, since we rely on the Particle-Mesh (PM) gravity solver to act as the bridge between the collisionless dark matter particles and the continuous baryonic gas, in the scale of a single grid cell **the gas and the dark matter are blind to each other**. The gravitational acceleration at the center of cell i is calculated using a symmetric central difference stencil, comparing the potential of its neighbors:

$$a_i = -\frac{\Phi_{i+1} - \Phi_{i-1}}{2\Delta x}$$

Notice that the potential of cell i itself is absent from this equation. The grid calculates the gradient *across* the cell, effectively ignoring the mass *inside* the cell.

Consequently:

1. **Dark matter ignores local gas:** When a dark matter particle interpolates its long-range force from the grid, it feels the pull of distant gas, but it doesn't feel the pull from the gas in the same cell. The short-range Particle-Particle (PP) solver only iterates over other dark matter particles, leaving the gas out of the sub-grid physics.
2. **Gas ignores local dark matter:** The Eulerian gas updates its momentum by reading this same central-difference grid. It feels no gravitational pull toward the dark matter particles moving around inside its own boundaries.

This limitation is known as the **intra-cell blind spot**, or lack of sub-grid resolution. At distances smaller than one grid cell (Δx), there is no gravitational coupling between the two fluids.

The Pseudo-Particle Solution

To bridge this sub-grid gap, we can compress the fluid mass of a cell into a **pseudo-particle** located at the center of the cell. Its mass is the local comoving density multiplied by the cell volume ($m_{\rm gas} = \rho_{\rm cell}\Delta x^3$).

During the PP step, as a dark matter particle searches its neighboring cells for other dark matter, it also calculates the $1/r^2$ Newtonian pull exerted by these stationary gas pseudo-particles.

For the gas to feel the dark matter, we can use a **momentum-conserving back-reaction**. When the dark matter particle calculates the gravitational force it feels *from* the gas, it applies an equal and opposite force *back* to the gas grid's momentum array. This force-matching guarantees that the sub-grid interactions conserve total momentum.

The Shell Theorem

Dark matter in our simulation represents a cold, collisionless fluid. We discretize this fluid into particles, and use small softening lengths (ϵ) so they can orbit each other tightly. Gas, however, cannot collapse into a singularity smaller than a grid cell.

The dark matter particles are flying through smooth, continuous clouds of gas. According to Newton's **Shell Theorem**, as an object moves deeper inside a uniform sphere of mass, the net gravitational pull it experiences smoothly decreases, reaching zero at the center.

To model this fluid physics, we can use a **Cubic Spline Softening Kernel**. We multiply the Newtonian force between a DM particle and a gas pseudo-particle by a weighting function, $W(r/h)$, where the boundary h is set to the cell size (Δx). This mimics the Shell Theorem for a uniform cloud of gas:

- **Outside the cell ($r \geq \Delta x$)**: The weight is $W = 1.0$. The dark matter particle feels the unmodified $1/r^2$ Newtonian pull of the gas mass.
- **Inside the cell ($r < \Delta x$)**: The weight smoothly drops as a cubic polynomial, reaching $W = 0$ at the center. This ramps down the force, representing the particle passing through a continuous fluid medium without experiencing a singularity.

The polynomial we'll use to calculate this weight is derived from the standard M_4 **Cubic Spline Kernel**, used by Monaghan (1992) for Smoothed Particle Hydrodynamics (SPH). In mathematical approximation theory, the M_n family of splines is generated by convolving a uniform boxcar function with itself n times. The M_4 cubic spline is the lowest-order spline that guarantees C^2 continuity—meaning the force and its derivative are smooth—which is required to prevent energy leaks in a symplectic KDK integrator.

Instead of treating the gas pseudo-particle as a uniform sphere, the cubic spline models it as a cloud whose density is highest at the center and smoothly drops to zero at the cell boundary ($h = \Delta x$). For a normalized distance $q = r/h$, the 3D density profile $\rho(q)$ of this cloud is defined as:

$$\rho(q) = \frac{8}{\pi h^3} M_{\rm gas} \begin{cases} 1 - 6q^2 + 6q^3 & \text{if } 0 \leq q < 0.5 \\ 2(1-q)^3 & \text{if } 0.5 \leq q < 1.0 \\ 0 & \text{if } q \geq 1.0 \end{cases}$$

According to Newton's Shell Theorem (a particle moving through this spherically symmetric cloud only feels the gravitational pull of the mass interior to its current radius), the true sub-grid force is scaled by the **fraction of enclosed mass**:

$$F = \frac{G M_{\rm gas} m_{\rm dm}}{r^2} \times \left(\frac{M(< r)}{M_{\rm gas}} \right)$$

To find this enclosed mass fraction, $W(q) = M(< r)/M_{\text{gas}}$, we must integrate the density profile over a spherical volume from the center out to the particle's radius r:

$$W(q) = \frac{1}{M_{\text{gas}}} \int_0^r 4\pi r'^2 \rho(r') dr'$$

By substituting the piecewise M_4 density profile into this integral and solving it, we get the polynomial weighting functions used to scale the force. For a dark matter particle deep inside the cell core ($0 \leq q < 0.5$), the integrated weight is:

$$W(q) = \frac{32}{3}q^3 - \frac{192}{5}q^5 + 32q^6$$

For a dark matter particle in the outer envelope of the cell ($0.5 \leq q < 1.0$), the integrated weight is:

$$W(q) = -\frac{1}{15} + \frac{64}{3}q^3 - 48q^4 + \frac{192}{5}q^5 - \frac{32}{3}q^6$$

When $q \geq 1.0$, the integral evaluates to 1.0, as the particle is outside the gas cloud and feels all of its mass.

The Computational Reality

The pseudo-particle technique is rarely implemented in modern cosmological codes. The reason lies in the computational cost and the philosophy of grid-based solvers.

Instead of performing expensive approximations to compute the forces inside a coarse cell, modern Eulerian codes solve the resolution problem by making the cells smaller. Using **Adaptive Mesh Refinement (AMR)**, these codes dynamically subdivide the grid whenever a region becomes dense, so that the grid itself provides the required spatial resolution.

Initial Conditions for the Gaseous Component

In cosmological simulations, the baryonic gas and the dark matter must be initialized to reflect the physical reality of the universe long after the epoch of recombination. By the time a typical simulation begins, the gas has already spent millions of years falling into the gravitational potential wells excavated by the dark matter.

Therefore, the gaseous component must be initialized in lockstep with the dark matter, sharing its exact density fluctuations and bulk velocity flows, governed by the Zel'dovich approximation.

1. Initial Density The gas density, ρ_{gas}, is not uniform; it must perfectly trace the initial density perturbations of the dark matter. Once the dark matter particles are displaced to their starting positions, their mass is mapped to the Eulerian grid (via schemes like Cloud-in-Cell) to establish the primordial dark matter density field. The gas density in each cell is then set directly proportional to this field, scaled by the cosmic baryon fraction—the ratio of total gas mass to total dark matter mass in the simulation.

2. Initial Peculiar Velocity The gas cannot start "at rest". Because it has been gravitationally influenced by the dark matter for millions of years prior to the start of the simulation, the gas shares the same large-scale velocity flows. To achieve this synchronization, the continuous primordial velocity field is calculated natively on the high-resolution Eulerian grid using the Zel'dovich approximation. The gas grid is directly populated with these continuous velocity vectors, $\mathbf{v}_{\text{pec}}$. The collisionless dark matter particles, rather than carrying the grid, simply sample their individual initial velocities from this same continuous background field based on their starting coordinates.

3. Initial Energy Because we employ the **Dual Energy Formalism**, we must initialize two separate energy fields. First, the explicitly tracked *internal* energy is initialized to a very low, uniform baseline. This ensures the gas starts "cold," meaning its thermal pressure is negligible and too weak to artificially resist the initial gravitational collapse. Second, the *total* energy of the gas grid is initialized as the sum of this internal thermal energy and the macroscopic kinetic energy ($E_k = \frac{1}{2}\rho v^2$) dictated by its initial primordial momentum.

To set the internal energy baseline, we link the internal energy density directly to the physical temperature (T) of the early universe using the ideal gas law:

$$ie = \frac{\rho k_B T}{(\gamma - 1)\mu m_p}$$

Here, k_B is the Boltzmann constant, m_p is the mass of a proton, and μ is the mean molecular weight (which is approximately 1.22 for the primordial, neutral mix of hydrogen and helium). In a typical cosmological simulation starting at a redshift of $z = 49$, the universe has expanded and cooled significantly since the Big Bang. At this epoch, the background gas temperature is roughly 50 K. By plugging this baseline temperature and the local cell density (ρ) into the equation above, we set the initial ie for every cell.

This setup creates a physically robust initial state. At $t = 0$, the simulation consists of two distinct but perfectly synchronized fluids: a collisionless dark matter component and a hydrodynamic gas component, both sharing the exact same primordial density peaks and velocity flows. When the simulation begins, the cosmic web collapses cohesively, with the gas naturally shocking and heating as it flows into the deepening dark matter halos.

Key Literature & Further Reading
Teyssier, R. (2002). *Cosmological hydrodynamics with adaptive mesh refinement. A new high-resolution code called RAMSES.* arXiv:astro-ph/0111367. Available at https://arxiv.org/abs/astro-ph/0111367
Riemann Solvers & MUSCL Schemes:
Toro, E. F. (2009). *Riemann Solvers and Numerical Methods for Fluid Dynamics: A Practical Introduction* (3rd ed.). Springer.

Gas Physics

While the laws of hydrodynamics describe how gas moves, the true engine of galaxy formation is **thermodynamics**—the physics of how gas heats up and cools down. The balance between these two processes acts as a cosmic thermostat, determining whether a gas cloud has enough pressure to resist gravity or whether it will collapse to form stars.

Temperature

First, it must be understood what "temperature" actually means when we talk about the near-vacuum of interstellar or intergalactic space.

Whether in the air around us or in the cosmic voids, the definition of **temperature** is the same: it is a direct measure of the **average random kinetic energy** of the gas particles. It is simply a statement about **how fast the particles are moving** relative to the bulk flow of the fluid, not how much they are interacting.

The difference lies entirely in density and collisions. In the dense air on Earth, countless atoms are constantly colliding with each other, rapidly transferring their kinetic energy—which is why high temperatures feel palpably hot to the touch. In the extremely sparse gas of the cosmos, particles are so far apart that they rarely ever collide.

- A **"hot"** cosmic gas is one where the individual atoms and ions are zipping around at high random speeds. The gas inside a galaxy cluster, for example, can reach millions of degrees. However, because it is less dense than any vacuum we can create on Earth, a physical thermometer placed in this gas would register near absolute zero, as there are not enough particle collisions to transfer that kinetic energy to the glass.
- A **"cold"** cosmic gas is one where the particles are moving relatively slowly.

In the hydrodynamics solver, we explicitly track the specific internal energy of the gas, u (which is the total internal energy density divided by the mass density, ie/ρ). However, to model the quantum processes that govern radiative cooling, we must translate this bulk macroscopic energy into a physical temperature, T, which directly defines the underlying velocity distribution of the particles.

We can map our code's specific internal energy to a physical temperature using the ideal gas law:

$$u = \frac{k_B T}{(\gamma - 1)\mu m_p}$$

By rearranging this equation, we can dynamically calculate the temperature of any gas cell in our simulation:

$$T = \frac{u(\gamma - 1)\mu m_p}{k_B}$$

Here, k_B is the Boltzmann constant and m_p is the mass of a proton. The two remaining parameters describe the specific nature of our cosmic fluid:

- γ is the adiabatic index (5/3 for a monatomic ideal gas).

- μ is the **mean molecular weight**, which represents the average mass of a particle in the gas in units of proton masses. For a primordial, fully ionized mix of 76% hydrogen and 24% helium, $\mu \approx 1.22$.

Gravitational Compression and Shocks

Cosmic gas increases its temperature when energy is added to it from large-scale astrophysical processes. The primary heating mechanism is **gravitational compression**.

As gas is pulled into the gravitational well of a dark matter halo, it accelerates to enormous speeds. When this rapidly falling gas meets the gas that has already accumulated, it collides violently, creating an immense **shock wave**. This shock wave is an almost instantaneous conversion of the gas's ordered, in-falling kinetic energy into disordered, random motion—in other words, heat. This process, known as **virial heating**, can raise the gas temperature to millions of degrees, creating the vast, hot atmospheres we observe in galaxy clusters.

Other significant heating sources include:

- **Supernova Feedback:** The explosive death of massive stars creates powerful blast waves that rip through the surrounding medium, shocking and heating the gas.
- **Radiation:** High-energy photons from stars and active galactic nuclei can ionize atoms, transferring their energy to the gas and heating it.

At this stage in our simulation architecture, the primary heating mechanism—virial heating—is already implemented. Our gravity operator accelerates the gas into the dark matter halos, generating an immense kinetic energy. When this rapidly infalling gas collides, the HLLC solver resolves the violent shock fronts (naturally dissipating the macroscopic kinetic energy into internal thermal energy), while the explicit $-P(\nabla \cdot \mathbf{v})$ source term in our Dual Energy Formalism captures the steady heating of adiabatic compression. With these mechanical operators already in place, the cosmic gas will shock and heat on its own.

Radiative Cooling

A gas cloud in the vacuum of space cannot cool down by conducting heat to a physical surface. The *only* way it can lose thermal energy is by radiating it away in the form of photons (light). This process, known as **radiative cooling**, is crucial for galaxy formation. If a gas cloud cannot cool, its internal thermal pressure will resist the pull of gravity, preventing it from condensing into stars.

Just as we used operator splitting to separate the fluid dynamics from gravity and cosmic expansion, we can integrate radiative cooling as one more independent operator in the sequence. After the gas flows and responds to gravitational forces, we temporarily freeze the macroscopic fluid motion to calculate how much the gas cools over the timestep Δt in every cell of the grid.

The Cooling Function (Λ)

We must determine how fast the gas is radiating energy away. Gas particles turn their kinetic energy into escaping photons through several different quantum mechanisms (such as line emission from atomic electron transitions or Compton scattering).

For the hot plasmas found inside collapsed dark matter halos, the dominant cooling mechanism is **Bremsstrahlung** (German for "braking radiation"), also known as free-free emission. In a fully ionized plasma, a fast-moving, negatively charged free electron will occasionally fly very close to a positively charged ion. The ion's electric field strongly deflects the electron, causing it to decelerate. According to the laws of electromagnetism, any charged particle undergoing a change in its energy or momentum (as measured in an inertial frame) emits radiation. As the electron is deflected by the ion, it emits a high-energy photon (typically an X-ray). Because this photon escapes into space carrying energy with it, the electron must permanently lose an equivalent amount of its own kinetic energy, thereby cooling the gas.

Astrophysicists encapsulate all of these complex light-emitting processes into a single mathematical master equation: the **Cooling Function**, denoted by $\Lambda(T, \rho)$.

The cooling function outputs the **volumetric energy loss rate**—the total amount of energy radiated away per unit volume, per second (typically expressed in erg cm^{-3} s^{-1}). For pure Bremsstrahlung, the cooling function can be approximated as:

$$\Lambda_{\text{brem}} \approx 1.4 \times 10^{-27} \sqrt{T} n_e n_i$$

Notice how this captures the physical reality of the plasma:

1. $\sqrt{T}$ **(The Velocity Limit):** It might seem intuitive that cooling should scale linearly with temperature (T), since thermal energy scales with T. However, cooling is a *rate* (energy lost per second). To emit a photon, an electron must fly past an ion. The rate at which these encounters happen depends on the electron's speed. Because kinetic energy is proportional to temperature ($v^2 \propto T$), the velocity of the electron scales with the square root of the temperature ($v \propto \sqrt{T}$). The cooling rate is fundamentally throttled by how fast the electrons can physically travel between encounters.
2. $n_e n_i$ **(The Collision Probability):** While mass density (ρ) measures the *mass* of the gas in a given volume, **number density** (n) simply counts the *number of particles* in that volume. Here, n_e is the count of electrons and n_i is the count of ions. For a flash of Bremsstrahlung radiation to occur, one electron and one ion must cross paths. The probability of this happening is the product of their populations ($n_e \times n_i$). Because both the number of electrons and the number of ions rise directly with the overall mass density of the gas (ρ), the resulting cooling rate scales with the **density squared** ($\Lambda \propto \rho^2$). This means gas violently compressed in the center of a dark matter halo will cool drastically faster than the diffuse gas on the outskirts.

The proportionallity above ($n_e \times n_i \propto \rho^2$) assumes the gas is a fully ionized plasma. However, around 10,000 K, primordial gas undergoes recombination. Protons capture the free electrons, turning the plasma into a gas of cold, neutral hydrogen and helium atoms. Because there are practically no free electrons or ions left, Bremsstrahlung cooling shuts down and the physical radiative cooling rate (Λ) plummets to zero below 10,000 K.*(Note: In the real universe, as the first stars and quasars ignite, they flood the cosmos with Ultraviolet radiation. This UV background acts as a universal heating source (Γ) that balances against this atomic cooling limit, creating a natural cosmic thermostat that holds the diffuse intergalactic gas steady at around 10,000 K.)*

We will take care of this **atomic cooling limit** later in our implementation.

The Differential Equation

In our Eulerian grid, we explicitly track the internal energy **density**, *ie* (energy per unit volume). However, when calculating how the gas in a specific cell cools over time, it is mathematically cleaner to formulate the differential equation using the **specific internal energy**, u (energy per unit mass). We extract this for each cell by simply dividing the energy density by the mass density ($u = ie/\rho$). Consequently, to balance the equation, we must also convert the volumetric cooling function Λ (energy lost per unit volume) into a specific rate by dividing it by the gas density ρ.

This leaves us with the fundamental ordinary differential equation (ODE) that governs radiative cooling:

$$\frac{du}{dt} = -\frac{\Lambda(T, \rho)}{\rho}$$

While the physics of this equation are straightforward, attempting to solve it numerically inside a simulation introduces a challenge in computational astrophysics. We'll see why in the following section.

Stiff Equations

Up to this point, our simulation has relied entirely on **explicit integration**. In an explicit method (such as the KDK leapfrog scheme), the rate of change is calculated using the current state of the system. This rate is then assumed to remain constant over a small forward timestep, Δt.

Applying a simple Forward Euler explicit integration to the cooling equation ($\frac{du}{dt} = -\frac{\Lambda}{\rho}$), the expression to advance the specific internal energy from its current state (u^n) to its new state (u^{n+1}) is written as:

$$u^{n+1} = u^n - \Delta t \cdot \frac{\Lambda(T^n, \rho^n)}{\rho^n}$$

However, this approach introduces a computational problem. To understand why, we must introduce the concept of **timescales**.

The Cooling Timescale (t_{cool})

Every physical process in the universe happens at a certain speed. In computational physics, this is quantified using a characteristic timescale—roughly, the amount of time it takes for a process to significantly change the state of the system.

For radiative cooling, the **cooling timescale** (t_{cool}) is defined as the time it would take for a gas cloud to radiate away 100% of its current internal thermal energy if it continued cooling at its current rate:

$$t_{\text{cool}} = \frac{u}{|du/dt|} = \frac{\rho u}{\Lambda}$$

This timescale is highly volatile. In the vast, empty voids of the cosmic web, the gas is so diffuse that Λ is practically zero, making t_{cool} billions of years. But when that gas falls into a dark matter halo and compresses, the density ρ increases dramatically. Because the Bremsstrahlung cooling rate scales with density squared ($\Lambda \propto \rho^2$), the cooling timescale shortens accordingly.

In the dense center of a collapsing halo, t_{cool} can drop to just a few thousand years. This creates a conflict with our global timestep Δt, which might be on the order of a few million years:

$$\Delta t_{\text{hydro}} \gg t_{\text{cool}}$$

Limitations of Explicit Integration

When a differential equation contains a timescale that is drastically shorter than the simulation's timestep, mathematicians refer to the equation as **stiff**. Explicit integrators become highly unstable when applied to stiff equations over standard simulation timesteps.

We can see exactly why by looking back at our explicit Forward Euler equation. Let's imagine a dense gas cell where t_{cool} is 10,000 years, but our hydro solver dictates we must take a step of $\Delta t = 1,000,000$ years.

$$u^{n+1} = u^n - (1,000,000) \cdot \frac{\Lambda}{\rho}$$

Because $t_{\text{cool}} = \frac{\rho u}{\Lambda} = 10,000$, we know mathematically that the cooling term ($\frac{\Lambda}{\rho}$) is exactly equal to $\frac{u}{10,000}$. Substituting this in:

$$u^{n+1} = u^n - (1,000,000) \cdot \left(\frac{u^n}{10,000}\right)$$

$$u^{n+1} = u^n - 100 \cdot u^n = -99u^n$$

The explicit solver assumes the gas continues to cool at its initial rate for the entire timestep. It completely overshoots the physical reality—where the gas would cool down, the cooling rate would drop, and the temperature would gently settle. Instead, the solver extracts 100 times more energy than the cell actually contains. The result is a **negative internal energy**, a physical contradiction to a real universe where energy is strictly non-negative.

At first glance, the obvious solution to this overshoot is simply to shrink the global timestep Δt to match the shortest cooling timescale. If the code took tinier steps, the explicit solver would remain stable. However, this is computationally unpractical. Because the timestep dictates the entire simulation cycle, forcing the massive, expensive hydrodynamics and gravity solvers to run hundreds of extra times just to cool a small fraction of dense gas cells isn't a good solution.

The standard solution is to abandon explicit integration for the thermodynamics and adopt a different approach.

Implicit Integration

To deal with the timescale mismatch between the slow hydrodynamics and the fast cooling, cosmological simulations typically use **Implicit Integration** within their cooling operator.

The Backward Euler Method

As we saw earlier, explicit integration fails because it calculates the cooling rate using the *current* temperature and blindly projects it forward, resulting in massive overshoots. To fix this, we can use an **implicit** method, like the **Backward Euler** scheme.

Instead of asking, *"How fast is the gas cooling right now?"*, the Backward Euler method asks, *"What future temperature would perfectly justify the energy lost to get there?"*

Mathematically, we evaluate the cooling function Λ not at the old state (u^n), but at the **unknown future state** (u^{n+1}):

$$u^{n+1} = u^n - \Delta t \cdot \frac{\Lambda(T^{n+1}, \rho)}{\rho}$$

This subtle shift in the equation is very convenient. Because the cooling rate Λ drops rapidly as the temperature drops ($\Lambda \propto \sqrt{T}$), evaluating it at the colder, future state creates an automatic, self-regulating feedback loop.

If we take a massive timestep (Δt), the equation assumes the gas rapidly cools down early in the step and spends the rest of the time radiating at a very slow, gentle rate. This guarantees **unconditional stability** (meaning the numerical solution will never mathematically diverge or 'blow up', regardless of the size of the timestep Δt). No matter how large the timestep is, the Backward Euler method will gracefully slide the internal energy down towards zero without ever overshooting into negative, unphysical numbers.

Newton-Raphson Root-Finding

While the Backward Euler equation is stable, it introduces a new problem: the equation cannot be solved directly. The unknown future state, u^{n+1}, is embedded within the non-linear cooling function $\Lambda(u^{n+1})$.

To solve for it, we must rewrite the equation as a root-finding problem. We define a function $f(u^{n+1})$ such that the correct physical answer occurs exactly when the function equals zero:

$$f(u^{n+1}) = u^{n+1} - u^n + \Delta t \cdot \frac{\Lambda(u^{n+1}, \rho)}{\rho} = 0$$

To find this root, we can employ the **Newton-Raphson method**. The process begins with an initial guess (usually assuming the future energy will remain equal to the current energy, $u_{\text{guess}} = u^n$). The slope (the derivative) of the function is then calculated at that initial guess and used to extrapolate a new, more accurate guess closer to zero.

The iterative update formula is:

$$u_{\text{next}} = u_{\text{guess}} - \frac{f(u_{\text{guess}})}{f'(u_{\text{guess}})}$$

Here, f' is the derivative of the root-finding function with respect to the internal energy. It is standard practice to compute this derivative numerically—by evaluating Λ at u_{guess} and at a slightly offset value to determine the local slope. This numerically decouples the root-finding algorithm from the specific physics being modeled, allowing the cooling function to be easily upgraded later with complex, tabulated atomic chemistry data without ever requiring a hard-coded analytical derivative.

The solver repeats this process, rapidly converging toward the true future energy. Once the guess stops changing (reaching a strict tolerance), the solver exits. Furthermore, we can easily inject a hard **temperature floor** (e.g., the temperature of the Cosmic Microwave Background, or the 10,000 K **atomic cooling limit** established earlier) into this solver; if a Newton-Raphson guess ever drops below this floor, the solver immediately overrides the answer to the floor value and exits.

This iterative engine allows the simulation to cool the gas, bridging the gap between the millions of years of cosmic expansion and the thousands of years of atomic radiation.

Coupling Cooling to the Simulation

The implicit solver calculates how much thermal energy the gas radiates away during a timestep. Now this must be integrated in the simulation.

In the hydrodynamics solver, we established the **Dual Energy Formalism** to survive hypersonic flows. This means the grid tracks two different energy variables for every cell: the total energy density ($E = E_{\text{kin}} + E_{\text{int}}$) and the internal thermal energy density ($ie = E_{\text{int}}$).

Radiative cooling strictly removes *thermal* energy. The photons escaping into space carry away heat, but they do not slow down the macroscopic bulk flow of the gas. The kinetic energy ($\frac{1}{2}\rho v^2$) must be preserved.

Subtracting the radiated thermal energy, ΔE_{vol}, solely from the independently tracked internal energy (ie) introduces a thermodynamic inconsistency. During the subsequent hydrodynamic step, the Dual Energy switch

may evaluate the cell, detect a kinetically dominated flow, and overwrite the recently cooled internal energy by recalculating it directly from the total energy ($ie = E - E_{\rm kin}$). Because the total energy array remained unmodified, this recalculation would artificially restore the lost thermal energy, causing the gas to spontaneously reheat.

To extract the radiated light while preserving the kinetic velocity of the gas, the lost thermal energy must be substracted from **both** arrays:

$$ie_{\rm new} = ie_{\rm old} - \Delta E_{\rm vol}$$
$$E_{\rm new} = E_{\rm old} - \Delta E_{\rm vol}$$

Because $E_{\rm new} - ie_{\rm new} = (E_{\rm old} - \Delta E_{\rm vol}) - (ie_{\rm old} - \Delta E_{\rm vol}) = E_{\rm old} - ie_{\rm old}$, the kinetic energy remains untouched. The gas cools down, the pressure drops, but the fluid continues to flow at its exact physical speed.

Thermodynamics in Comoving Coordinates

In our cosmological simulation, the Eulerian grid is fixed in comoving space. While the comoving volume of a cell remains perfectly constant, the true physical volume it represents stretches with the scale factor, a.

Since the comoving velocity is defined as the time derivative of the comoving position: $v_{\rm code} = \dot{x}$, the physical peculiar velocity of the gas is $v_{\rm phys} = a \cdot v_{\rm code}$. Because kinetic energy scales with the square of the velocity, the physical kinetic energy natively carries an a^2 dependence ($E_{\rm kin,\ phys} \propto a^2 v_{\rm code}^2$).

To maintain absolute thermodynamic consistency within the Dual Energy Formalism, the internal energy must be scaled identically before it can be summed with the kinetic energy. Therefore, the internal energy density tracked by the solver ($ie_{\rm code}$) relates to the physical internal energy density ($ie_{\rm phys}$) via this exact scale factor:

$$ie_{\rm phys} = a^2 \cdot ie_{\rm code}$$

This mathematical transformation dictates every interaction between the cooling module and the hydrodynamic grid:

1. Temperature and Cooling Conversions When computing the physical temperature of the gas or querying complex, tabulated cooling functions (Λ), the solver cannot use the array values directly. It must first recover the physical specific internal energy by multiplying the code variables by a^2. Conversely, once the physical cooling rate is calculated, the required energy deduction ($\Delta E_{\rm vol}$) must be mathematically mapped back to the grid by dividing the physical value by a^2 before applying the subtraction.

2. Adiabatic Expansion (PdV Work) This coordinate transformation fundamentally alters the integration of adiabatic expansion. In a physical universe, a gas cools adiabatically as its volume expands, governed by its specific adiabatic index (γ). According to standard thermodynamics, the temperature of an expanding gas scales with its density as $T \propto \rho^{\gamma-1}$. Because the physical density drops proportionally to the expanding volume ($\rho \propto a^{-3}$), the temperature—and therefore the physical internal energy for the constant mass of gas within a comoving cell—scales as:

$$ie_{\rm phys} \propto (a^{-3})^{\gamma-1} = a^{-3(\gamma-1)}$$

Taking the time derivative of this mathematical relationship reveals the exact rate of energy loss. Applying the chain rule to the scale factor dependence yields:

$$\frac{d}{dt}(ie_{\rm phys}) \propto -3(\gamma-1)a^{-3(\gamma-1)-1}\dot{a}$$

By factoring out the original $a^{-3(\gamma-1)}$ scaling and substituting the definition of the Hubble parameter ($H = \frac{\dot{a}}{a}$), it becomes clear that the physical internal energy strictly decays at a rate of $-3(\gamma-1)H$:

$$\frac{d}{dt}(ie_{\rm phys}) = -3(\gamma-1)\left(\frac{\dot{a}}{a}\right)a^{-3(\gamma-1)} = -3(\gamma-1)H \cdot ie_{\rm phys}$$

(Note: For a standard monoatomic gas where $\gamma = 5/3$, this perfectly recovers the classical physical decay rate of $-2H$.)

However, because the simulation arrays must store the scaled ie_{code} variable, the rate of change for the grid is governed by the chain rule:

$$\frac{d}{dt}(a^2 \cdot ie_{\text{code}}) = -3(\gamma - 1)H(a^2 \cdot ie_{\text{code}})$$

Evaluating the derivative on the left side (and noting that $\dot{a}/a = H$) yields:

$$a^2 \frac{d}{dt}(ie_{\text{code}}) + 2Ha^2 \cdot ie_{\text{code}} = -3(\gamma - 1)Ha^2 \cdot ie_{\text{code}}$$

Subtracting the expansion term to isolate the update rate for the simulation array results in:

$$a^2 \frac{d}{dt}(ie_{\text{code}}) = [-3(\gamma - 1) - 2]\, Ha^2 \cdot ie_{\text{code}}$$

By factoring out the a^2 scaling on both sides and simplifying the mathematical coefficient (since $-3\gamma + 3 - 2 = -3\gamma + 1 = -(3\gamma - 1)$), the generalized code update rate emerges:

$$\frac{d}{dt}(ie_{\text{code}}) = -(3\gamma - 1)H \cdot ie_{\text{code}}$$

The physical thermodynamic cooling ($-3(\gamma - 1)H$) intrinsically combines with the coordinate system's mathematical stretching ($-2H$) to produce a strict $-(3\gamma - 1)H$ decay requirement. To consistently simulate the cosmological PdV work and prevent the expansion of the universe from artificially heating the gas, the integrator must drain the internal and total energy arrays at this mathematically derived rate.

The Optically Thin Approximation

In our implementation of radiative cooling, when a gas cell cools down, we mathematically subtract that thermal energy from the grid and permanently delete it from the simulation. From a strict conservation standpoint, this means our simulated universe is an open system leaking energy.

In the real universe, a photon emitted by a decelerating electron could indeed travel across space, strike another atom, and transfer its energy back into heat. To simulate this accurately, we would need to upgrade our hydrodynamics engine to **Radiation Hydrodynamics (RHD)** by solving the Radiative Transfer equation.

However, doing so is computationally prohibitive. To avoid this, standard cosmological codes rely on the **Optically Thin Approximation**. We assume that the cosmic gas is "optically thin," meaning it is transparent to its own radiation.

For the hot plasmas found inside collapsed dark matter halos, this is a good physical assumption. Even though the gas is compressed by gravity, cosmological densities are still a hard vacuum. When a high-energy X-ray photon is emitted via Bremsstrahlung cooling, its mean free path is so large that it will almost certainly sail entirely out of the galaxy, out of the dark matter halo, and completely out of the simulation box without ever striking another atom. Therefore, assuming the photon escapes into the void and permanently deleting its energy from the computational domain is a reliable representation of the physics.

There are, of course, specific environments where this approximation breaks down. When gas condenses into star-forming molecular clouds, it becomes "optically thick" and opaque, trapping heat inside. Furthermore, during the Epoch of Reionization, dense neutral hydrogen violently absorbed incoming Ultraviolet light from the first stars. To account for these complex radiation effects without actually running a Radiative Transfer solver, modern simulations employ sub-grid approximations—mathematical rules that reproduce physics occurring on scales smaller than a single grid cell. A common technique is to assume the universe is bathed in a uniform, invisible glow of UV light, which we mimic by enforcing an artificial temperature floor—such as the 10,000 K atomic cooling limit—preventing the diffuse gas from cooling below the thermodynamic baseline set by this universal radiation background.

The Subgrid Clumping Factor

In an ideal, infinite-resolution simulation, the Eulerian equations of hydrodynamics capture the collapse of gas into Dark Matter halos. However, computational cosmology is limited by grid resolution. When modeling large cosmological volumes, the physical size of a single grid cell may span tens or hundreds of kiloparsecs. This discretization introduces a thermodynamic problem: the artificial suppression of radiative cooling.

The Problem of Eulerian Smoothing

In reality, gas is not a uniform fog; supersonic turbulence (chaotic, swirling macroscopic gas motions driven by colliding inflows) shreds it into a multiphase medium consisting of dense, cold knots surrounded by hot, sparse bubbles.

Radiative cooling mechanisms, such as Bremsstrahlung (free-free emission), are two-body collisional processes. Consequently, the cooling rate (Λ) is proportional to the square of the gas density:

$$\Lambda \propto \rho^2$$

Because an Eulerian solver assigns a single, smoothed average density ($\langle\rho\rangle$) to an entire cell, the grid calculates the cooling rate based on the square of that average. However, due to a mathematical principle known as **Jensen's Inequality**, the square of an average is smaller than the average of the squares:

$$\langle\rho\rangle^2 < \langle\rho^2\rangle$$

By smoothing out the dense clumps, the grid underestimates the true cooling rate. The thermal energy becomes trapped inside the cell, causing artificial pressure to build up. This numerical artifact results in an "adiabatic bounce," where the gas is blasted out of the gravitational well rather than condensing into a galactic core.

The Subgrid Clumping Model

To solve this without requiring computationally expensive grid resolutions, we employ a **Subgrid Clumping Factor** (often denoted as C). This model injects the missing, small-scale physics back into the macroscopic grid cells.

The clumping factor is defined as the ratio of the average of the squared densities to the square of the average density:

$$C = \frac{\langle\rho^2\rangle}{\langle\rho\rangle^2}$$

To implement this dynamically, we tie the subgrid variance to the macroscopic environment. As a cosmological structure collapses, supersonic turbulence increases, leading to more severe clumping. Therefore, the clumping factor can be modeled as a function of the local overdensity (Δ), which is the ratio of the local cell density to the mean density of the universe:

$$\Delta = \frac{\rho_{local}}{\bar{\rho}}$$

In our model, we adopt a generalized power-law scaling for collapsing regions ($\Delta > 1$) to account for unresolved gas structures. The modified cooling factor is computed as:

$$C = 1 + A\Delta^B$$

While the conceptual foundation for scaling subgrid clumping with local overdensity and turbulence is well-established in studies (e.g., Mao et al. 2020; Federrath et al. 2008), this formulation is a custom fit where the parameters A and B are empirically calibrated based on the physics and resolution of the simulation:

- **The Exponent (B):** This parameter captures how fast the subgrid gas fragments into dense knots as the halo collapses. As the gas compresses, its rising temperature provides thermal pressure support against further fragmentation. Because this thermal resistance grows stronger as the density increases, the efficiency of subgrid fragmentation gradually diminishes, preventing the clumping from scaling proportionally with the macroscopic overdensity. Thus B is set to be sublinear (< 1.0).
- **The Amplitude (A):** This parameter serves as a correction factor specific to the simulation's grid resolution. It represents the degree to which the mesh fails to resolve the small-scale clumping. This amplitude is calibrated empirically so that the simulation matches macroscopic physical observables, such as the cosmic star formation rate or the global cold gas fraction.

The subgrid clumping factor acts as a correction to the squared average density of the grid cell:

$$\Lambda_{true} = (C \times \langle\rho\rangle^2) f(T)$$

where $f(T)$ represents the temperature-dependent physics of the cooling function. By enabling this subgrid model, the simulation acknowledges that overdense cells contain unresolved, dense knots of gas. The thermal energy is radiated away, allowing the pressure to drop and the gas to undergo collapse.

To calibrate the subgrid clumping amplitude (A), we require the simulation to reproduce the macroscopic condensed baryon fraction at $z = 0$. Because our hydrodynamics model does not include a subgrid recipe for star formation, gas that cools below $T < 10^4$ K and reaches halo overdensities of $\Delta > 100$ remains trapped in the fluid phase. Therefore, our simulated cold dense gas fraction acts as a proxy for the total collapsed baryon budget—comprising both the interstellar medium (ISM) and stellar mass.

Observational baryon censuses in the local universe indicate that stars and stellar remnants account for roughly 5% to 7% of the total cosmic baryon density, while cold neutral gas (H I, He I and H_2) accounts for an additional 1.5% to 2% (Shull, Smith, & Danforth 2012). Consequently, we tune the parameter A such that the final simulated cold dense gas mass fraction at $z = 0$ settles within the interval 7% to 10%.

Key Literature & Further Reading
Radiative Physics:
Rybicki, G. B., & Lightman, A. P. (1979). *Radiative Processes in Astrophysics.* Wiley-VCH. (See Chapter 5 for the derivation of the Thermal Bremsstrahlung cooling rates).

Mo, H., van den Bosch, F. C., & White, S. (2010). *Galaxy Formation and Evolution.* Cambridge University Press. (See Chapter 8 for the application of cooling functions in cosmological structure formation).

Numerical Implementation of Stiff Cooling:
Anninos, P., Zhang, Y., Abel, T., & Norman, M. L. (1997). *Cosmological Hydrodynamics with Multi-Species Chemistry and Nonequilibrium Ionization.* New Astronomy, 2(3), 209-224. Available at https://arxiv.org/abs/astro-ph/9608041. (Details the standard use of operator-split, implicit backward-Euler integration with Newton-Raphson iterations for cosmological cooling).

Subgrid clumping factor:
Daisuke Nagai, Erwin Lau. (2011). *Gas Clumping in the Outskirts of Lambda-CDM Clusters.* The Astrophysical Journal. Available at https://arxiv.org/abs/1103.0280.

Validation of the Hydrodynamic Solver

While the N-body component of our simulation is validated by checking its adherence to conservation laws (energy, momentum) in a static universe, validating the hydrodynamic component is more complex. The goal is not simply to conserve energy; in fact, hydrodynamic shocks are *designed* to convert kinetic energy into thermal energy, a process that must be captured correctly.

A hydrodynamic solver is instead validated by its ability to accurately reproduce the known analytical solutions to a set of classic, standardized test problems. These tests are the "unit tests" of computational fluid dynamics.

Conservation in a Closed Box

The most fundamental test is to ensure the solver correctly conserves all quantities in the absence of external forces.

- **The Setup:** A periodic, non-expanding box is initialized with a random distribution of gas densities, pressures, and velocities. The gravitational solver is turned off.
- **The Validation:** The simulation is run for many time steps. At each step, the code must verify that the following total quantities, summed over all grid cells, remain constant to machine precision:
  1. **Total Mass:** $M_{total} = \sum_i \rho_i L^3$
  2. **Total Momentum:** $\mathbf{P}_{total} = \sum_i (\rho \mathbf{v})_i L^3$
  3. **Total Energy:** $E_{total} = \sum_i E_i L^3$
- **What it Proves:** This test confirms that the numerical flux calculations are perfectly balanced—that any mass, momentum, or energy that leaves one cell correctly enters its neighbor, with no numerical "leaks" or "sources."

The 1D Shock-Tube

This test has a known, exact analytical solution (the **Sod Shock Tube** is the most famous variant) that validates the code's ability to handle all three fundamental wave structures.

- **The Setup:** A 1D tube of gas is initialized with a "diaphragm" at its center. The gas on the "Left" state has a high density and pressure, while the gas on the "Right" state has a low density and pressure. At $t = 0$, the diaphragm is removed.
- **The Expected Result:** The collision of the two states generates a self-similar wave structure (meaning its shape remains perfectly constant as it stretches over time). This structure is complex, splitting into three distinct features (see below).
- **The Validation:** After evolving the system to a time t, a snapshot of the simulation's density, pressure, and velocity along the 1D line is plotted. This numerical result must be compared directly against the known, exact mathematical solution. A successful test will correctly capture the speed, position, and amplitude of the three key features:
  1. A **Shock Wave** (an abrupt, discontinuous compression) propagating into the low-density region.
  2. A **Rarefaction Fan** (a smooth, continuous expansion) propagating back into the high-density region.
  3. A **Contact Discontinuity** (a jump in density, but not pressure) separating the two materials. Capturing this specific feature sharply, without excessive smearing, is the primary benchmark for the **HLLC Riemann solver**.

The Sedov-Taylor Blast Wave (Point Explosion)

This is the classic multi-dimensional test for how a code handles a powerful, symmetric explosion, such as a supernova.

- **The Setup:** A uniform, low-density gas fills the grid, initially at rest. At $t = 0$, a very large amount of thermal energy is deposited into a single central cell.
- **The Expected Result:** A strong, spherical shock wave propagates outwards from the center, sweeping the surrounding gas into a dense, hot shell. Because this explosion creates extreme hypersonic velocities (high Mach numbers), this is a premier stress-test for the **Dual Energy Formalism** and the **MUSCL slope limiters**, proving the code can handle violent energy conversions without crashing due to negative pressures.
- **The Validation:** This test has a known self-similar solution and is validated in two ways:
  1. **Symmetry:** The shock front must remain perfectly spherical. Any "boxy" or distorted shape indicates that the dimensional splitting in the solver is introducing errors.
  2. **Propagation Speed:** The radius of the shock front, R, must grow with time, t, according to a specific power law. For an explosion in a uniform medium, this is $R(t) \propto t^{2/5}$. A log-log plot of radius versus time must produce a straight line with a slope of $2/5$.

The Kelvin-Helmholtz Instability

This test is not about shocks, but about the code's ability to correctly model fluid mixing and the growth of instabilities.

- **The Setup:** A 2D box is initialized with two layers of fluid sliding past each other in opposite directions. For example, the top half has a velocity $v_x = +v$ and the bottom half has $v_x = -v$. A tiny, sinusoidal perturbation is introduced at the interface.
- **The Expected Result:** The shear at the interface is unstable. The small initial perturbation should grow exponentially, causing the interface to roll up into a characteristic series of vortices.
- **The Validation:** This is a test of the solver's numerical diffusion. Resolving these vortices correctly is a major victory for **second-order spatial reconstruction** and the **HLLC solver**. Simpler, first-order solvers (like standard HLL) introduce so much artificial viscosity that they smear out the interface, artificially damping the instability and preventing the vortices from ever forming.

Passing this standard suite of tests provides strong confidence that a hydrodynamic code is correctly solving the Euler equations and is ready for use in complex physical simulations.

Adaptive Timestep

Computational cosmology simulations are filled with different components. Dark matter particles interact only through gravity, a long-range force that can be relatively slow. Baryonic gas, however, interacts through

hydrodynamic pressure, leading to shock waves that propagate at very high speeds, and it radiates thermal energy, which can cause temperatures to drop very fast.

This creates a challenge: the simulation evolves on many different timescales simultaneously. If we were to use a single, "fixed" timestep (Δt) for the entire simulation, we would be forced to choose the *smallest* possible timescale. This would make the simulation to waste resources where bigger steps could be safely taken.

A solution to this problem is the **adaptive timestep**. At every cycle, the shortest timescale required to maintain stability for every physical component is computed. The final timestep used to advance the simulation is the minimum of all of these, ensuring both physical accuracy and computational efficiency.

The Courant-Friedrichs-Lewy (CFL) Condition for hydrodynamics

For the grid-based hydrodynamic solver, the most restrictive limit is the **Courant-Friedrichs-Lewy (CFL) condition**. This condition is based on the principle that information (i.e., a sound wave or the fluid itself) must not be allowed to travel more than one grid cell (Δx) in a single timestep (Δt).

If a parcel of gas moves two cells in one step, the numerical solver for the adjacent cell never "sees" it, leading to a breakdown of the solution.

To enforce this, we must first find the maximum "signal velocity" (v_{signal}) anywhere in the simulation. This is the sum of the bulk fluid velocity (v) and the local sound speed (c_s). The sound speed itself depends on the gas pressure (P) and density (ρ) through the adiabatic index (γ):

$$c_s = \sqrt{\frac{\gamma P}{\rho}}$$

$$v_{\text{signal}} = v + c_s$$

The simulation must find the maximum signal velocity across all N cells, $v_{\max} = \max(v_{\text{signal},i} \text{ for } i \in [1, N])$. The CFL timestep limit is then the time it would take this fastest signal to cross one cell, scaled by a "safety factor" (C_{CFL}, typically 0.1 to 0.5) to ensure stability:

$$\Delta t_{\text{CFL}} = C_{\text{CFL}} \cdot \frac{\Delta x}{v_{\max}}$$

The Cooling Timestep Limiter

The implicit Backward Euler method guarantees that our thermodynamics solver will never crash, even if the timestep is absurdly large. It is unconditionally stable. However, "stable" does not mean "accurate".

If t_{cool} in the center of a dense halo is 10,000 years, and we take a single massive timestep of 10 million years, the implicit solver will safely drop the temperature to the cosmic floor. But in reality, over those 10 million years, that gas might have been hit by a shockwave, compressed further by gravity, or pushed out of the halo entirely. Taking a massive leap forward blinds the simulation to the complex, shifting dynamics of galaxy formation.

To maintain physical accuracy, we must force the simulation to respect the speed of the thermodynamics. We do this by adding a new constraint to our architecture: the **cooling timestep limiter**.

During every cycle, the simulation evaluates the cooling timescale ($t_{\text{cool}} = \frac{\rho u}{\Lambda}$) for every single gas cell on the grid. We then mandate that the global simulation timestep cannot exceed a small safety fraction (typically 10%) of the shortest cooling time found anywhere in the universe:

$$\Delta t_{\text{cool}} = 0.1 \times \min\left(\frac{\rho u}{\Lambda}\right)$$

This forces the simulation to take smaller, more frequent steps whenever a gas cloud enters a phase of violent, runaway cooling. By restricting the clock, we ensure that the implicit solver only ever has to predict cooling over a modest, physically reasonable interval.

The Gravitational Timestep

For the N-body (particle) integrator, a different stability criterion applies. Particles in a strong gravitational field (e.g., near a dense halo or during a close encounter) experience high acceleration. If the timestep is too large, the integrator will "overshoot" the correct trajectory, artificially adding energy to the system and making it unstable.

The principle here is that the timestep must be a small fraction of the local dynamical time, which can be defined as the time it takes a particle to cross the gravitational softening length (ϵ) given its current acceleration.

This is a kinematic constraint. From basic kinematics ($x = \frac{1}{2}at^2$), the time to travel a distance ϵ under acceleration a is $t \sim \sqrt{\epsilon/|a|}$.

To ensure stability for all particles, the simulation must find the maximum acceleration experienced by any particle, $a_{\max} = \max(|a_i|)$. The gravitational timestep limit is then:

$$\Delta t_{\text{grav}} = C_{\text{grav}} \cdot \sqrt{\frac{\epsilon}{a_{\max}}}$$

Where C_{grav} is a dimensionless safety factor (e.g., 0.1 to 0.3) that controls the accuracy of the particle integration.

The Global Timestep

At each cycle, the simulation computes all relevant timestep limits. The **global timestep**, Δt, which will be used to advance *all* components (particles, gas, and thermodynamics) from time t to $t + \Delta t$, must be the single most restrictive (smallest) value.

Furthermore, it is common practice to introduce a user-defined maximum timestep, $\Delta t_{\max}$. This acts as a "ceiling," preventing the timestep from becoming excessively large in quiet regions of the simulation (which could reduce accuracy) and ensuring that data snapshots are saved at reasonably regular intervals.

The final global timestep is therefore the minimum of all constraints:

$$\Delta t = \min(\Delta t_{\text{CFL}}, \Delta t_{\text{grav}}, \Delta t_{\text{cool}}, \Delta t_{\max})$$

This ensures that the simulation proceeds as fast as possible while respecting the physical constraints present anywhere in the computational domain, guaranteeing that no part of the simulation evolves into an unstable, non-physical state.

Key Literature & Further Reading
Bryan, G. L., Norman, M. L., O'Shea, B. W., et al. (2014). *ENZO: An Adaptive Mesh Refinement Code for Astrophysics*. The Astrophysical Journal Supplement Series, 211(2), 19. arXiv:1307.2265. Available at https://arxiv.org/abs/1307.2265

Subcycled Operator Splitting

In the previous chapters, we proposed the use of an adaptive timestep. To maintain numerical stability, the global simulation clock must tick forward at a rate dictated by the fastest physical process occurring anywhere in the simulation.

However, this creates a computational bottleneck. A simulation consists of multiple physical operators—gravity, hydrodynamics, and thermodynamics. These processes evolve on different timescales. At a given instant, particle-particle (PP) gravitational interactions might require a timestep of just 10,000 years to remain stable. Meanwhile, the hydrodynamics solver might allow a step of 2 million years.

If we force the entire simulation to step together using a single global timestep (the minimum of all constraints), we still waste computational power. We are forcing the hydrodynamics solver to run hundreds of times simply because gravity is acting quickly.

The **Subcycled Operator splitting** is an architectural solution to this timescale mismatch. Instead of locking all physics to the tightest constraint, we decouple the operators. The simulation advances using a large global timestep (the "macro-step") dictated by the *slowest* physics, while the faster, more restrictive physical processes run in a localized loop (the "micro-steps") to catch up.

Adaptive Multirate Operator Splitting

The standard Kick-Drift-Kick (KDK) integrator separates the physics sequentially: it applies a gravitational "Kick" for half a step, a "Drift" for a full step, and then a second "Kick" for a half step. This symmetric $A/2 \to B \to A/2$ structure is mathematically known as **Strang Splitting**, and it guarantees second-order accuracy. Operator subcycling takes this splitting a step further by nesting loops inside the operators without breaking the outer symmetry. A robust way to handle this in a cosmological context is through an **Adaptive Multirate Operator Splitting scheme**.

This dynamic subcycling engine evaluates both the gravitational timestep limit ($\Delta t_{\rm grav}$) and the hydrodynamic CFL limit ($\Delta t_{\rm hydro}$) at the start of every cycle. It then designates the *larger* of the two as the global **macro-step**, and the *smaller* to be subcycled.

$$\Delta t_{\rm macro} = \max(\Delta t_{\rm hydro}, \Delta t_{\rm grav})$$

This macro-step is then capped by the cosmological expansion limiter (e.g., ensuring the universe expands by no more than 1% per step) and any user-defined maximums to ensure the simulation doesn't skip past critical output snapshots.

- **If Gravity is Slow ($\Delta t_{\rm grav} > \Delta t_{\rm hydro}$):** The overall cycle advances by the gravity timestep. Because the gas flows faster than the dark matter, it must take multiple sub-steps. However, if the gas were to drift without feeling gravity for the entire macro-step, it would artificially expand out of dark matter halos. To solve this, we employ a predictor-corrector approach: we compute an approximation of the future gravitational field to guide the gas, and linearly interpolate it, allowing the gas to undergo a full sequence of micro-KDK steps against a smoothly evolving background potential.

  1. **Compute Macro-Forces & Macro-Kick 1:** Calculate (or retrieve) the current gravitational field and save it as our "old" state. Apply the gravity and expansion kicks to the dark matter for $\Delta t_{\rm macro}/2$.
  2. **Macro-Drift (Dark Matter):** Advance the collisionless dark matter positions by $\Delta t_{\rm macro}$.
  3. **Predictor Force Solve:** Advance the global scale factor to the end of the macro-step and solve Poisson's equation. Because the dark matter has moved but the gas has not, this yields an accurate *predictor* of the future gravitational field.
  4. **Interpolated Gas Subcycling:** Enter an iterative subcycling loop. Dynamically re-evaluate $\Delta t_{\rm hydro}$ based on the shifting gas state, and run the hydrodynamics solver repeatedly until the gas time catches up to $\Delta t_{\rm macro}$. Inside this loop, the gas undergoes its own KDK sequence:
     - **Interpolation:** Calculate a blending factor (α) based on the current sub-step time. Linearly interpolate the "old" and "predictor" scale factors, Hubble parameters, and gravitational forces to the midpoint of the sub-step.
     - **Micro-Kick 1 (Gas):** Apply the interpolated gravity and expansion kicks to the gas for $\Delta t_{\rm hydro}/2$.
     - **Hydro Drift:** Advance the fluid solver (and apply radiative cooling) for $\Delta t_{\rm hydro}$.
     - **Micro-Kick 2 (Gas):** Apply the interpolated gravity and expansion kicks to the gas for the remaining $\Delta t_{\rm hydro}/2$.
  5. **Corrector Force Solve:** With the gas and dark matter now fully synchronized at the end of the macro-step, solve Poisson's equation a second time to obtain the true, final gravitational field.
  6. **Macro-Kick 2 (Dark Matter):** Apply the true, synchronized gravity kick to the dark matter for the remaining $\Delta t_{\rm macro}/2$.

- **If Gravity is Fast ($\Delta t_{\rm grav} < \Delta t_{\rm hydro}$):** The overall cycle advances by the hydrodynamics timestep. To maintain the Strang splitting symmetry, we must split the gravity subcycles into two halves wrapping the single hydro step.

  1. **First Gravity Half-Cycle:** Enter a `while` loop to advance gravity by $\Delta t_{\rm macro}/2$. Inside this loop, the particles undergo smaller KDK steps (Kick, Drift, Recompute Forces, Kick). Because the universe is expanding, the scale factor (a) and Hubble parameter (H) used for these micro-kicks must be linearly interpolated to the sub-step time. During this phase, the gas density is "frozen", providing a static background potential for the dark matter.
  2. **Macro-Drift (Gas):** The hydrodynamics solver takes a single, large step to advance the gas grid by $\Delta t_{\rm macro}$.
  3. **Second Gravity Half-Cycle:** Enter a second `while` loop to advance the gravity by the remaining $\Delta t_{\rm macro}/2$, using the newly updated, post-hydro gas density to source the Poisson solver.

Decoupling the Thermodynamics

Noticeably absent from the macro-step logic is the cooling timestep (Δt_{cool}). Because radiative cooling is a stiff equation, we utilize an implicit integration scheme (the Newton-Raphson Backward Euler method). This way, cooling is unconditionally stable over large leaps in time. We handle it through **local cell subcycling**.

During the Hydrodynamics "Drift" stage, whether it is taking a micro-step or a macro-step, the fluid solver iterates over the grid. If a specific gas cell has a cooling timescale shorter than the current hydro step, that individual cell enters its own private `while` loop.

To determine the size of these micro-steps, we evaluate the instantaneous cooling timescale ($t_{\text{cool}} = \frac{\rho u}{\Lambda}$) for the cell. To ensure the implicit solver remains accurate, the cell restricts its thermodynamic micro-step to a safe fraction—typically 10%—of this timescale:

$$\Delta t_{\text{cell}} = 0.1 \times \frac{\rho u}{\Lambda}$$

Breaking the hydro step down into these thermodynamic micro-steps allows the gas to resolve its energy loss without slowing down the simulation.

Accuracy and Symplecticity in a Subcycled Architecture

For a cosmological simulation running for millions of steps, second-order accuracy is a requirement. As introduced earlier, our architecture achieves this through the time-centering of the **Strang Splitting**. Because we apply half the gravity, flow the gas for a full step, and then apply the second half of the gravity, the sequence of operations is perfectly symmetric.

Strang splitting does not dictate the internal complexity of the middle operator; it strictly requires temporal symmetry. In our subcycled architecture, this central operator might consist of a single hydrodynamic advance, or it may encompass an iterative subcycling loop that evolves the gas against an interpolated background potential. From a mathematical perspective, the number of internal micro-steps is irrelevant. Provided the outer operator is advanced for half a timestep before the inner sequence begins, and half a timestep after it concludes, the time-reversibility of the algorithm is preserved, guaranteeing that the global simulation remains second-order accurate.

The Symplectic Nature of Subcycled KDK

As established in earlier chapters, a symplectic integrator (like KDK leapfrog) is designed to preserve the volume of phase space, preventing the energy drift of non-symplectic methods. We will now consider if a subcycled KDK scheme retains this property.

A powerful property of symplectic integrators is that **the composition of any number of symplectic maps is itself a symplectic map.** If we look at the regime where gravity is subcycling, the dark matter takes dozens of tiny micro-KDK steps while the gas grid is temporarily frozen. A single micro-KDK step is symplectic. Therefore, stacking fifty micro-KDK steps in a row is mathematically symplectic. Evolving the gas, and then doing another fifty micro-KDK steps, is still symplectic. Because the subcycling loops are built out of symplectic building blocks, the dark matter integration retains its strict phase-space preservation.

On the other hand, symplectic integration strictly requires the physics to be governed by a **Hamiltonian system**—a framework where processes are fundamentally reversible and the volume of phase space is perfectly conserved. The moment we introduce hydrodynamics, our simulation breaks these rules. Gas physics is inherently dissipative and irreversible. When gas flows collide, supersonic shocks irreversibly generate entropy. Furthermore, with radiative cooling active, the gas actively deletes its own thermal energy by emitting photons into the void. Because these processes permanently destroy phase-space information and leak energy out of the system, the total combined simulation physically cannot be symplectic.

Ultimately, the non-symplectic nature of the combined system is not a consequence of operator subcycling; it is an unavoidable—and necessary—reality of simulating baryonic matter. By marrying these operators in a Strang-split architecture, we maintain the baseline physics of the simulation: the collisionless dark matter remains symplectic, while the gas models the dissipative, irreversible thermodynamics required to forge galaxies—all without sacrificing the global second-order accuracy of the solver.

Key Literature & Further Reading

Springel, V. (2005). *The cosmological simulation code GADGET-2. Monthly Notices of the Royal Astronomical Society*, 364(4), 1105-1134. arXiv:astro-ph/0505010. Available at https://arxiv.org/abs/astro-ph/0505010

Almgren, A. S., Bell, J. B., Lijewski, M. J., Lukić, Z., & Van Andel, E. (2013). *Nyx: A massively parallel AMR code for computational cosmology.* The Astrophysical Journal, 765(1), 39. Available at https://arxiv.org/pdf/1301.4498

Strang, G. (1968). *On the construction and comparison of difference schemes.* SIAM Journal on Numerical Analysis, 5(3), 506-517. Available at https://www.pas.rochester.edu/astrobear/raw-attachment/blog/shuleli08202013/Strang1968.pdf

Analytical Requirements for Resolution and Volume

Because computational resources are finite, choosing the parameters of the simulation requires a trade-off between the overall size of the simulated box (how much of the universe we are going to simulate) and the size of the individual grid cells (at which resolution). If we choose poorly, our virtual universe will fail to represent the actual cosmos.

Cosmic Variance and the Box Size

To simulate a "representative" patch of the universe, the statistical properties of the simulation box—the number of galaxy clusters, the sizes of the voids, the web-like structure of the filaments—should look identical to any other randomly selected patch of the real universe of the same size.

However, the universe is only uniform on extremely large scales. If we look at a small patch of space (e.g., 10 Megaparsecs across), we might accidentally center our view on a massive supercluster, or we might look at an entirely empty void. This statistical uncertainty is known as **Cosmic Variance**.

If a simulation box is too small, it suffers from severe cosmic variance. A small box cannot contain the longest wavelengths of the density field, meaning it will never form massive superstructures. Furthermore, the periodic boundary conditions will cause the few structures that do form to artificially interact with themselves across the boundaries. In cosmology, the accepted threshold for a simulation volume to be considered statistically representative of the large-scale structure is a comoving box length of roughly **100 Mpc** (Megaparsecs) or larger. At this scale, the simulation volume is vast enough to contain a statistically average mix of all cosmic environments.

However, while cosmic variance dictates the minimum size of a simulation, the laws of General Relativity dictate the maximum. Our Newtonian Poisson solver calculates gravity instantaneously across the entire grid. To ensure this approximation remains valid, the comoving length of the box (L) must remain strictly sub-horizon ($L \ll c/H$).

Today, the Hubble radius (c/H) is roughly 4,300 Mpc. If we attempt to push our box size too close to this scale, the instantaneous gravity approximation creates a physical violation, allowing causally disconnected regions of the universe to pull on each other faster than the speed of light. Therefore, the tolerable maximum for a standard Newtonian N-body code without relativistic corrections is generally kept below 1,000 to 2,000 Mpc (the Gigaparsec scale).

Ultimately, cosmological simulations must have a comoving box large enough to defeat cosmic variance ($\geq$ 100 Mpc), but small enough to ignore relativistic light-travel delays ($\leq$ 1,000 Mpc).

A Reference for Cosmic Scales

Because the Megaparsec (Mpc) is an vast unit of distance (1 Mpc $\approx$ 3.26 million light-years), it can be difficult to build an intuition for the scale of a simulation grid. To help anchor these numbers to reality, here is a quick-reference guide to the approximate diameters of common astronomical structures:

Structure	Approximate Diameter (Mpc)	Notes
Earth-Sun Distance (1 AU)	$\sim 5 \times 10^{-12}$ Mpc	The distance light travels in 8 minutes.
The Solar System	~ 0.00001 Mpc	Reaching out to the edge of the Oort Cloud.
Milky Way (Visible Stellar Disk)	~ 0.03 Mpc	The glowing spiral of stars and gas we can see.
Milky Way (Dark Matter Halo)	~ 0.3 Mpc	The invisible gravitational well hosting our galaxy.

Structure	Approximate Diameter (Mpc)	Notes
The Local Group	~ 3.0 Mpc	Our local neighborhood, including Andromeda.
Typical Galaxy Cluster	~ 2.0 to 10.0 Mpc	Hundreds of galaxies bound in a single hot gas node.
Typical Cosmic Void	~ 20.0 to 50.0 Mpc	Vast, underdense regions between filaments.
Representative Simulation Box	$\mathbf{100.0}+$ Mpc	The minimum scale required to combat Cosmic Variance.

Full-size Representative Simulations

When designing a cosmological simulation, the box size (L) and the grid dimension (N) cannot be chosen arbitrarily, and are not only constrained by the cosmic variance. The specific physics modules implemented in the code dictate strict minimum bounds for these parameters.

To run a simulation that accurately captures the correct tendencies—specifically the interplay of structure growth, shock-heating, and radiative cooling—we must balance three analytical constraints: the Cooling Threshold, the Mass Resolution Limit, and the Fundamental Mode.

The Cooling Threshold

The first constraint is defined by the thermodynamics of the baryonic gas. In any hydrodynamical setup, gas is allowed to radiate thermal energy away, but this cooling is ultimately halted at a specific temperature floor (T_{floor}). Depending on the simulation's goals, this floor might represent a true physical barrier (such as the limit of primordial atomic cooling) or an artificial numerical safety net designed to prevent grid singularities.

For the fluid solver to demonstrate the complete cycle of hierarchical structure formation (infall, shock-heating, and subsequent core condensation), the gravity solver must be capable of forming dark matter halos massive enough to shock-heat the infalling gas to at least this T_{floor} threshold.

The analytical relationship between a dark matter halo's mass (M_{vir}) and the temperature of the gas shock-heated within its potential well (T_{vir}) is derived from the Virial Theorem. By equating the thermal kinetic energy of the gas to the gravitational binding energy of the halo, we get the equation for virial temperature:

$$T_{\text{vir}} = \frac{\mu m_p G M_{\text{vir}}}{2 k_B R_{\text{vir}}} \approx 1.98 \times 10^4 \text{ K} \left(\frac{M}{10^8 M_\odot} \right)^{2/3}$$

Where:

- μ is the mean molecular weight of the gas (roughly **0.6** for fully ionized primordial gas).
- m_p is the mass of a proton.
- G is the gravitational constant.
- k_B is the Boltzmann constant.
- R_{vir} is the virial radius of the collapsed dark matter halo.

Because the physical radius of a halo (R_{vir}) is tied to its mass and the background density of the universe at the time of collapse, this equation is frequently parameterized to directly link temperature and mass. By scaling from standard cosmological limits at redshift zero, we can isolate the minimum halo mass (M_{min}) required to reach a specific temperature floor (T_{floor}):

$$M_{\text{min}} \approx 10^8 M_\odot \left(\frac{T_{\text{floor}}}{10^4 \text{ K}} \right)^{3/2}$$

To successfully trigger the chosen cooling curve and validate the fluid solver, the simulation's volume and mass resolution must be capable of forming halos of at least M_{min}.

Mass Resolution

Knowing the minimum halo mass (M_{min}) required to trigger our specific thermodynamics, we must now ensure the grid is fine enough to actually resolve it. In a grid-based solver utilizing Cloud-In-Cell (CIC) mass assignment or similar schemes, structures that are only one or two cells wide are heavily distorted by numerical diffusion

and grid noise. Standard computational practice dictates that a dark matter halo must span a minimum number of grid cells, N_{halo} (typically 64), to be considered physically resolved and dynamically stable.

Therefore, the maximum allowable mass resolution (m_{cell}, the mean mass of an unperturbed background cell) is constrained by:

$$m_{\text{cell}} \leq \frac{M_{\text{min}}}{N_{\text{halo}}}$$

The mass resolution can be defined also as a function of the total background comoving matter density of the simulated universe (ρ_m), the physical box size (L), and the 1D grid resolution (N):

$$m_{\text{cell}} = \rho_m \left(\frac{L}{N}\right)^3$$

By equating these two principles, we can obtain the maximum allowable physical width of a single grid cell ($\Delta x = L/N$):

$$\Delta x_{\text{max}} \leq \left(\frac{M_{\text{min}}}{N_{\text{halo}}\rho_m}\right)^{1/3}$$

The Fundamental Mode

To satisfy the Δx_{max} requirement, one might assume the easiest solution is to run a small simulation volume (L) at a low resolution (N). However, doing so may violate the boundaries of the expansion metric.

The longest gravitational wavelength a simulation can model is equal to the box size itself, known as the Fundamental Mode ($k = 2\pi/L$). If the amplitude of density fluctuations at this scale grows too large, the fundamental mode will undergo non-linear collapse, causing the periodic boundary conditions to fail and crushing the simulated universe.

In linear perturbation theory, non-linear collapse occurs when a region reaches a critical overdensity threshold of $\delta_c \approx 1.68$ (the origin of this value is explained later). To ensure the fundamental mode does not reach this threshold, we measure its global variance, $\sigma(L)$.

In simple terms, the variance represents the "typical" or "average" strength of the density ripples at the scale of the entire box. Because cosmic density fluctuations follow a standard Gaussian bell curve, some regions will inevitably be denser than this average. If we allow the typical ripple strength ($\sigma(L)$) to grow too close to the critical collapse limit of 1.68, there is a high probability that a random density fluctuation within the box will cross the threshold and trigger global collapse.

To ensure stability, we must keep the typical ripple so small that hitting 1.68 becomes a statistical impossibility. We do this by restricting the fundamental variance to a safety threshold:

$$\sigma_{\text{safe}} \leq 0.3$$

At a variance of 0.3, a density spike of 1.68 becomes almost six times larger than the average (5.6σ). This renders the probability of global collapse practically zero.

Deriving the Minimum Box Size (L_{min})

Because variance grows over time and scales inversely with physical size, setting a hard limit of $\sigma_{\text{safe}} = 0.3$ dictates the minimum allowable box size for a simulation.

We can approximate the variance at any given scale L (in Mpc) and any given scale factor a using the amplitude of primordial fluctuations (σ_8) and the linear growth factor ($D(a) \approx a$):

$$\sigma(L, a) \approx a \cdot \sigma_8 \left(\frac{8}{L}\right)^{0.9}$$

This formula is a power-law approximation of the ΛCDM matter power spectrum. To predict the variance at an arbitrary physical scale, we anchor the equation to two universally established cosmological standards: the linear growth factor ($D(a) \approx a$) to scale time, and σ_8 (the known present-day variance at exactly 8 Mpc) to scale space. Theoretical cosmology dictates that variance scales spatially according to the effective spectral index of the universe (n, not to be confused with the primordial spectral index; the effective index represents the local slope of the matter power spectrum after it was bent by early-universe radiation, evaluated specifically at the physical scale of the simulation box), following the relation $\sigma(R) \propto R^{-(n+3)/2}$. While the spectral index bends across vast cosmic distances, in the specific "sandbox" regime of small-scale simulations (roughly 1 to 20 Mpc), it is remarkably constant at $n \approx -1.2$. Plugging this local index into the theoretical relation yields exactly -0.9, giving us a rule to estimate variance for restricted box sizes.

To find the minimum allowable box size (L_{min}) that survives until the end of the simulation (a_{stop}), we set the variance equal to the safety threshold:

$$\sigma_{\text{safe}} = a_{\text{stop}} \cdot \sigma_8 \left(\frac{8}{L_{\text{min}}} \right)^{0.9}$$

By algebraically inverting this formula, we get the equation for the smallest volume we are safe to simulate:

$$L_{\text{min}} = 8 \left(\frac{a_{\text{stop}} \cdot \sigma_8}{\sigma_{\text{safe}}} \right)^{1/0.9}$$

For example: If we are running a standard cosmology ($\sigma_8 = 0.81$) and we want the simulation to safely reach $a = 0.5$ without exceeding the $\sigma_{safe} = 0.3$ limit, the formula dictates a minimum box size of roughly 11.1 *Mpc.*

Final Synthesis

Combining these independent limits tells us that the simulation must be at least as large as the global collapse limit (L_{min}), and its individual cells can be no larger than the required physical resolution limit (Δx_{max}).

By dividing the minimum volume by the maximum cell size, we arrive at the equation for the minimum required grid dimension (N):

$$N \geq \frac{L_{\text{min}}}{\Delta x_{\text{max}}}$$

Substituting our derived physics back into this equation yields the analytical requirement for any grid-based cosmological simulation:

$$N \geq L_{\text{min}} \left(\frac{N_{\text{halo}} \rho_m}{M_{\text{min}}} \right)^{1/3}$$

If a simulation is run at a lower resolution, the mass of a single cell artificially swells beyond M_{min}. Because a single grid cell becomes more massive than the smallest structures the code is attempting to resolve, the physics engine becomes blind to those environments. This disconnects the grid from the true hierarchical growth of the cosmos, resulting in numerical artifacts such as artificial overcooling.

Representative Sandbox Simulations

Running a fully resolved, science-grade simulation (e.g., a 100 Mpc volume at 1024^3 resolution) requires thousands of CPU hours on a supercomputing cluster. During the development phases of a cosmological code, waiting days to see if a new hydrodynamics module or gravity solver works is impractical.

Developers need a "sandbox"—a small, low-resolution simulation that runs in minutes on a workstation but still exhibits the qualitative tendencies of the universe, such as shock-heating, radiative cooling, and hierarchical structure formation.

However, as established before, ignoring the model constraints will lead to numerical artifacts. To make a simple simulation representative of true cosmological physics, we can engineer the sandbox using two approximations derived from Linear Growth Theory and the Truelove Criterion.

Limiting the Timeline

As explained before, if the box size (L) is too small relative to the simulated timeline, the longest gravitational wavelength the grid can model will go non-linear and collapse before the simulation reaches the present day ($a = 1.0$), crushing the background metric.

However, the amplitude of these fluctuations scales directly with the linear growth factor $D(a)$. Considering that in the early-to-mid universe, $D(a) \approx a$, we can derive a safe stopping epoch (a_{stop}) by restricting the variance to a strict safety threshold (σ_{safe}):

$$\sigma(L, a_{\text{stop}}) \approx a_{\text{stop}} \cdot \sigma(L, a = 1.0) \leq \sigma_{\text{safe}}$$

By intentionally halting the simulation at a_{stop}, we ensure the fundamental mode remains in the linear regime. Simultaneously, because smaller interior sub-structures ($R \ll L$) possess higher variances, they will have already crossed the $\delta_c \approx 1.68$ threshold and collapsed. This way the sandbox avoids global collapse while still triggering the shock-heating necessary to validate the hydrodynamics engine.

The Truelove Criterion

Another hurdle of a coarse sandbox is gravitational fragmentation. When testing a cooling solver, it is tempting to allow the gas to cool to its natural absolute limits (e.g., down to the limits of atomic physics or the cosmic microwave background). However, on a coarse grid, this triggers a numerical failure governed by the Jeans Length (λ_J).

The Jeans Length defines the physical size of a gas cloud where internal thermal pressure exactly balances gravitational collapse. It is proportional to the sound speed and inversely proportional to density:

$$\lambda_J = \sqrt{\frac{\pi \gamma k_B T}{\mu m_p G \rho}} \propto \sqrt{\frac{T}{\rho}}$$

In 1997, Truelove et al. established a fundamental rule for computational fluid dynamics: if a simulation allows the Jeans Length of a condensing gas cloud to drop below the width of a minimum number of computational grid cells (typically $N_J = 4$), the numerical solver can no longer accurately resolve the pressure gradient. When this happens, gravity artificially dominates, and the gas collapses into an infinitely dense, single-cell numerical singularity.

For a simulation with box size L and 1D resolution N, the physical size of a single computational cell at the stopping epoch a_{stop} is:

$$\Delta x_{\text{phys}} = a_{\text{stop}} \left(\frac{L}{N} \right)$$

To satisfy the Truelove Criterion, the gas must maintain enough thermal pressure so that its Jeans Length never drops below $4\Delta x_{\text{phys}}$, even at the maximum resolvable density (ρ_{max}). By rearranging the Jeans equation, we can calculate the absolute minimum temperature floor (T_{floor}) required to prevent numerical fragmentation:

$$T_{\text{floor}} \geq \frac{\mu m_p G \rho_{\text{max}}}{\pi \gamma k_B} \left(4\Delta x_{\text{phys}}\right)^2$$

Because the required temperature scales with the square of the physical cell size, coarse grids demand massive thermal buffers. By setting the cooling floor configuration parameter to T_{floor}, we provide just enough artificial thermal pressure to ensure the densest gas clouds remain larger than the coarse grid cells, effectively bypassing the Truelove limit without crashing the Riemann solver.

The Spherical Collapse Limit

The value 1.68 (more precisely, 1.686) is a universal threshold derived from a classic analytical framework known as the Spherical Collapse Model. This model tracks a spherical region of the universe that is slightly denser than the background. As the universe expands, this denser sphere also expands, but its extra gravitational pull slows its expansion down. Eventually, it reaches a “turnaround” point and collapses back in on itself to form a dark matter halo.

To calculate this, cosmologists treat the surface of the sphere like a ball thrown straight upward against gravity. Its trajectory (radius and time) follows a parametric curve called a cycloid (think of the trajectory of a fixed point of a spinning wheel), driven by a "development angle" (θ):

- $\theta = 0$**:** The Big Bang (expansion begins).
- $\theta = \pi$**:** The turnaround point (expansion halts, collapse begins).
- $\theta = 2\pi$**:** The singularity (the sphere completely crushes itself).

According to the cycloid equations, the amount of time that passes is tied to this angle via the relation $t \propto (\theta - \sin\theta)$. Therefore, the moment of physical collapse occurs at a time proportional to 2π.

However, alongside this exact model, cosmologists run a parallel **Linear Perturbation Theory** model. Linear theory is a simplified mathematical approach that assumes gravity never goes non-linear, allowing the density fluctuation (δ) to grow smoothly and continuously without ever curving back in on itself.

To find the theoretical "finish line" for a simulation, we map the true physical collapse onto the simplified linear timeline. We do this by taking a Taylor expansion of the cycloid equations in the extreme early universe ($\theta \ll 1$), where gravity is still linear:

- The Time Equation: The Taylor expansion of $\sin\theta$ is approximately $\theta - \theta^3/6$. When we plug this into the time equation ($t \propto \theta - \sin\theta$), the leading θ terms cancel out, leaving $t \propto \theta^3/6$. By inverting this, we find that the development angle grows relative to time as $\theta \propto (6t)^{1/3}$.
- The Density Equation: By similarly expanding the cycloid's radius equation ($R \propto 1 - \cos\theta$) using the Taylor series for cosine, we can isolate the leading-order growth mode. The linear density contrast becomes $\delta_{\text{linear}} = \frac{3}{20}\theta^2$.

By substituting our time equation into our density equation, we get the linear growth formula:

$$\delta_{\text{linear}} = \frac{3}{20}(6t)^{2/3}$$

To find the critical overdensity threshold, δ_c, we simply fast-forward this linear equation to the time we know the physical sphere crushes to a singularity ($t = 2\pi$):

$$\delta_c = \frac{3}{20}(6 \times 2\pi)^{2/3} = \frac{3}{20}(12\pi)^{2/3} \approx 1.686$$

Therefore, we know that if the linear equations dictate a density wave has reached an amplitude of 1.68, the wave has collapsed.

Key Literature & Further Reading

Truelove, J. K., Klein, R. I., McKee, C. F., Holliman II, J. H., Howell, L. H., & Greenough, J. A. (1997). *The Jeans condition: a new constraint on spatial resolution in simulations of isothermal self-gravitational hydrodynamics.* The Astrophysical Journal Letters, 489(2), L179. Available at https://iopscience.iop.org/article/10.1086/310975

Barkana, R., & Loeb, A. (2001). *In the beginning: the first sources of light and the reionization of the universe.* Physics Reports, 349(2), 125-238. Available at https://arxiv.org/abs/astro-ph/0010468

Architectures of Cosmological Simulations

Because computational power is finite, it is physically impossible to simulate the entire visible universe at the resolution required to resolve individual star systems or even individual molecular clouds. Cosmological simulation is fundamentally an exercise in managing the "computational trade-off" between volume and resolution.

To answer different distinct physical questions, cosmologists have developed several distinct architectures, or strategies, for setting up their simulation domains. These architectures dictate how the initial conditions are generated and how computational resources are allocated across the grid.

Large-Volume "Uniform Box" Simulations

This is the standard architecture we have been assuming throughout this text. In a uniform box simulation, a massive cubic volume of space (often hundreds of Megaparsecs across) is simulated at a consistent, uniform mass and spatial resolution throughout the entire domain.

- **The Goal:** These simulations are designed to capture the statistical properties of the large-scale structure. They are the ideal tool for studying the cosmic web, determining the mass function of galaxy clusters, measuring the sizes of voids, and testing how different dark energy models alter the expansion history of the universe.
- **The Trade-off:** Because the simulated volume is so massive, the computational resolution is relatively coarse. A uniform box simulation can reliably predict *where* a dark matter halo will form and how much mass it contains, but it lacks the resolution to model the intricate, internal physics of the galaxy residing inside it, such as the structure of its spiral arms.
- **Notable Examples:** The Millennium Simulation, IllustrisTNG, EAGLE.

Zoom-in Simulations

If uniform boxes provide the macro-view of the universe, zoom-in simulations are the cosmic microscope. They allow developers to dedicate nearly all of their computational resources to a single, highly resolved object—such as a Milky Way-sized galaxy—while still embedding it within a true cosmological environment.

Creating a zoom-in simulation requires a multi-step computational technique:

1. **The Base Run:** First, the developer runs a fast, low-resolution, dark matter-only simulation of a large uniform volume.
2. **Target Selection:** Once the run reaches $z = 0$, the developer searches the data to find a specific, interesting structure (for example, an isolated halo with a mass of $10^{12} M_{\odot}$).
3. **Tracing the Lagrangian Region:** The code identifies every particle that ended up inside that target halo and traces them backward in time to their exact starting positions in the initial conditions (e.g., at $z = 100$). This specific patch of the initial grid is called the target's *Lagrangian region*.
4. **Refining the Initial Conditions:** The initial conditions generator is run a second time. However, this time, high-frequency density waves (fine-grained details) are injected *only* into that specific Lagrangian region. The rest of the box is left at low resolution.
5. **The Final Run:** The simulation is re-run. The high-resolution particles collapse into a highly detailed galaxy in the center of the box, while the massive, low-resolution particles on the outskirts simply act as boundary conditions, providing the correct long-range gravitational tidal forces.

- **The Goal:** To study the intricate, internal physics of individual galaxies, such as supernova feedback, supermassive black hole accretion, and the formation of galactic disks.
- **The Trade-off:** Because they are intensely expensive, they lack statistical power. A zoom-in simulation usually models only one or a handful of galaxies. If the chosen target happened to be a statistical outlier, it might lead to incorrect assumptions about general galaxy formation.
- **Notable Examples:** The FIRE (Feedback In Realistic Environments) project, the Auriga simulations.

Constrained Simulations

In standard uniform boxes and zoom-in simulations, the initial conditions are generated using a random mathematical seed. This creates a statistically valid but completely random, generic patch of space. Constrained simulations, however, aim to simulate our exact physical neighborhood.

Using advanced mathematical techniques (such as Wiener filtering), cosmologists run complex algorithms backwards to reverse-engineer the initial density field based on actual 3D telescopic surveys of the real sky.

- **The Goal:** To force the primordial density ripples to collapse into exact replicas of the Milky Way, Andromeda, the Virgo Cluster, and the Local Void, in the exact coordinates where we observe them today. This allows cosmologists to test structure formation theories against the most well-observed region of the universe.
- **The Trade-off:** The reverse-engineering of the density field is mathematically grueling, heavily dependent on the accuracy of observational data, and is currently only viable for the relatively local universe.
- **Notable Examples:** The CLUES project (Constrained Local UniversE Simulations), SIBELIUS.

The Physics Split: Gravity-Only vs. Hydrodynamics

Beyond spatial architecture, simulations are also categorized by the physical forces they compute.

Dark Matter-Only (N-body) simulations completely ignore baryonic gas, stars, and radiation. Everything is treated as collisionless particles governed purely by the Poisson solver and the expansion of space. Because they are computationally inexpensive, they can simulate enormous Gigaparsec-scale volumes, making them the primary tool for testing pure cosmology and General Relativity.

Hydrodynamical simulations introduce the complex fluid dynamics of normal matter. By integrating the Euler equations alongside gravity, these codes can model shock waves, the heating and cooling of gas, and "sub-grid" physics like star formation. While they are required to understand what the universe actually "looks" like to telescopes, adding hydrodynamics increases the computational cost by orders of magnitude. For this reason, hydrodynamics are most frequently paired with Zoom-in architectures or smaller uniform boxes.

Diagnosing Cosmological Simulations

A cosmological simulation generates billions of data points at every time step. Validating the accuracy of such an immense system requires aggregating this raw data into global macroscopic metrics.

This chapter outlines the standard diagnostic plots used to evaluate the health of a cosmological simulation, detailing the underlying physics and the expected theoretical behaviors.

Cosmic Expansion History

The most basic cross-check of a cosmological simulation is verifying its background geometry. Because the simulation box represents a comoving volume of the universe, its physical size must expand according to the Friedmann equations.

To validate this, the scale factor a is plotted against the physical simulation time (typically in Gigayears). The shape of this curve is dictated entirely by the cosmological parameters chosen for the run, specifically the matter density parameter (Ω_m) and the dark energy density parameter (Ω_Λ).

In a matter-dominated universe, the expansion decelerates over time. However, in a standard ΛCDM cosmology ($\Omega_\Lambda > 0$), dark energy eventually dominates. On the plot, this manifests as a curve that initially decelerates but gradually bends upward at late times (around $a \approx 0.5$ to 1.0), reflecting the accelerated expansion of the universe.

Structure Growth and Linear Theory

Once the background expansion is validated, the next step is verifying the behavior of gravity. We must confirm that dark matter is clumping together at the correct rate. This can be tracked by plotting the **Dark Matter Density Variance** (σ^2) against the scale factor.

The Physics of Density Variance

In cosmology, we do not usually measure absolute density; instead, we measure the density contrast, δ, which defines how much denser or emptier a specific region is compared to the cosmic mean density $\bar{\rho}$:

$$\delta(\mathbf{x}) = \frac{\rho(\mathbf{x}) - \bar{\rho}}{\bar{\rho}}$$

To calculate the variance of the entire simulation, the continuous distribution of dark matter mass is mapped onto a uniform spatial grid. For every discrete volume element in the simulation, the local density contrast δ is calculated. The variance σ^2 is simply the mean of the squared density contrasts across the entire volume:

$$\sigma^2 = \langle \delta^2 \rangle$$

This single number (σ^2) represents the global "clumpiness" of the universe. A completely smooth universe has a variance of zero.

Linear Perturbation Theory

To know if the simulated variance is correct, it is compared against Cosmological Linear Perturbation Theory. In the very early universe, density fluctuations are microscopic ($\delta \ll 1$). Under these conditions, the equations of dark matter can be linearized. Theory dictates that these small perturbations grow uniformly according to a Linear Growth Factor, $D(a)$:

$$\delta(\mathbf{x}, a) = \delta_0(\mathbf{x}) \frac{D(a)}{D(a_0)}$$

In a purely matter-dominated universe (an Einstein-de Sitter cosmology), the growth factor is directly proportional to the scale factor ($D(a) \propto a$), meaning the variance simply grows with the square of the scale factor ($\sigma^2 \propto a^2$).

However, in a ΛCDM universe, the accelerated stretching of spacetime fights against the gravitational pull of the dark matter halos, causing the rate of structure growth to slow down.

To validate the numerical solver in both cases, the theoretical model for the variance curve is driven by the integral of the expanding Hubble metric $H(a)$:

$$D(a) \propto H(a) \int_0^a \frac{da'}{(a'H(a'))^3}$$

With:

$$H(a) = H_0 \sqrt{\Omega_{m,0} a^{-3} + \Omega_{\Lambda,0}}$$

Usually approximated as:

$$D(a) \propto \frac{5\Omega_m(a)}{2} \left[\Omega_m(a)^{4/7} - \Omega_\Lambda(a) + \left(1 + \frac{\Omega_m(a)}{2}\right)\left(1 + \frac{\Omega_\Lambda(a)}{70}\right) \right]^{-1} \cdot a$$

By plotting this growth curve against the discrete numerical variance calculated from the grid, we can prove that the N-body gravity solver is capturing both the early epoch of rapid hierarchical assembly and the late-stage suppression of structure formation caused by Dark Energy.

Expected Simulation Behavior

- **The Linear Regime:** At early times, the simulated variance must track the theoretical line. The initial smooth distribution of matter gently amplifies exactly as linear equations predict.
- **The Non-Linear Regime:** At later times (typically around $a > 0.4$), the simulated variance will curve upward, breaking away from linear theory. As dense clumps form ($\delta > 1$), the linear approximation fails. Gravity becomes highly localized and non-linear, pulling matter into dark matter halos much faster than the background linear theory suggests.

The 1-Point Density PDF

While the variance provides a single number for global structure growth, the **1-Point Density Probability Distribution Function (PDF)** provides a complete statistical picture of the matter distribution. Most commonly plotted as a **Volume-Weighted PDF**, this histogram shows the volume fraction of the universe occupied by gas at various overdensities ($\rho/\bar{\rho}$).

By plotting the simulation's PDF at different evolutionary stages (e.g., $a = 0.1$, $a = 0.5$, $a = 1.0$) against analytical models, we can verify the migration of mass.

The Early Universe: The Gaussian Model

In the linear regime of the early universe, primordial density fluctuations are symmetrical. The probability $P(\delta)$ of finding a region with a specific density contrast δ follows a classic Gaussian distribution, dictated by the global variance σ^2:

$$P(\delta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{\delta^2}{2\sigma^2}\right)$$

On a plot, the early-universe PDF will resemble a very narrow, tall bell curve centered exactly at $\rho/\bar{\rho} = 1$. Matching this theoretical curve proves the initial conditions and background metric are stable.

The Late Universe: The Lognormal Model

As gravity drives the universe into the non-linear regime, the symmetry of the Gaussian curve breaks. Mass evacuates from cosmic voids (which are strictly bounded, as density cannot drop below zero) and compresses into halos (which have no upper density limit). The peak of the PDF shifts slightly to the left, representing the massive volume of empty voids, while the right side develops a massive "tail" stretching into extreme overdensities.

Cosmological theory dictates that this asymmetric, late-stage density field is well approximated by a Lognormal distribution. If we define a new variable $A = \ln(1 + \delta)$, the lognormal PDF is given by:

$$P(A) = \frac{1}{\sqrt{2\pi\sigma_A^2}} \exp\left(-\frac{(A - \langle A \rangle)^2}{2\sigma_A^2}\right)$$

This transition from a narrow Gaussian to a broad, lognormal distribution is the mathematical signature of the cosmic web forming. By comparing these analytical models to the simulation's dynamic histogram, we can verify that the code's non-linear mass transport adheres to theoretical expectations.

The Matter Power Spectrum

The global density variance (σ^2) tells us how "clumpy" the universe is as a single number, but it cannot tell us the *sizes* of those clumps. A universe filled with tiny, dense galaxies could produce the same variance as a universe filled with a few massive superclusters.

To evaluate the structural health of the simulation across all spatial scales, we must decompose the density field into its constituent frequencies using a Fourier Transform. The resulting metric is the **Matter Power Spectrum**, $P(k)$.

The Physics of the Power Spectrum

Instead of looking at the density contrast $\delta(\mathbf{x})$ in physical space, we transform it into k-space (Fourier space) to get $\delta(\mathbf{k})$. The variable k is the **wavenumber**, which is inversely proportional to the physical size of a structure (λ):

$$k = \frac{2\pi}{\lambda}$$

Low values of k represent massive, large-scale structures (like voids and superclusters), while high values of k represent small-scale structures (like individual galaxy halos). The Power Spectrum $P(k)$ is defined as the variance of the density amplitudes at a specific wavenumber:

$$P(k) = V\langle|\delta(\mathbf{k})|^2\rangle$$

where V is the volume of the simulation box. On a conventional plot, the x-axis represents the wavenumber k (typically in units of h Mpc^{-1}), and the y-axis represents the power $P(k)$ (in units of $(h^{-1}\text{Mpc})^3$). Because both the scales and the amplitudes vary by many orders of magnitude, this is always plotted on a log-log scale.

Computing the Power Spectrum from the Simulation

In our simulation the dark matter exists as discrete particles, while the gas is mapped onto a fixed Eulerian grid. To compute the global density variance in Fourier space, we must project the particles onto the gas grid, combining them into a unified density field. The pipeline consists of four steps:

1. Mass Assignment (Cloud-In-Cell) First, the dark matter mass must be distributed to the stationary grid cells. To do this, we can use the Cloud-In-Cell (CIC) technique, explained in a previous chapter. At the end of this step, we obtain the dark matter density field, $\rho_{dm}(\mathbf{x})$.

2. The Unified Density Contrast With the dark matter mapped to the grid, the total mass density in any given cell is the sum of the dark matter and gas densities: $\rho(\mathbf{x}) = \rho_{dm}(\mathbf{x}) + \rho_{gas}(\mathbf{x})$.

We then calculate the dimensionless density contrast for the unified grid:

$$\delta(\mathbf{x}) = \frac{\rho(\mathbf{x}) - \bar{\rho}}{\bar{\rho}}$$

where $\bar{\rho}$ is the mean density of the entire simulation box.

3. Fourier Transform and Deconvolution We pass the grid of $\delta(\mathbf{x})$ through a Fast Fourier Transform to obtain the complex field $\delta(\mathbf{k})$. Then we need to "un-smear" the raw output by applying the CIC deconvolution.

$$\delta_{\text{true}}(\mathbf{k}) = \frac{\delta_{\text{grid}}(\mathbf{k})}{W(\mathbf{k})}$$

$$W(\mathbf{k}) = \left[\frac{\sin(\frac{k_x \Delta x}{2})}{\frac{k_x \Delta x}{2}}\right]^2 \left[\frac{\sin(\frac{k_y \Delta x}{2})}{\frac{k_y \Delta x}{2}}\right]^2 \left[\frac{\sin(\frac{k_z \Delta x}{2})}{\frac{k_z \Delta x}{2}}\right]^2$$

4. Spherical Averaging (Binning) To generate the final 1D Power Spectrum plot, we must collapse the Fourier grid down to a single line. First, we calculate the squared magnitude $|\delta_{\text{true}}(\mathbf{k})|^2$ for every cell in the Fourier grid.

Because the simulated universe is isotropic (statistically identical in all directions), we do not care about the orientation of a given density wave, only its physical scale. For each cell, we calculate its scalar magnitude $k = \sqrt{k_x^2 + k_y^2 + k_z^2}$, which represents its distance from the origin in Fourier space.

We then define the x-axis of the final plot by dividing this continuous 3D space into discrete "bins" (effectively carving Fourier space into concentric spherical shells of increasing radius). By collecting all the grid cells that fall within the boundaries of a given shell and averaging their squared amplitudes together, we isolate the variance at that specific physical scale. Multiplying this average by the volume of the simulation box yields the final 1D power spectrum value, $P(k)$.

Theoretical Comparison

We can test our simulation by plotting the data directly against established theoretical models. A standard practice is to generate these theoretical curves using accurate Boltzmann solvers, such as CAMB (Code for Anisotropies in the Microwave Background).

CAMB is an industry-standard "Einstein-Boltzmann solver". Instead of moving particles, it solves the exact, continuous differential equations of General Relativity and thermodynamics that govern the universe. When provided with a set of cosmological parameters (such as Ω_m, Ω_b, and the Hubble constant), CAMB calculates how the primordial quantum fluctuations from the Big Bang evolved over billions of years.

It does this by simultaneously solving the Einstein field equations (which dictate how gravity expands and bends space) alongside the Boltzmann transport equations (which track how dark matter, baryonic gas, photons, and neutrinos interact, scatter, and push against each other). By tracking these continuous fluids as the universe cools, CAMB calculates the theoretical power spectra for the large-scale distribution of matter.

When diagnosing the simulation against these theoretical curves, we look at two distinct regimes:

1. **The Linear Regime (Low k):** On large spatial scales, gravity is relatively weak, and structures collapse slowly. In this regime, the shape of the power spectrum is preserved, and its amplitude simply scales upwards over time proportional to the square of the linear growth factor, $D^2(a)$. If the simulation overlaps the theoretical curves at low k, we know our Initial Conditions (such as the σ_8 normalization and the unwinding of the dark energy growth suppression) were generated correctly, and our gravity solver is advancing at the correct cosmological rate.
2. **The Non-Linear Regime (High k):** On small spatial scales, density perturbations become so severe that gravity causes them to collapse violently into dense halos. This causes the power spectrum to surge upward, breaking away from linear theory. Because pure Boltzmann solvers are built on linear equations, they cannot naturally calculate this violent collapse. To bridge this gap, CAMB internally applies an algorithm called Halofit—a highly calibrated set of mathematical fitting formulas derived from massive supercomputer simulations.

A standard practice is to plot the power spectrum from multiple simulation snapshots on the same axes. By observing the curve shift upwards as the scale factor a increases, we can track the timeline of the universe and verify that the simulation's large-scale structures are evolving in step with the theoretical linear growth rate.

Global Gas Energy Inventory

To ensure the fluid solver is stable and accurately capturing energy transformations, we can plot a global energy inventory over time. This displays the total Kinetic Energy, total Thermal Energy, and the cumulative Radiated Energy of the gas. We can also plot the Fractional Energy Error to prove that the Riemann solver is obeying the First Law of Thermodynamics.

The Physics of Energy Conversion

As the universe evolves, the gravitational potential wells created by dark matter halos begin to pull the surrounding gas inward. As the gas accelerates toward the center of these halos, it gains immense velocity, causing an increase in the global Kinetic Energy of the simulation.

However, unlike dark matter, gas is collisional. When flows of gas from opposite sides of a halo converge in the center, they violently collide. These supersonic collisions create massive shockwaves that convert the ordered kinetic energy of the infalling gas into disordered thermal energy.

Once the gas is shock-heated and densely packed in the center of these halos, it begins to emit photons (primarily via Bremsstrahlung radiation), allowing energy to escape the simulation volume.

The Fractional Energy Error

The Fractional Energy Error is a diagnostic metric that reveals whether the numerical Riemann solver is artificially creating or destroying energy. In a static physics simulation, proving energy conservation is as easy as checking if the final energy matches the initial energy. However, because our cosmological box is expanding and gravity is constantly doing work, the total energy of the gas is *supposed* to change.

To calculate the true numerical error, the simulation must maintain a running ledger. At every single time step, the engine integrates the amount of energy added by gravitational work (W_{grav}), the energy drained by cosmic expansion work (W_{exp}), and the thermal energy lost to radiation (E_{rad}).

$$W_{\text{grav}} = \int \left(\int \rho \mathbf{v} \cdot \mathbf{g} \, dV \right) dt$$

$$W_{\text{exp}} = \int \left(\int 3\frac{\dot{a}}{a} P \, dV \right) dt$$

By evaluating these values using comoving code units (to strip away the diluting effects of the expanding metric), we can isolate the fluid solver's numerical error:

$$\text{Absolute Error} = \Delta E_{\text{tot}} - (W_{\text{grav}} - W_{\text{exp}} - E_{\text{rad}})$$

Dividing this absolute error by the initial total energy of the system provides the unitless Fractional Energy Error. If the fluid equations are solved correctly, this ledger balances to zero.

Expected Simulation Behavior

On an energy evolution plot, these processes tell a chronological story:

- **The Adiabatic Expansion Phase:** In the early universe, before structures form, the Thermal Energy curve will initially drop. This is the cosmological PdV work: the expansion of the universe forces the primordial gas to expand and cool down adiabatically (losing thermal energy because its volume is increasing, rather than radiating heat away).
- **The Infall Phase:** As gravity begins to win against the expansion, gas falls into emerging potential wells and the Kinetic Energy curve steadily rises.
- **The Shock Phase:** As the first structures collapse, the Kinetic Energy curve plateaus or begins to drop. Simultaneously, the Thermal Energy curve shoots upward. This exchange between the two curves shows the hydrodynamics engine converting kinetic energy into thermal heat during supersonic shocks.
- **The Cooling Phase:** As the gas reaches extreme temperatures and densities, the cumulative Radiated Energy curve begins to rapidly climb. As billions of Ergs are radiated away, the gas loses its thermal pressure support, allowing it to condense into the cold, dense cores necessary for star formation.
- **The Conservation Baseline:** If the Riemann solver and KDK integrators are healthy, Fractional Energy Error line will remain nearly flat at zero (typically within a tolerance of 10^{-4} or 10^{-3}) across billions of years of evolution.

Maximum Gas Density Evolution

To understand how tightly the baryonic matter is compressing, we can track the densest gas cell in the simulation over time. This metric exposes the ongoing battle between gravitational collapse and thermal pressure.

Tracking Hydrogen Number Density

Instead of plotting the raw bulk mass density of the fluid (ρ), it is the industry standard to track the **Hydrogen number density** (n_H), measured in particles per cubic centimeter (cm^{-3}). There are two main reasons for this:

1. **Composition and Collisions:** The primordial universe is roughly 76% hydrogen by mass. The physical processes that drive galaxy evolution—such as radiative cooling and star formation—depend on how frequently individual particles collide, which is dictated by number density, not just overall mass.
2. **Observational Parity:** Astronomers cannot easily "weigh" a distant gas cloud, but they can count hydrogen atoms by measuring the light absorbed or emitted by them. Plotting n_H allows direct comparisons between our simulated universe and telescope data.

To calculate this metric from the simulation data, we must convert our internal code variables into physical units. We first divide the comoving mass density by the expansion factor cubed ($\mathbf{a^3}$) to find the true physical density. We then multiply by the primordial hydrogen mass fraction ($X_H = 0.76$) to isolate the hydrogen mass, and finally divide by the mass of a single proton to count the exact number of atoms per unit volume.

Expected Simulation Behavior

As gas falls into a dark matter halo, gravity forces it into an increasingly smaller volume. However, as the gas shock-heats, its thermal pressure increases, pushing back against the gravitational collapse.

- **The Expansion-Dominated Phase:** In the very early universe, the maximum physical density curve will drop. During this linear regime, the expansion of the universe outpaces the slow gravitational assembly of the initial dark matter clumps, causing the gas to dilute.
- **Turnaround and Collapse:** As dark matter halos grow massive enough to overcome the background Hubble expansion—a threshold known as "turnaround"—the gas is aggressively pulled into these deepening potential wells. At this point, the density curve reverses course and begins to rise steeply, mirroring the non-linear structure growth.
- **Late Epochs and Core Collapse:** In a healthy, fully featured simulation (with active radiative cooling and high spatial resolution), this curve will continue to climb dramatically. As the gas radiates away its shock-heated thermal energy, it loses outward pressure support and collapses deeply into the halo cores, achieving the extreme densities necessary to trigger star formation. *Conversely*, in a purely adiabatic simulation (without cooling) or one limited by coarse grid resolution, this curve will prematurely flatten out and hit an artificial ceiling. In those restricted scenarios, the unabated thermal pressure prevents the gas from compressing any further than a few grid cells across.

Maximum Temperature Evolution

Tracking the maximum temperature—the single hottest cell in the universe at any given time—helps verifying the simulation's shock-capturing capabilities.

The Physics of Shock Heating

In the void regions of the universe, gas naturally cools due to adiabatic expansion ($T \propto a^{-2}$). However, inside halos, the gravitational potential is so deep that infalling gas reaches velocities far exceeding the local speed of sound. When this gas collides, the resulting shockwaves heat the medium to the **virial temperature** of the halo (the temperature a cloud of gas naturally reaches when it falls into a gravitational well and its inward fall is violently halted by collisions, transforming all that gravitational energy into heat), which scales with the halo's mass. Massive galaxy clusters can possess deep enough gravity wells to heat gas to temperatures exceeding 10^7 K.

Expected Simulation Behavior

- **The First Light (First Shocks):** At the beginning of the simulation, the maximum temperature remains low. Suddenly, when the very first non-linear structures collapse, the plot will show a violent, near-vertical spike. The maximum temperature jumps from a few tens of Kelvin to hundreds of thousands or millions of Kelvin in a fraction of a Gigayear.
- **Virial Equilibrium:** After the initial spike, the curve levels off, forming a stable, slightly rising plateau. This plateau corresponds to the virial temperature of the most massive dark matter halo that has formed within the simulation box.

The Temperature-Density Phase Diagram

The **Temperature-Density Phase Diagram** maps the thermodynamic state of every single gas cell in the universe. Plotted as a 2D histogram (often with logarithmic scales), the x-axis represents the gas overdensity ($\rho/\bar{\rho}$), and the y-axis represents the temperature. The position of gas in this phase space reveals which physical processes are dominating its behavior.

The Physics of the Phase Space

Gas in the universe naturally segregates into distinct thermodynamic regimes based on its environment:

- **The Diffuse Intergalactic Medium (IGM):** In the extremely low-density voids, gas simply expands with the universe. Because it is doing work as it expands, it cools adiabatically. In the phase diagram, this gas forms a tight, diagonal line at low densities and low temperatures, defined by the relationship $T \propto \rho^{\gamma-1}$ (the **Adiabatic Track**).
- **The Warm-Hot Intergalactic Medium (WHIM):** As gas falls into filaments and dark matter halos, it compresses and shocks. This shock-heated gas breaks away from the adiabatic track, scattering upward into a broad cloud of high-temperature, moderate-density material.
- **The Condensed Cores:** If the simulation includes micro-physics like Bremsstrahlung (free-free) or line cooling, gas that reaches high densities can radiate its thermal energy away as photons. Because the cooling rate scales with the square of the density ($\Lambda \propto \rho^2$), cooling becomes fiercely efficient in the deepest parts of the dark matter halos.

Expected Simulation Behavior

- **In an Adiabatic Simulation:** Without radiative cooling, gas that gets shocked into the high-temperature regime stays hot. As it compresses into halos, it moves to the right on the phase diagram (higher density) but remains in a thick, hot plateau above the adiabatic track. Its thermal pressure eventually physically halts further compression.
- **In a Radiative Cooling Simulation:** When cooling physics is enabled (and spatial resolution is high enough to allow dense cores to form), a dramatic structural shift occurs. At high overdensities, the cooling time of the gas drops below the age of the universe. The hot gas loses its pressure support and plummets downward on the graph, forming a vertical "cooling waterfall." This gas pools at the bottom right of the phase diagram, hitting the artificial temperature floor (often set around **10,000 K** for primordial cooling, or lower if molecular cooling is simulated).

Cold Dense Gas Fraction (The Star Formation Precursor)

Cosmological simulations are fundamentally attempting to explain galaxy formation. However, individual stars are too small to simulate in a box that is millions of parsecs wide. Instead, astrophysicists use sub-grid models to spawn "star particles" out of gas that is physically ready to condense. The **Cold Dense Gas Fraction** tracks the total mass of this eligible raw material over time.

The Physics of Condensation

For a cloud of gas to collapse and form stars, it must overcome its own internal thermal pressure. This requires two specific conditions to be met simultaneously:

1. **High Density:** The gas must be deep inside a gravitational potential well (typically an overdensity $\delta > 100$), ensuring gravity is strong enough to pull it together.
2. **Low Temperature:** The gas must have successfully radiated away its shock-heated thermal energy (typically dropping below **10,000 K**), sapping the outward pressure that would otherwise resist collapse.

Expected Simulation Behavior

Tracking the mass fraction of gas that meets these two exact criteria serves as the ultimate benchmark for the simulation's thermodynamic pipeline.

- **Pure Hydrodynamics:** In a simulation without radiative cooling, or one crippled by low resolution, this curve remains entirely flat at zero. The gas never simultaneously achieves high density and low temperature.
- **Complete Physics:** In a high-resolution simulation with active cooling, this curve will remain at zero during the early epochs. However, shortly after the first halos collapse and the cooling "waterfall" triggers in the phase diagram, this line will rapidly spike upward. This rising curve represents the exact mass of

gas that has decoupled from the hot halo and is successfully condensing into proto-galactic disks, acting as the immediate fuel source for the universe's first stars.